The Latent Cognition Trilogy



Paulo Leocadio [1]

[1]Zinnia Labs, Broward County, FL, United States

The Latent Cognition Trilogy

DOI: 10.13140/RG.2.2.29958.74561
(was: 10.13140/RG.2.2.32449.11363)

Paulo H. Leocadio

Researcher / Zinnis Labs

July 31, 2026

Published by Zinnia Labs Publishing
A research imprint of Zinnia Holdings LLC Coral Springs, Florida, USA
zinnialabs.science

The Latent Cognition Trilogy
Copyright © 2026 by Paulo H. Leocadio.

All rights reserved. No part of this publication may be reproduced, distributed, or transmitted in any form or by any means, including photocopying, recording, or other electronic or mechanical methods, without the prior written permission of the publisher, except in the case of brief quotations embodied in critical reviews and certain other noncommercial uses permitted by copyright law.

First Edition: 2026

ISBN Placeholders:
Hardcover: 978-0-000000-00-0
Paperback: 978-0-000000-01-7
E-book: 978-0-000000-02-4

Publisher Information:
Zinnia Labs Publishing
Coral Springs, FL, 33076
Website: https://zinnialabs.science

Printed in the United States of America.

Edition Status
Release Candidate 1 (RC1) June 2026

This edition represents the first complete architectural formulation of the Latent Cognition research program.

Future editions may incorporate additional theoretical, mathematical, and empirical findings.

For those who question the status quo,
and for the editors who make our words better.

Acknowledgments

I would like to express my gratitude to everyone who helped bring this book to life. Special thanks to my editors, mentors, and the entire team at Publisher Name for their invaluable feedback, patience, and guidance throughout this writing process.

Contents

Preface **iii**

Prologue **v**

The Latent Cognition Trilogy **vi**
Contextual overview vi
Introduction vii
Contributions viii
Overview of the Paper viii
Architectural Framework and Methodology ix
Architectural Foundations ix
Latent Memory Representation x
Retrieval and Routing Mechanism x
Mathematical Interpretation xii
Algorithmic Retrieval Process xiii
Conceptual Implementation Framework xiv
Comparative Architecture Analysis xv
Limitations and Theoretical Scope xvi
Predicted Architectural Behaviors xvi
Predicted Retrieval and Memory Behaviors xvii
Predicted Failure Modes and Architectural Risks xviii
C xviii
Discussion xviii
Retrieval Integration and Memory Coordination xix
Learning Dynamics and Adaptation xix
Bias, Safety, and Retrieval Integrity xix
Routing and Architectural Control xx
Broader Architectural Implications xx

Closing thoughts xxi

Note on the Research Program **xxiii**

Canonical Structure of the Research Program **xxvii**

Conceptual Foundations **xxxi**

Introduction to the Trilogy **xli**
Purpose of the Trilogy xli
Volume I: Latent Memory and Retrieval-Native Cognition xli
Volume II: Symbolic Stabilization under Compression xlii
Volume III: Hierarchical Routing and Architectural Coordination . xlii
Relation to Companion Works xlii
Scope and Limits xlii

Volume I: Memory Fields **1**

Abstract **2**
Overview of the Paper 6
The Unified Cognitive Operator Field 6
Mathematical Formalization of the Memory Operator M . . . 7
Boundary Interface: $M \longrightarrow S \longrightarrow R$ 7
Epistemological Framework and Empirical Falsification 8
Proposition I: The Field Collapse Invariant 9
Proposition II: Boundary Transition Degeneracy 9
Proposition III: Topological Conservation and Scaling Limits 10
Theoretical Construction and Implications 10
Hypothesis Decomposition 10
Architectural Modeling 11
Comparative Contextualization 11
Theoretical Predictions 12
Mathematical Framing of Differentiable Latent Indexing 14
Latent Key–Value Memory 14
Query Projection 15
Similarity and Sparse Retrieval 15
Retrieved Representation 15
Gated Fusion 16
Training Objective 16
Gradient Flow and Optimization Continuity 16
Memory Construction 18

Quantized Latent Indexing Mechanics 18
Layer Integration 19
Training and Fine-Tuning 19
Inference Behavior 19
Evaluation Plan 19
Comparative Literature Analysis 19
Discussion 20
Cognitive Implications 21
Learning Dynamics and Adaptation 21
Bias, Safety, and Privacy Considerations 21
Architectural Control and Gating Mechanisms 21
Toward Cognitive-Complete Transformer Architectures 22

Volume I Conclusion **23**

Volume II: Symbol Fields **24**

Symbol Fields and Curriculum-Induced Symbol Invention **27**
Introduction 27
Problem Space: From Continuous Representation to Symbolic Stability 28
Contributions 28
Overview of the Volume 29

Related Work **30**
Discrete Latent Representations 30
Information Bottleneck and Compression 30
Curriculum Learning and Compositional Generalization 31
Emergent Communication and Latent Semiotics 31
Probing and Interpretability 31

Propositions and Falsification Conditions **33**

The Informational Bottleneck and Vector Quantization **35**
Vector Quantization Mechanics 35
The Compression Objective 36
Full CISI Training Objective 36
Codebook Diagnostics and Symbolic Stability 37

Staged Curriculum Design **39**

Semiographic Projection and Interpretation **40**
Linear Probing and Linguistic Translation 40

Expected Failure Modes **41**
Codebook Collapse 41
Symbol Fragmentation 41
Pseudo-Symbol Formation 41
Curriculum Overfitting 42
Probe-Induced Overinterpretation 42
Brittle Symbolic Transfer 42

Empirical Evaluation Protocol **43**
Zero-Shot Out-of-Distribution Evaluation 43
Ablation of the Compression Interface 43
Topological Clustering and Code-Reuse Metrics 44
Human Semiotic Mapping and Consistency Studies 44

Discussion **45**
Cognitive Interpretation 45
Relation to Volume I 45
Interpretability and Accountability 46
Limits of the Framework 46

Volume II Conclusion **47**
Transition to Volume III 47

Volume III: Routing Fields **48**

Routing Fields and Hierarchical Topic-Conditioned Routing **50**
Introduction 50
Problem Space: Token-Level Routing and Architectural Fragmentation 51
Contributions 52
Overview of the Volume 52

Related Work **54**
Sparse Mixture-of-Experts Models 54
Routing Simplification and Load Balancing 54
Large-Scale Sparse Language Models 55
Alternative Routing Mechanisms 55
Distributed MoE Training and Hardware Efficiency 55

Long-Context Inference and KV-Cache Locality 56

Propositions and Falsification Conditions **57**

The Dual-Stage Hierarchical Routing Architecture **59**
Stage 1: Macroscopic Segment Routing 59
Stage 2: Microscopic Token Dispatch 60
Routing Stability and Expert Locality Metrics 60
Routing Entropy and Load Distribution 61
Communication Cost Model 61
Cache Locality Objective 62

Algorithmic Specification **63**

Structural Regularization and Starvation Prevention **65**
Full HTCR Training Objective 65
Shortlist Annealing 66

Expected Failure Modes **67**
Topic Collapse 67
Routing Inertia 67
Segment-Boundary Error 67
Expert Starvation 68
Over-Specialized Experts 68
Communication Overhead from Macro-Routing 68
Reduced Flexibility under Mixed-Topic Contexts 68

Empirical Evaluation Protocol **69**
Distributed Throughput and Communication Profiles 69
Annealing Path and Capacity Ablations 69
Expert Activation Entropy and Balancing Verification 70
Cache Locality and Memory Footprint Mapping 70

Discussion **71**
Routing as Architectural Coordination 71
Relation to Volume I and Volume II 71
Systems Interpretation 72
Interpretability and Accountability 72
Limits of the Framework 72

Volume III Conclusion **74**

Conclusion: Toward a Layered Theory of Latent Cognition **76**

Technical Specification for Autonomous Incident Response **82**

Introduction and Scope **83**
The Action Gatekeeper (Γ) Boundary Equations 83
The Cognitive Audit Trail (CAT) 84
Empirical Operational Reality: The R_s Delta 84
Operational Limits of Appendix A 85
Interpretive Scope 86

Transitional Note **88**

Exploratory Convergence Pathways: Representation, Operators, and Spectral Geometry **89**
From Token Statistics to Spectral Organization 91
Transform Domains as Representational Coordinate Systems . . . 93
Operators and Dynamical Constraints 96
Operator Geometry and Coordinated Interfaces 99
Spectral Constraints and Bounded Emergence 103
Constraints and Scientific Discipline 106
Toward a Physics of Representation 108
The Boundary Envelope and Unresolved Pathways 110
Non-Asymptotic Spectral Dynamics under Stochastic Optimization 110
Topological Discontinuities in Dynamic Routing 110
Representational Entropy and Invariant Measures 111
Epistemic Taxonomy and Research Frontiers 113

Bibliography **115**

List of Figures

1 Canonical architecture of the Latent Cognition research program. The DLI-LLM manuscript serves as the architectural foundation, while the three volumes of the Trilogy form the main theoretical sequence, progressing from memory fields to symbol fields to routing fields. Companion hypotheses and future extensions orbit the main sequence without replacing it. xxix

2 Recursive coupling among the latent memory operator M, symbolic stabilization operator S, and routing operator R. The Trilogy proposes that cognition-like behavior emerges not from isolated processing stages, but from continuous bidirectional interaction across a unified latent operator field. xxxiv

3 Differentiable Latent Indexing architecture. The diagram illustrates how latent key–value memory slots are queried from the current hidden state and fused back into the Transformer attention pathway during the forward pass. . . . 12

4 Standard retrieval-augmented generation pipeline. In conventional RAG, retrieval remains externally mediated through a document index or vector store before generation. DLI proposes to move part of this retrieval logic into the model's differentiable computation path. 14

5 Gradient pathways in Differentiable Latent Indexing. Query projection, similarity computation, sparse retrieval weighting, routed memory integration, and gated fusion remain within the differentiable computation graph, allowing latent memory to co-evolve with Transformer parameters. 17

6 Training flow and gating control in DLI-LLM. Product-quantized key–value lookup, attention fusion, routing gates, and gradient-based memory adaptation interact across Transformer blocks. 22

Zinnia Labs Research Monographs

Volume I
Memory Fields

Volume II
Symbol Fields

Volume III
Architecture and Completeness

Datasets

Differentiable Latent Indexing inside Large Language Models (DLI-LLM) - Hypothesis 1

Curriculum-Induced Symbol Invention for Emergent Reasoning Primitives (CISI) – Hypothesis 2

Hierarchical Topic-Conditioned Routing for Mixture-of-Expert Language Models (HTCR) – Hypothesis 3

Preface

The increasing scale and capability of contemporary neural architectures have exposed a persistent gap in their theoretical characterization. While considerable progress has been made in optimizing performance, scaling training regimes, and extending model capacity, the internal organization of learned representations remains only partially formalized. In particular, the interaction between memory, abstraction, and computation is often treated as a set of separate problems: memory as external retrieval, symbolic reasoning as emergent but unstructured behavior, and architectural routing as an engineering optimization. This fragmentation limits the ability to reason about model behavior within a unified conceptual frame.

The present work is motivated by the need for a more structured account of internal model dynamics. Rather than introducing a single empirical benchmark or an isolated architectural modification, the Trilogy adopts a conceptual and architectural approach to identify recurring patterns across modern neural systems. Its purpose is not to claim a complete theory of machine cognition, but to establish a minimal set of constructs through which memory, symbolic stabilization, and computational organization can be described as related dimensions of latent model behavior.

The Latent Cognition Trilogy is organized as a sequence of three investigations, each addressing a distinct but interdependent layer of internal computation. The first volume examines latent memory and the conditions under which retrieval may be understood as an intrinsic process rather than a purely external augmentation procedure. The second volume considers the emergence of symbolic stability under compression, curriculum structure, and representational constraint. The third volume addresses architectural organization, with particular attention to hierarchical routing mechanisms that coordinate computation across specialized components while preserving coherence at scale.

Taken together, these studies do not assume a unified theory in advance. They develop a layered description of internal model structure, moving from memory to symbolic representation to architectural coordination. Each volume isolates a specific class of mechanisms and proposes corresponding formalizations, interpretive constructs, and testable propositions. The progression is therefore not intended as a claim of completeness, but as a structured pathway for analyzing latent cognition in artificial systems.

This preface situates the individual volumes within that broader program. The sections that follow introduce the research program note, the canonical relationship among the companion works, and the conceptual foundations shared across the Trilogy.

The following chapter reproduces, with minor editorial adaptation, the foundational DLI-LLM manuscript that motivated the broader research program developed throughout the Trilogy.

Prologue

The Latent Cognition Trilogy

[1]

Contextual overview

.

The problem addressed in this work is the persistent separation between parametric memory, encoded within model weights, and non-parametric memory retrieved through external databases or retrieval systems in modern **Retrieval-Augmented Generation** (**RAG**) architectures. Although RAG-based systems have improved factual grounding in **large language models** (**LLMs**), they introduce architectural fragmentation between the retrieval and generation components, which are often optimized independently and coupled only at inference time.

This separation introduces several limitations, including retrieval latency, semantic drift between the retrieved evidence and the model's context, and restricted end-to-end gradient flow between evidence selection and downstream reasoning. Existing memory-augmented architectures partially address these issues but typically maintain retrieval as an external or weakly integrated process.

The *DLI-LLM framework* proposes an alternative architectural formulation in which differentiable retrieval mechanisms are integrated directly into Transformer layers via a latent key-value memory structure [8]. Re-

[1] Chapter originally released under DOI 10.2139/ssrn.6821299 by ORCID:0000-0002-4992-4541

trieval queries are generated from hidden-state representations and routed through learned attention-gating mechanisms, enabling the retrieved vectors to participate directly within the transformer's forward computation pathway.

The objective of this paper is not to claim empirical validation but to formalize the architectural implications, theoretical properties, and comparative positioning of a retrieval-integrated transformer architecture within the broader landscape of retrieval- and memory-augmented transformer architectures [57, 47, 69, 42].

Introduction

Retrieval-Augmented Generation (RAG) architectures have improved factual grounding in large language models (LLMs) by combining parametric generation with external retrieval systems. However, these architectures often maintain an operational separation between retrieval and inference, leading to architectural fragmentation, increased retrieval latency, and limited end-to-end differentiability between evidence selection and downstream generation.

This separation introduces several limitations, including retrieval latency, semantic drift between the retrieved evidence and the model's context, and restricted end-to-end gradient flow between evidence selection and downstream reasoning. Existing memory-augmented architectures partially address these issues but typically maintain retrieval as an external or weakly integrated process.

The DLI-LLM framework proposes an alternative architecture in which differentiable retrieval mechanisms are integrated directly into Transformer layers via a latent key-value memory structure. Retrieval queries are generated from hidden-state representations and routed through learned attention-gating mechanisms, enabling the retrieved vectors to participate directly in the forward computation.

The objective of this paper is not to claim empirical validation, but to formalize the architectural implications, theoretical properties, and comparative positioning of this retrieval-integrated transformer architecture framework within the broader landscape of retrieval-augmented and memory-augmented transformer architectures.

Contributions

This work contributes:

- A conceptual architectural framework that integrates latent memory retrieval directly into transformer computation pathways.
- A mathematically formulated retrieval mechanism with differentiable routing and integration of latent key-value memory.
- A theoretical blueprint for retrieval-integrated transformer architectures operating within the forward computation graph.
- A comparative analysis situating DLI-LLM relative to retrieval-augmented and memory-augmented transformer systems.
- A structured discussion of theoretical behaviors, architectural limitations, and future experimental directions associated with differentiable latent retrieval.

Overview of the Paper

Section 2 develops the methodological and *architectural foundation of the DLI-LLM framework*[2] by decomposing the originating hypothesis into its principal architectural components. The section formalizes the retrieval mechanism through mathematical definitions, algorithmic specification, and implementation-oriented architectural modeling. It also situates the proposed framework in relation to existing retrieval- and memory-augmented transformer architectures through comparative analysis. . Originally written as a journal paper, this piece was later included in the book as a prologue because it describes the architectural phenomenon that inspired the broader investigation.

Section 3 examines the predicted architectural behaviors and limitations of differentiable latent retrieval, including retrieval dynamics, memory-scaling considerations, retrieval coherence, and potential failure modes, in the absence of an empirical implementation. **Section 4** discusses broader implications for the design of retrieval-integrated transformer architectures, internal memory architectures, and future experimental investigations.

[2]This chapter is available on SSRN at 6821299 or via DOI 10.2139/ssrn.6821299

Architectural Framework and Methodology

This paper does not present empirical validation or benchmarked implementation results. Instead, it develops a conceptual architectural framework based on the **Differentiable Latent Indexing** inside *Large Language Models (DLI-LLM) hypothesis*, as reproduced in **Appendix A**. The methodology follows a structured process of architectural decomposition, mathematical formalization, comparative analysis, and implementation-oriented modeling to examine how differentiable retrieval mechanisms can be integrated directly into transformer computational pathways.

Architectural Foundations

The *DLI-LLM framework* is derived from four principal architectural assumptions:

1. The integration of a differentiable memory module within transformer layers.

2. The use of latent key-value memory structures to support scalable retrieval operations.

3. The existence of a shared latent vector space for token representations and retrieved document embeddings.

4. A learned routing-and-gating mechanism that dynamically controls retrieval activation during inference.

Together, these components define a retrieval-integrated transformer architecture in which latent memory access is integrated into the forward computation rather than occurring as an external retrieval stage. Unlike conventional **Retrieval-Augmented Generation** (**RAG**) systems, in which retrieval and generation are often decoupled, DLI-LLM embeds retrieval directly into transformer-layer operations. Retrieval queries are generated from hidden-state representations and used to access latent memory structures through differentiable similarity functions. Retrieved vectors are subsequently fused back into the attention pathway through learned gating mechanisms.

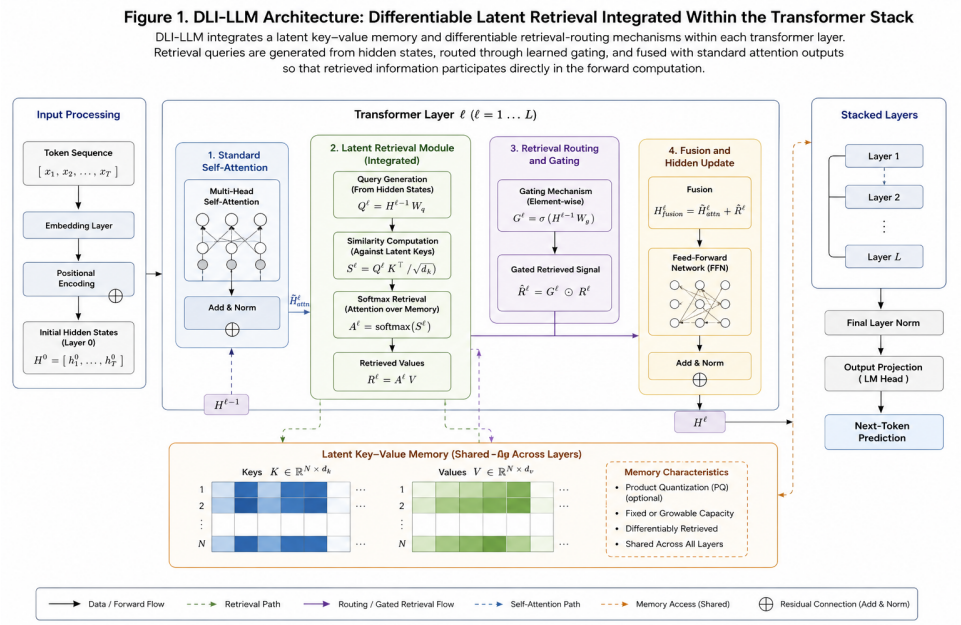


Figure 1 illustrates the conceptual retrieval pathway and latent memory integration process within the transformer stack.

Latent Memory Representation

The proposed framework assumes the existence of a latent key-value memory structure:

$$M = \{(k_i, v_i)\}_{i=1}^{N}, k_i, v_i \in \mathbb{R}^d$$

The architecture assumes that both token representations and retrieved document embeddings can coexist in a shared latent vector space, enabling retrieval operations to occur in the same representational domain as transformer hidden states. To support scalability, the framework optionally incorporates product quantization strategies inspired by large-scale vector retrieval systems and differentiable memory architectures. This enables latent retrieval to remain computationally tractable even when memory capacity grows substantially.

Retrieval and Routing Mechanism

At each transformer layer, the hidden representation:

$$h_t \in \mathbb{R}^d$$

$$q_t = W_q h_t$$

$$s_i = \langle q_t, k_i \rangle$$

$$\alpha_i = \frac{\exp(s_i/\tau)}{\sum_j \exp(s_j/\tau)}$$

where τ denotes a temperature parameter that controls the sharpness of retrieval. The retrieved latent representation becomes:

$$r_t = \sum_{i=1}^{N} \alpha_i v_i$$

A learned routing gate modulates integration between retrieved memory and the original transformer hidden state:

$$g_t = \sigma(W_g[h_t; r_t])$$

The fused representation is subsequently computed as:

$$\widetilde{h}_t = g_t \cdot r_t + (1 - g_t) \cdot h_t$$

This formulation enables retrieved latent memory vectors to participate directly in transformer-layer computation [88] while preserving differentiable gradient propagation through both the retrieval and attention pathways [33, 34, 74]

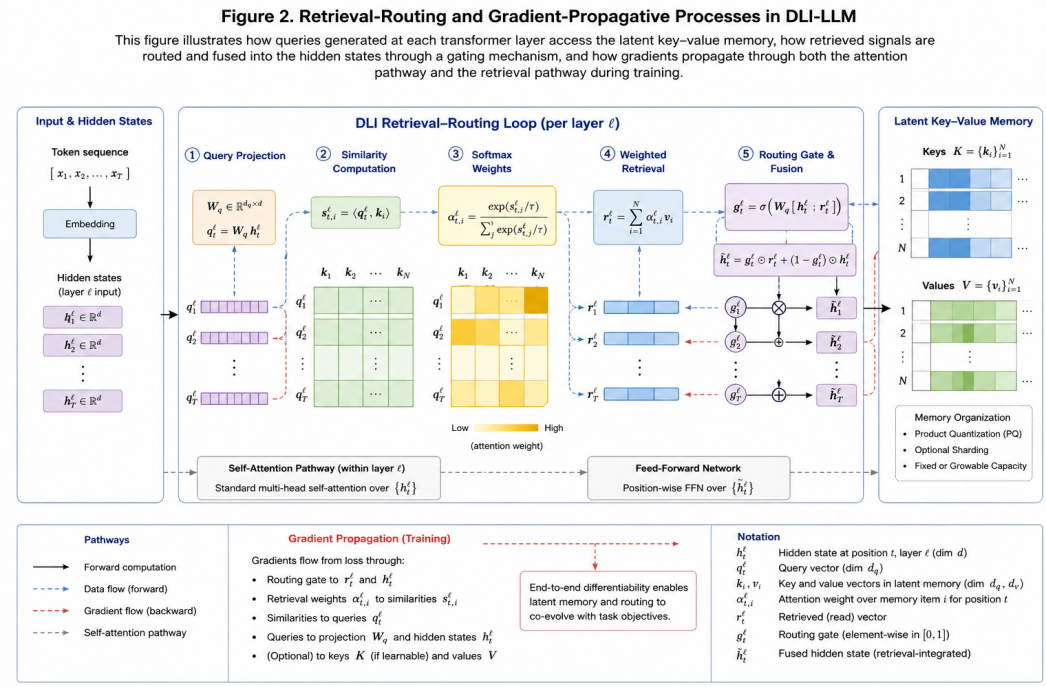


Figure 2 illustrates the retrieval-routing and gradient-propagation processes within the proposed framework.

Mathematical Interpretation

The proposed formulation extends the standard transformer computation by introducing a differentiable retrieval pathway that operates alongside self-attention mechanisms. Unlike external retrieval architectures, in which retrieval systems operate independently of the language model, DLI-LLM integrates retrieval directly into the computational graph. The framework draws conceptually on differentiable memory systems, attention-based addressing mechanisms, neural memory architectures, and retrieval-augmented transformer systems [34]. However, the distinguishing feature of DLI-LLM is the integration of latent retrieval directly within transformer-layer computation rather than through externally orchestrated retrieval pipelines. The resulting architecture enables retrieval decisions, routing behavior, and memory integration to evolve jointly during optimization through shared gradient propagation.

Algorithmic Retrieval Process

Algorithm 1 summarizes a single retrieval operation within one transformer layer. **Input:** Hidden state h_t, latent memory $M = \{(k_i, v_i)\}$, parameters W_q, W_g **Algorithm 1:** Differentiable Latent Retrieval Step **Output:** Fused representation $\widetilde{h}_t$

A. $q_t \leftarrow W_q h_t$

B. Compute similarity scores $s_i = \langle q_t, k_i \rangle$

C. Compute retrieval weights $\alpha = \text{softmax}(s/\tau)$

D. Retrieve latent vector:

$$r_t = \sum_i \alpha_i v_i$$

E. Compute routing gate:

$$g_t = \sigma(W_g[h_t; r_t])$$

F.

$$\widetilde{h}_t = g_t \cdot r_t + (1 - g_t) \cdot h_t$$

G. Return $\widetilde{h}_t$

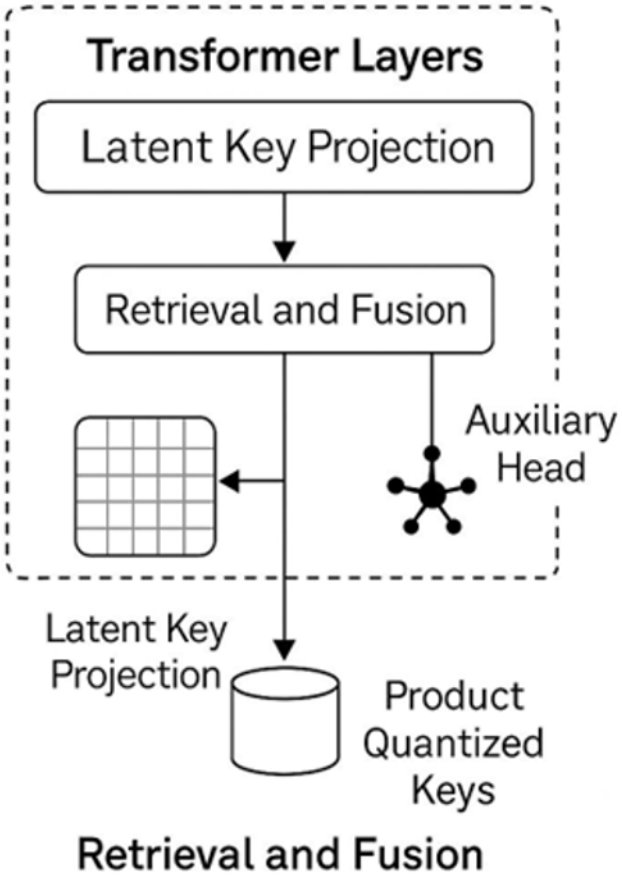


Figure 3 presents the conceptual computational graph for the retrieval-routing process.

Conceptual Implementation Framework

Although this work remains theoretical, an implementation-oriented DLI prototype would conceptually involve four principal stages: **Memory Construction**

- Extraction of latent document embeddings
- Construction of latent key-value memory banks
- Optional application of product quantization strategies for scalable retrieval

Transformer Layer Integration

- Insertion of DLI retrieval blocks after self-attention layers
- Integration of learned routing and gating mechanisms
- Fusion of retrieved latent vectors within the transformer layer computation

Training Objective A conceptual joint objective may combine language modeling and retrieval optimization:

$$\mathcal{L} = \mathcal{L}_{LM} + \lambda \mathcal{L}_{retrieval}$$

where λ controls the relative contribution of retrieval optimization. **Inference Behavior** Potential inference configurations may include:

- Frozen latent memory
- Sparse top-k retrieval
- Memory sharding
- External memory swapping mechanisms

Comparative Architecture Analysis

The *DLI-LLM framework* conceptually intersects with multiple memory-augmented and retrieval-integrated transformer architectures [90].

Architecture	**Relation to DLI-LLM**	**Limitation Relative to DLI-LLM**
Transformer-XL	Long-context recurrence	No latent key-value retrieval
RETRO	External retrieval augmentation	Retrieval remains externally orchestrated
Memorizing Transformers	Learnable memory tokens	No unified latent retrieval pathway
Differentiable Neural Computers	Key-value differentiable memory	Not integrated into transformer-layer computation
Recurrent Memory Transformers	Persistent memory states	Limited retrieval routing flexibility

Architecture	Relation to DLI-LLM	Limitation Relative to DLI-LLM
KNN-LM	Post-hoc retrieval augmentation	Retrieval remains external and non-differentiable
Perceiver IO	Latent bottleneck architecture	No integrated latent retrieval routing
Hyena Hierarchy	Long-range implicit sequence modeling	No explicit memory retrieval mechanism

Limitations and Theoretical Scope

This work does not present empirical implementation, benchmark evaluation, or experimental validation. The proposed framework should therefore be interpreted as a conceptual architectural model for formalizing differentiable latent retrieval within transformer-based systems. Several limitations remain unresolved, including:

- computational scaling of latent memory structures,
- memory saturation and retrieval interference,
- retrieval interpretability,
- training stability,
- and the computational overhead associated with differentiable memory routing.

The framework also assumes that it is feasible to jointly optimize retrieval and language modeling objectives within a unified transformer computation graph, an assumption that requires future empirical validation. Accordingly, the present work should be understood as a theoretical systems architecture proposal designed to motivate future implementation-oriented and experimental investigations into retrieval-integrated transformer architectures.

Predicted Architectural Behaviors

Because this work does not present empirical implementation or benchmark evaluation, the behaviors discussed in this section should be interpreted as theoretical architectural predictions derived from the proposed

DLI-LLM framework and related findings in prior memory-augmented language model research.

Predicted Retrieval and Memory Behaviors

Retrieval Efficiency

By integrating retrieval directly into the transformer's forward pass, DLI-LLM may reduce the external retrieval bottlenecks commonly associated with Retrieval-Augmented Generation (RAG) architectures [90]. This prediction is consistent with prior observations that reducing external retrieval overhead can improve retrieval efficiency and inference coordination in memory-augmented systems.

Grounding Stability and Hallucination Reduction

The integration of latent memory retrieval pathways may improve contextual grounding during generation by enabling retrieved latent representations to participate directly in transformer-layer computation. Similar retrieval-assisted grounding effects have been observed in retrieval-augmented generators and nearest-neighbor language models.

Memory-Assisted Generalization

Because latent memory retrieval is integrated differentiably within transformer layers, the framework may exhibit memory-assisted generalization similar to that observed in hybrid retrieval and memory-augmented architectures.

Few-Shot Adaptation Dynamics

The proposed routing and gating mechanisms may improve adaptation under limited supervision by selectively integrating retrieved latent representations according to contextual relevance. Similar dynamics have been reported in memory-augmented neural systems employing differentiable memory access.

Long-Context Retrieval Coherence

Cross-layer retrieval propagation may improve retrieval consistency across extended token sequences by allowing latent memory interactions to evolve

throughout transformer-layer computation. This behavior aligns conceptually with trends observed in long-context and persistent-memory transformer architectures [16].

Predicted Failure Modes and Architectural Risks

As with other memory-augmented architectures, finite latent memory capacity may lead to retrieval saturation under large-scale document loads or prolonged retrieval accumulation.

Retrieval Redundancy

High-dimensional latent retrieval may yield semantically redundant value-vector patterns, potentially reducing retrieval precision without appropriate compression or quantization.

Contextual Retrieval Drift

Cross-layer retrieval propagation may introduce retrieval-context misalignment when latent query representations diverge from earlier contextual states. Similar forms of retrieval instability have been observed in persistent memory architectures.

Retrieval Poisoning and Memory Contamination

Because latent retrieval pathways participate directly in transformer computation, corrupted or adversarial memory representations may propagate erroneous retrieval signals throughout downstream generation pathways. This introduces potential risks associated with latent memory contamination and retrieval integrity.

C

omparative Architectural Behavior Analysis

Discussion

The *DLI-LLM framework* proposes a retrieval-integrated transformer architecture in which latent memory access is incorporated into the forward-computation pathway rather than into an externally orchestrated retrieval

stage. This architectural formulation has implications for how memory coordination, retrieval dynamics, and contextual grounding may be modeled within transformer-based systems.

Retrieval Integration and Memory Coordination

By embedding differentiable latent retrieval directly within transformer layers, the proposed framework reduces the operational separation between retrieval and inference commonly observed in Retrieval-Augmented Generation (RAG) architectures. Similar principles have appeared in memory-augmented neural systems and differentiable memory architectures. However, DLI-LLM extends these concepts by integrating shared latent retrieval pathways that operate directly within transformer-layer computation. The proposed framework suggests that retrieved latent representations may participate directly in contextual computation rather than functioning as externally injected retrieval artifacts.

Learning Dynamics and Adaptation

The framework also suggests potential implications for continual learning and adaptive memory integration [37]because latent memory structures can be updated independently of full model retraining, and retrieval-integrated transformer architectures could support incremental memory adaptation amid evolving information conditions. At the same time, this flexibility introduces architectural challenges associated with memory scaling, retrieval interference, latent redundancy, and retrieval saturation. Efficient pruning, quantization, and routing strategies will likely be necessary to maintain stable retrieval behavior at scale.

Bias, Safety, and Retrieval Integrity

Integrating retrieval pathways directly within transformer computation also introduces safety and interpretability considerations. Latent memory structures may preserve biased, corrupted, or privacy-sensitive representations that become difficult to inspect once integrated into differentiable retrieval pathways. These concerns partially resemble challenges observed in retrieval-augmented systems. Still, they may become more difficult to isolate when retrieval coordination occurs internally through latent routing mechanisms rather than through externally inspectable retrieval pipelines.

Routing and Architectural Control

A central component of the proposed framework is the retrieval-routing gate, which regulates how retrieved latent vectors influence transformer hidden states. This routing process determines the degree to which retrieved memory participates in downstream computation and contextual inference. **Figure 4** presents a conceptual comparison between DLI-LLM and conventional retrieval-augmented architectures across multiple architectural dimensions, including retrieval coordination, grounding stability, memory coherence, and the distribution of retrieval failures.

Figure 1. DLI-LLM Architecture: Differentiable Latent Retrieval Integrated Within the Transformer Stack

DLI-LLM integrates a latent key–value memory and differentiable retrieval-routing mechanisms within each transformer layer. Retrieval queries are generated from hidden states, routed through learned gating, and fused with standard attention outputs so that retrieved information participates directly in the forward computation.

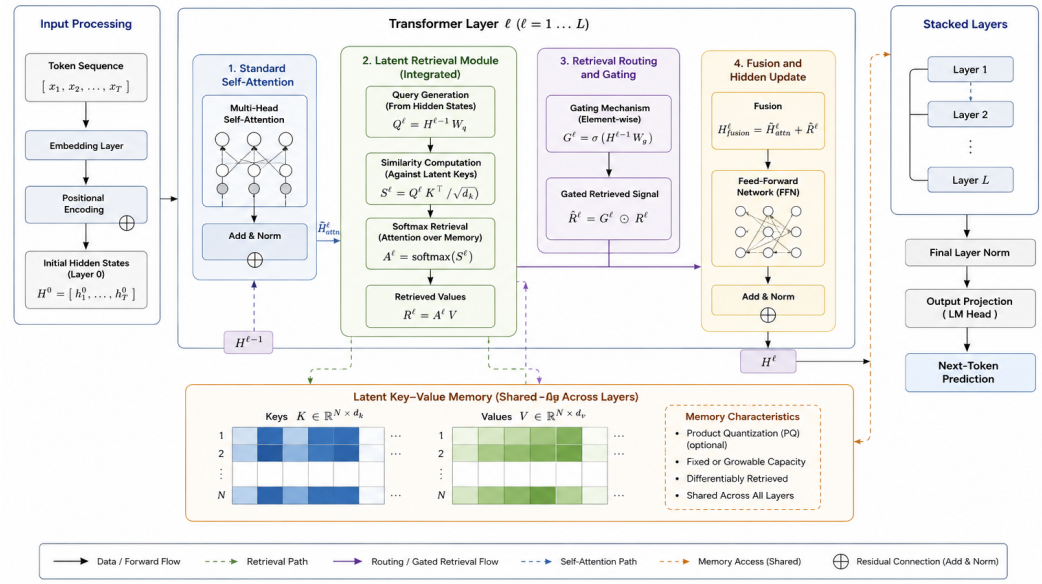


Figure 4. Conceptual illustration of retrieval fragmentation and modality separation in conventional retrieval-augmented architectures. The figure does not present empirical benchmark data and should instead be interpreted as a theoretical comparative model derived from architectural analysis and prior observations in memory-augmented transformer research.

Broader Architectural Implications

More broadly, DLI-LLM contributes to ongoing research exploring how retrieval, memory coordination, and contextual inference may become

more tightly integrated within transformer-based architectures. Rather than treating retrieval exclusively as an external augmentation layer, the framework proposes a conceptual direction in which latent retrieval mechanisms participate directly in model computation. Although the present work remains theoretical and requires empirical validation, it provides a formal architectural basis for future investigation into retrieval-integrated transformer architectures and differentiable coordination of latent memory.

Closing thoughts

This paper formalized the *DLI-LLM hypothesis* as a conceptual architectural framework for integrating differentiable latent retrieval directly within transformer-layer computation. Unlike conventional retrieval-augmented architectures that maintain an operational separation between retrieval and inference, the proposed framework introduces a unified latent retrieval pathway operating within the transformer's forward pass. The work presented a mathematical formulation of differentiable latent retrieval, an implementation-oriented architectural model, and a comparative analysis relative to existing memory-augmented and retrieval-integrated transformer systems. In addition, the paper examined predicted architectural behaviors, retrieval dynamics, and potential limitations associated with latent memory integration. Although the framework remains theoretical and requires empirical validation, the proposed architecture provides a structured basis for future investigation into retrieval-integrated transformer architectures, differentiable memory coordination, and latent retrieval mechanisms operating directly within the model's computational pathways.

Note on the Research Program

The present manuscript belongs to a structured theoretical research program aimed at formalizing internal mechanisms of machine cognition. The program, titled *The Latent Cognition Trilogy*, is organized around three major stages: latent memory and retrieval-native representation, symbolic stabilization under compression and curriculum constraints, and architectural coordination through hierarchical routing mechanisms.

The Trilogy should not be read as an isolated sequence of essays. It is part of a broader research architecture that includes a foundational systems paper, companion hypothesis papers, and future domain-specific extensions. Within this architecture, *Differentiable Latent Indexing in LLMs: Toward Native Retrieval as a Cognitive Function* functions as the architectural foundation. That work develops the systems-level premise that retrieval may be modeled as a native differentiable function within transformer-based systems rather than as an external procedural layer.

The three volumes of the Trilogy extend that architectural foundation into a broader theoretical sequence. Volume I examines latent memory and retrieval-native cognition. Volume II addresses the stabilization of symbolic structure within compressed latent spaces. Volume III analyzes hierarchical routing and architectural coordination as mechanisms for maintaining coherence across specialized computational components.

The companion hypothesis papers occupy a different position. They are not appendices to the Trilogy, nor substitutes for its main sequence. They function as satellite formalizations: each isolates a specific theoretical claim that supports, constrains, or extends the program without overloading the central manuscript.

The development of the *Latent Cognition Trilogy* is prompted not by a desire to introduce incremental optimization benchmarks to existing deep learning stacks, but to address a foundational systemic crisis looming over computational theory. The contemporary artificial intelligence industry remains tethered to an empirical paradigm of brute-force scaling—expanding token context windows, multiplying raw parameter counts, and assembling fragmented, non-differentiable pipelines (e.g., decoupled Retrieval-Augmented Generation frameworks and token-isolated Mixture-of-Experts routing).

This research program asserts that current methodologies represent an immense misallocation of structural efficiency. By maintaining an artificial bifurcation between parametric weight structures and non-parametric data repositories, standard architectures introduce severe semantic drift, catastrophic communication overhead across distributed hardware clusters, and terminal vulnerability to data poisoning.

The triple operator framework $\mathcal{A} = (M, S, R)$ developed across these three volumes establishes an integrated, mathematically continuous alternative.

Canonical Predictions of the Framework

By formalizing the interactions between Differentiable Latent Indexing (M), Curriculum-Induced Symbol Invention (S), and Hierarchical Topic-Conditioned Routing (R), this framework yields three verifiable, high-consequence macro-architectural predictions:

1. Mitigation of the Synthetic Semantic Collapse

As autonomous networks increasingly generate the data substrate upon which future models are trained, standard flat-token architectures inevitably succumb to cognitive decay and information entropy (the classical model collapse or "Metaverse Black Hole"). The integration of an internal discrete information bottleneck (S) is designed to systematically condense incoming ungrounded data streams into a stable, localized vocabulary of emergent reasoning primitives.The system predicts that symbol-grounded architectures will remain invariant to synthetic data degradation, transforming sprawling informational spaces into crystalline, hyper-dense symbolic kernels.

2. Bounded Constant-Time Cognitive Locality ($O(1)$ Scaling Target)

In standard attention-gated networks, retrieval overhead scales systematically with context expansion, creating persistent latency boundaries. Under the unified latent substrate ($M \times R$), the computational footprint required to isolate, contextually route, and synthesize cross-sequence historical dependencies is designed to approach invariance relative to the scale of the background non-parametric repository. Under constrained retrieval geometries, this formulation projects an asymptotic flattening of retrieval overhead, positioning $O(1)$ operational efficiency as a bounded locality-preserving target.

3. Amortization of Hardware Communication Topology

Flat token-isolated Mixture-of-Experts systems trigger persistent, volatile data synchronization loops across distributed hardware nodes during inference, saturating physical interconnect bandwidth. By utilizing macrosegment topic-conditioned shortlists (R), expert allocation trajectories are structurally stabilized. This program predicts a definitive reduction in inter-node `All-to-All` communication primitives, allowing distributed hardware clusters to scale parameter exposure near line-rate capacity without network choking.

Through these lenses, the trilogy ceases to be a speculative exercise in mathematical cognitive science; it stands as an architectural blueprint for a viable pathway to transition from brute-force language mapping to self-managed, natively grounding machine intelligence.

Canonical Structure of the Research Program

The Latent Cognition Trilogy should be read as the main theoretical sequence within a broader research program on latent cognition, retrieval-native intelligence, and bounded artificial agency. The Trilogy is not an isolated manuscript collection, nor is it a container for every related hypothesis, architectural proposal, or companion paper. Rather, it functions as the central conceptual axis around which a wider body of work is organized.

Within this structure, *Differentiable Latent Indexing in LLMs: Toward Native Retrieval as a Cognitive Function* serves as the architectural foundation of the research program. That work develops the systems-level premise that retrieval should not remain an external augmentation layer, as in conventional retrieval-augmented generation architectures, but may instead be modeled as a differentiable cognitive function embedded within the representational dynamics of transformer-based systems. For this reason, the DLI-LLM paper should be understood as the architectural substrate beneath the Trilogy rather than as a supplementary appendix.

The three companion hypothesis papers occupy a different position. They do not define the program's base architecture. Instead, they formalize adjacent theoretical claims that support, constrain, or extend the main conceptual sequence developed in the Trilogy. Their purpose is to preserve analytical precision around claims that would otherwise overload the central manuscript.

Accordingly, the present work adopts the following canonical structure: the DLI-LLM paper serves as the foundational architecture paper; the Latent Cognition Trilogy constitutes the main theoretical sequence; and

the companion hypothesis papers function as formal satellite works within the same research program. This organization allows the Trilogy to remain coherent while preserving the broader intellectual architecture required for future papers, extensions, and domain-specific applications.

Table 2: Canonical organization of the Latent Cognition research program.

Work	**Canonical Role**	**Relationship to the Trilogy**
Differentiable Latent Indexing in LLMs	Architectural foundation / formal paper version	Provides the systems-level substrate for Volume I and the broader Trilogy.
Volume I: Memory Fields	Main sequence, first volume	Develops retrieval-native cognition and latent memory fields.
Volume II: Symbolic Stabilization	Main sequence, second volume	Examines how compressed latent representations acquire symbolic stability.
Volume III: Hierarchical Routing	Main sequence, third volume	Analyzes routing as architectural coordination across specialized computation.
Companion Hypothesis Paper I	Formal companion work	Isolates and formalizes a supporting theoretical claim.
Companion Hypothesis Paper II	Formal companion work	Extends the framework into an adjacent conceptual domain.
Companion Hypothesis Paper III	Formal companion work	Tests or constrains the boundaries of the main theoretical sequence.
Future extensions	Parallel works	Apply or expand the framework into cybersecurity, conscious AI, health sciences, bounded autonomy, or governance-aware AI systems.

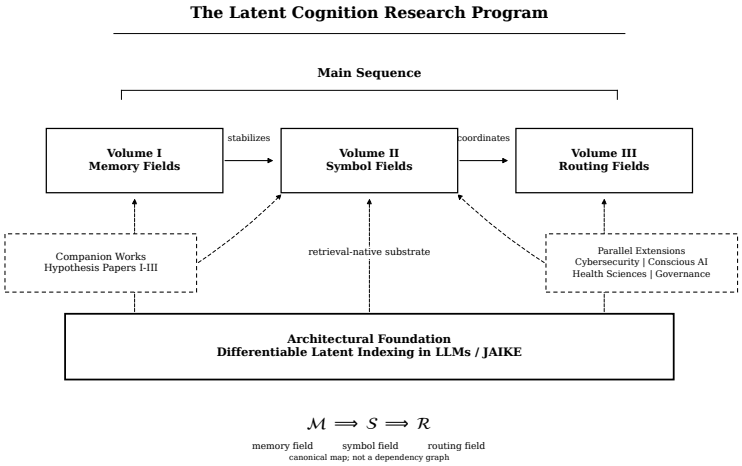


Figure 1: Canonical architecture of the Latent Cognition research program. The DLI-LLM manuscript serves as the architectural foundation, while the three volumes of the Trilogy form the main theoretical sequence, progressing from memory fields to symbol fields to routing fields. Companion hypotheses and future extensions orbit the main sequence without replacing it.

Conceptual Foundations

Core Constructs

The Trilogy operates on a set of shared constructs that describe internal representations and computational organization in neural architectures. These constructs are not intended to form a closed ontology. They provide a working vocabulary for examining how memory, symbolic stabilization, and routing may interact within latent computational systems.

Latent State. A latent state is represented by $\mathbf{h}_t \in \mathcal{L}$, encoding the model's internal state at computational step t. It is not treated merely as an intermediate activation, but as a structured representational state through which information, context, and transformation history may be partially expressed.

Memory Field. A memory field, denoted $\mathcal{M}$, is a structured collection of latent representations that are accessible via differentiable retrieval. Within the Trilogy, the memory field provides the conceptual basis for treating retrieval as an internal representational operation rather than as an external lookup procedure.

Symbolic Representation. A symbolic representation is denoted as $s \in \mathcal{S}$ and refers to a compressed or stabilized encoding derived from latent states. The term does not imply classical symbolic manipulation in the strict sense. Rather, it designates a representational condition in which compressed latent structures acquire sufficient stability to support abstraction, reuse, and interpretation.

Routing Function. A routing function, denoted $\mathcal{G}(\cdot \mid \mathcal{E})$, governs the allocation of computation across specialized modules, experts, or representational pathways. In this framework, routing is treated not only as

an efficiency mechanism but as a structural principle through which large-scale architectures maintain coherence across distributed computation.

Structural Relationship

The Trilogy examines the interaction between these constructs as a coupled latent operator field analyzed across three primary architectural dimensions:

$$\mathcal{M} \implies \mathcal{S} \implies \mathcal{R}$$

This progression should not be interpreted as a strict causal chain. Rather, it represents the analytical sequence adopted by the Trilogy. The first stage examines how latent memory may be structured and retrieved. The second stage considers how compressed representations may acquire symbolic stability. The third stage analyzes how routing mechanisms coordinate computation across specialized structures.

The shared hypothesis across the Trilogy is that these constructs are not independent engineering conveniences. They may represent interdependent layers of a broader latent cognitive architecture: memory supplies retrievable structure, symbolic stabilization supplies abstraction, and routing supplies coordinated computational organization.

Operator Coupling Conditions

The preceding sections introduced memory, symbolic stabilization, and routing as distinct analytical constructs. However, the Latent Cognition Trilogy does not treat these operators as isolated architectural conveniences nor as strictly sequential processing stages. The central hypothesis of the Trilogy is that latent cognition emerges from the continuous interaction of retrieval, abstraction, and computational coordination within a unified operator space.

Contemporary neural architectures frequently separate these mechanisms into disconnected engineering layers. Retrieval is treated as an external augmentation pipeline, symbolic behavior is interpreted post hoc as an emergent property of scale, and routing is optimized primarily as a systems-efficiency mechanism. This separation introduces discontinuities

between memory access, abstraction formation, and distributed computation. The Trilogy instead proposes that these mechanisms become behaviorally inseparable once embedded inside a shared latent computational substrate.

Accordingly, the total architectural state of the framework is formalized as:

$$A = (M, S, R)$$

where M denotes the latent memory operator, S the symbolic stabilization operator, and R the routing and coordination operator.

The sequence adopted across the three volumes:

$$M \Rightarrow S \Rightarrow R$$

should therefore not be interpreted as a unidirectional pipeline. It represents the analytical order through which the Trilogy isolates each layer for examination. In operational systems, these operators interact recursively and continuously.

Memory-to-Symbol Coupling

The first coupling condition concerns the interaction between latent retrieval and symbolic stabilization. Symbolic representations are not assumed to emerge independently of memory organization. Instead, the retrieval dynamics generated by M condition, in which latent structures become repeatedly compressed, reused, and stabilized within the symbolic operator space S.

Let h_t denote the latent state at computational step t. Symbolic projection may then be conditioned on retrieved latent structure:

$$S_t = \Pi_S(M(h_t))$$

where Π_S denotes the symbolic projection operator that maps retrieved latent trajectories to compressed symbolic representations.

The recursive interaction among the three primary operators is illustrated in **Figure** 2.

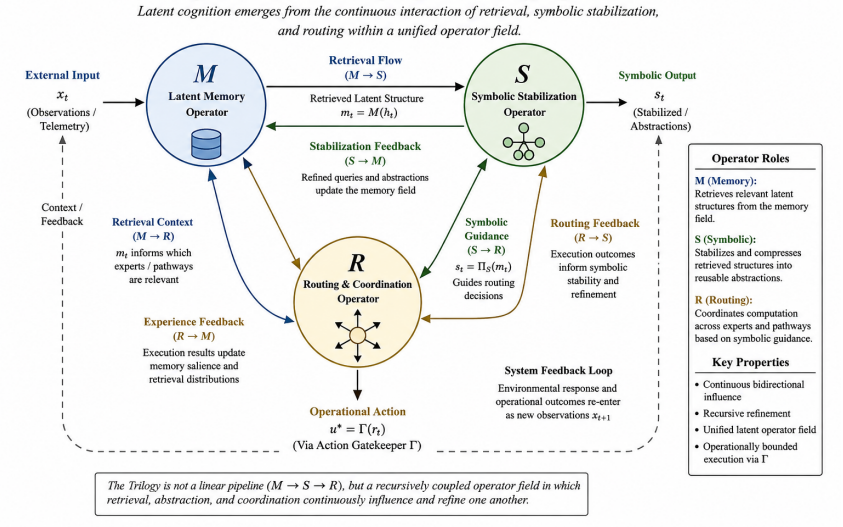


Figure 2: Recursive coupling among the latent memory operator M, symbolic stabilization operator S, and routing operator R. The Trilogy proposes that cognition-like behavior emerges not from isolated processing stages, but from continuous bidirectional interaction across a unified latent operator field.

Under this formulation, symbolic stabilization depends not only on compression pressure but also on the continuity and recurrence properties of the latent memory field itself.

Symbol-to-Routing Coupling

The second coupling condition concerns the relationship between symbolic stabilization and routing coordination. In the Trilogy framework, routing decisions are not governed exclusively by token-local activations or hardware scheduling constraints. Stabilized symbolic structures constrain which computational pathways become preferentially activated across distributed expert systems.

Routing allocation may therefore be conditioned on symbolic state trajectories:

$$R_t = G(S_t)$$

where $G(\cdot)$ denotes the routing function over stabilized symbolic representations.

This formulation implies that expert selection is partially determined by emergent symbolic organization rather than solely by local token statistics. Symbolic stabilization thereby functions as an intermediate coordination layer between latent representation and distributed computation.

Routing-to-Memory Coupling

The third coupling condition concerns the feedback relationship between routing and memory accessibility. Once computation is distributed across constrained expert pathways, the routing topology itself influences which latent memory structures remain visible or computationally reachable during subsequent inference steps.

The latent memory operator may therefore be conditioned on routing topology:

$$M_{t+1} = M(h_t \mid R_t)$$

Under this formulation, routing is not merely downstream execution control. It partially shapes the future informational geometry of the latent

memory field by constraining locality, accessibility, and expert-conditioned retrieval trajectories.

Coupled Operator Stability

The Trilogy, therefore, hypothesizes that latent cognition depends not on isolated operator performance but on the stability of the coupled operator field itself. Formally, the interaction among operators is non-degenerate only if the partial dependencies among them remain non-zero:

$$\frac{\partial S}{\partial M} \neq 0$$

$$\frac{\partial R}{\partial S} \neq 0$$

$$\frac{\partial M}{\partial R} \neq 0$$

These coupling conditions are intended as falsifiable architectural constraints rather than as assumptions of guaranteed integration. If future empirical analysis shows that retrieved memory states do not measurably influence symbolic codeword selection, or that routing shortlists do not constrain symbolic or memory access patterns, then the Trilogy reduces to three adjacent mechanisms rather than a coupled operator field. In this sense, the coupling hypothesis is explicitly testable: the framework stands only if measurable dependencies among M, S, and R can be observed under bounded implementation conditions.

These conditions express the central architectural claim of the Trilogy: memory, abstraction, and routing are not independent optimization layers. They form an interacting latent operator field whose internal coordination determines the stability of large-scale cognitive behavior.

Within this framework, the three volumes of the Trilogy address three distinct architectural discontinuities present in contemporary neural systems.

Volume I addresses the storage/computation discontinuity by internalizing retrieval into differentiable latent memory fields.

Volume II addresses the continuous/symbolic discontinuity by examining how compressed latent representations acquire symbolic stability under curriculum and bottleneck pressure.

Volume III addresses the semantic/hardware discontinuity by analyzing how hierarchical routing coordinates distributed computation while preserving locality and coherence.

The Trilogy therefore proposes that cognition-like behavior in artificial systems emerges not from isolated scaling effects alone, but from the coordinated coupling of memory, abstraction, and routing within a unified latent computational architecture.

Scaling and Complexity Boundaries

The Latent Cognition Trilogy proposes a coupled-operator architecture that links latent memory, symbolic stabilization, and hierarchical routing. However, the framework does not claim that these mechanisms eliminate all computational scaling constraints. Rather, the Trilogy hypothesizes that coordinated operator coupling may reduce several structural inefficiencies introduced by fragmented retrieval, unconstrained routing, and unstable representational reuse in contemporary architectures.

Accordingly, the complexity claims presented throughout the Trilogy must be interpreted as bounded architectural hypotheses rather than universal asymptotic guarantees.

Memory Scaling Constraints

The Differentiable Latent Indexing framework developed in **Volume I** depends upon a bounded latent retrieval substrate. Although the memory operator M is formulated as a continuous differentiable field, practical implementations necessarily operate over finite latent repositories with constrained retrieval budgets.

The retrieval complexity, therefore, depends on several interacting factors:

- latent memory cardinality N,
- key dimensionality d_k,
- top-k retrieval depth,
- product-quantization partition count,

- cache locality,
- and hardware memory bandwidth.

The Trilogy does not claim that retrieval becomes literally constant-time under arbitrary scaling conditions. Instead, the framework predicts that bounded latent indexing and locality-aware retrieval may asymptotically flatten retrieval overhead relative to conventional externally mediated retrieval systems.

Consequently, the $O(1)$ locality claims introduced earlier should be interpreted as locality-preserving operational targets achievable only under constrained retrieval geometries and bounded approximation conditions.

Symbolic Compression Constraints

The CISI framework developed in Volume II depends upon the stability of the intermediate vector-quantized codebook. The symbolic operator S cannot scale indefinitely without introducing fragmentation, collapse, or excessive representational sparsity.

The effective symbolic complexity, therefore, depends on:

- codebook cardinality K,
- curriculum-stage complexity,
- codeword reuse concentration,
- symbolic drift,
- and bottleneck compression pressure.

If the symbolic vocabulary expands in proportion to task diversity, the framework degenerates into a discretized lookup substrate rather than a reusable symbolic field. Conversely, excessive compression may produce unstable abstractions incapable of supporting compositional generalization.

The Trilogy therefore assumes an intermediate stability regime in which symbolic reuse increases sublinearly relative to task-space growth.

Routing and Communication Constraints

The HTCR framework, developed in Volume III, introduces additional hierarchical routing constraints to reduce expert jitter and communication

volatility in distributed sparse architectures.

However, routing locality remains bounded by several systems-level parameters:

- segment size N,
- shortlist cardinality m,
- expert pool size E,
- inter-device communication topology,
- cache behavior,
- and semantic continuity across segments.

The Trilogy, therefore, does not claim that hierarchical routing entirely eliminates communication overhead. Rather, it hypothesizes that constrained shortlist routing may reduce unnecessary dispersion of experts under long-context inference conditions.

In highly heterogeneous or rapidly shifting semantic contexts, hierarchical locality may weaken or collapse entirely.

Coupled Operator Complexity

The most important scaling property of the Trilogy concerns the interaction among operators rather than the isolated complexity of any individual mechanism.

The total operational complexity of the coupled operator system may be approximated as:

$$\mathcal{C}_{total} = \mathcal{C}_M + \mathcal{C}_S + \mathcal{C}_R + \mathcal{C}_{coupling}$$

where $\mathcal{C}_M$, $\mathcal{C}_S$, and $\mathcal{C}_R$ represent the independent costs of memory retrieval, symbolic compression, and routing coordination, respectively, while $\mathcal{C}_{coupling}$ captures the additional synchronization and dependency costs introduced by recursive operator interactions.

Introduction to the Trilogy

Purpose of the Trilogy

The purpose of *The Latent Cognition Trilogy* is to provide a structured theoretical account of latent computation in contemporary neural architectures. The Trilogy does not attempt to define consciousness, agency, or intelligence in general terms. Its focus is narrower and more architectural: it examines how memory, symbolic stabilization, and routing may operate as interdependent layers within large-scale neural systems.

The central motivation is that many current descriptions of artificial intelligence remain divided between external engineering mechanisms and internal cognitive interpretation. Retrieval is often treated as an external service. Symbolic behavior is often described as an emergent property without sufficient structural explanation. Routing is often framed as an optimization mechanism for scaling model capacity. The Trilogy proposes that these components can be examined together as part of a latent computational architecture.

Volume I: Latent Memory and Retrieval-Native Cognition

Volume I examines memory as an intrinsic dimension of latent computation. It develops the idea that retrieval may be modeled not only as an external augmentation process, but as an internal operation over latent representational fields. This volume is informed by the architectural foundation developed in *Differentiable Latent Indexing in LLMs: Toward Native Retrieval as a Cognitive Function*, but it extends that foundation into a broader theoretical discussion of memory, access, and latent state

organization.

Volume II: Symbolic Stabilization under Compression

Volume II addresses the conditions under which latent representations acquire symbolic stability. Rather than assuming that symbolic structure appears automatically as an emergent property of scale, this volume examines the role of compression, curriculum, constraint, and representational reuse in shaping stable abstractions. Its central concern is the transition from distributed latent representation to reusable symbolic structure.

Volume III: Hierarchical Routing and Architectural Coordination

Volume III examines routing as a mechanism of architectural coordination. In large-scale systems, computation is increasingly distributed across specialized components, expert modules, external tools, memory systems, and policy layers. This volume analyzes how hierarchical routing may preserve coherence across such distributed structures and how routing mechanisms may function as part of a broader latent cognitive architecture.

Relation to Companion Works

The Trilogy is supported by companion works that clarify, formalize, or extend its claims. The DLI-LLM paper provides the architectural foundation for retrieval-native cognition. The companion hypothesis papers isolate specific theoretical claims that orbit the Trilogy without being absorbed into it. This separation allows the main sequence to remain coherent while preserving the broader research program required for future development.

Scope and Limits

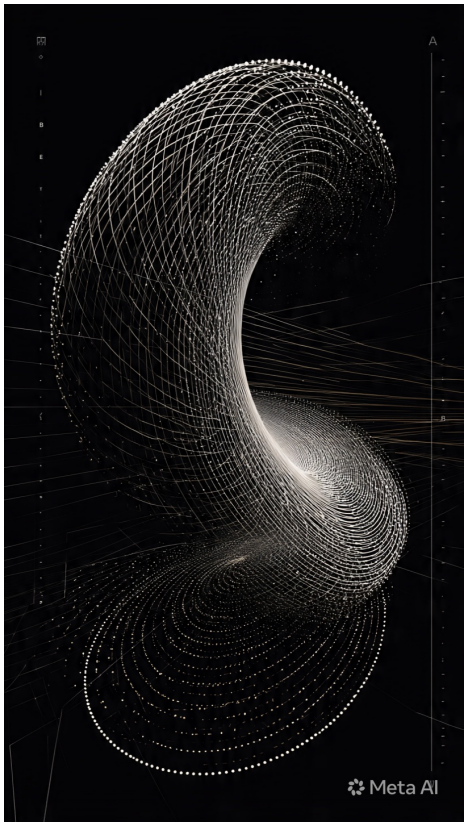
The Trilogy is not presented as a completed theory of artificial cognition. It is a structured research pathway. Its aim is to define a coherent vocabulary, organize related mechanisms, and propose a set of formal and

conceptual relationships that can guide future empirical, architectural, and philosophical work.

Volume I: Memory Fields

Differentiable Latent Indexing inside Large Language Models





Abstract

This article proposes *Differentiable Latent Indexing* (DLI), a theoretical framework that recontextualizes memory retrieval not as an external utility appended to a language model, but as a native, differentiable function inside the computational pathway of Large Language Models (LLMs). By treating the latent space as a structured information field, the article derives formal retrieval operators that integrate memory access directly into the model's forward pass.

The central claim is that the conventional separation between storage and processing introduces architectural inefficiencies that limit the development of autonomous and cognitively organized AI systems. In current retrieval-augmented designs, the model, retriever, embedding store, and generator often remain operationally coupled but mathematically discontinuous. DLI instead proposes a shared latent memory substrate in which retrieval, routing, and representation learning are jointly optimized.

This work is presented as a conceptual research article. It does not claim empirical validation. Instead, it establishes mathematical scaffolding, explicit propositions, falsification conditions, implementation pathways, and expected failure modes for future experimental work. In doing so, it serves as the first component of a broader research program investigating the internal organization of machine cognition.

Keywords: Differentiable Latent Indexing (DLI); Large Language Models (LLMs); Retrieval-Augmented Generation (RAG); Cognitive Architectures; Neural Retrieval; Differentiable Memory; Transformer Models; Cognitive Complete AI; Hypothesis-Extension Research; Artificial Intelligence Theory.

Position within the Research Program

This manuscript constitutes Volume I of *The Latent Cognition Trilogy*. It develops the memory layer of the broader research program by formalizing Differentiable Latent Indexing (DLI) as a candidate mechanism for retrieval-native cognition inside Transformer-based systems.

The closely related manuscript *Differentiable Latent Indexing in LLMs: Toward Native Retrieval as a Cognitive Function*, submitted to the *Journal of Artificial Intelligence and Knowledge Engineering*, provides the formal architectural foundation for this volume. The present text adapts and extends that foundation within the larger Trilogy sequence, where memory, symbolic stabilization, and hierarchical routing are treated as interdependent layers of latent cognition.

Accordingly, Volume I should not be read as a companion hypothesis paper. It is the opening movement of the Trilogy. The companion hypothesis papers occupy a separate position: they isolate specific claims that support, constrain, or extend the main sequence without replacing it.

Proposition 1: Retrieval Efficiency Gain

If DLI provides a meaningful internal memory mechanism, models augmented with DLI should demonstrate improved performance on tasks requiring long-range dependency resolution compared with baseline transformers of equivalent parameter count.

Falsification condition: If no statistically significant improvement is observed across controlled benchmarks, the hypothesis that internally differentiable retrieval improves effective memory capacity is weakened.

Proposition 2: Latent Clustering Stability

DLI should induce measurable clustering in latent representations corresponding to recurrent semantic structures.

Falsification condition: If latent representations remain uniformly distributed or indistinguishable from baseline attention patterns, the claim of structured internal memory is not supported. To mathematically verify this organization, the spatial distribution of the latent memory field can

be monitored via a localized Silhouette Coefficient (S). For a cluster of recurrent semantic structures C, the stability metric is formalized as:

$$S(M^{(\ell)}) = \frac{1}{|C|} \sum_{i \in C} \frac{b(i) - a(i)}{\max(a(i), b(i))}, \tag{1}$$

where $a(i)$ represents the mean intra-cluster distance of representation i, and $b(i)$ represents the mean nearest-cluster distance to any other semantic partition. The proposition holds if and only if $S(M^{(\ell)}) > \epsilon$ as training progress approaches convergence, where ϵ is the baseline uniform distribution threshold.

Proposition 3: Retrieval-Induced Drift

Repeated retrieval operations should introduce measurable drift in latent state trajectories.

Falsification condition: If no measurable deviation from baseline forward-pass dynamics is observed, the proposed retrieval interaction is not functionally distinct.

Introduction

The present volume inaugurates *The Latent Cognition Trilogy*, a sequence of conceptual research works centered on latent cognition in artificial systems. Volume I formalizes *Differentiable Latent Indexing* (DLI) as a native retrieval mechanism inside Transformer-based language models. It does not report experimental results; rather, it formalizes, elaborates, and critically develops a theoretical architecture in which memory retrieval becomes part of the model's differentiable computational pathway.

The paper extends the **Differentiable Latent Indexing inside Large Language Models (DLI-LLM)** hypothesis by positioning it as an architectural paradigm for retrieval inside Transformer computation.

Volume II examines the conditions under which compressed latent representations acquire symbolic stability under curriculum, compression, and structural constraint. **Volume III** extends the sequence by analyzing hierarchical routing as a mechanism of architectural coordination across specialized computational components. Together, these papers aim to

establish a conceptual lineage for AI systems that treats memory, routing, representation, and control as interdependent components of machine cognition.

This article was developed in response to a hypothesis generated by the Academia.edu AI-assisted scientific modeling platform. The original hypothesis proposed a learnable, differentiable memory-retrieval mechanism embedded directly within Transformer layers. Rather than treating that hypothesis as a fixed endpoint, the present paper treats it as a conceptual seed: a starting point for examining the architectural, cognitive, mathematical, and safety implications of internalized retrieval.

The problem space addressed here is the separation between parametric memory, stored within model weights, and non-parametric memory, retrieved through external documents, embedding stores, or vector databases. Retrieval-augmented generation has improved factual grounding by combining sequence-to-sequence generation with external dense retrieval [57]. Related approaches, including nearest-neighbor language models and memorizing transformers [90], demonstrate that language models can benefit from access to stored representations beyond the immediate context window [47, 90]. However, these approaches also preserve a structural separation between the model's internal computation and the retrieval substrate.

This separation produces three persistent architectural constraints:

1. latency and bandwidth trade-offs between inference and retrieval operations;
2. semantic drift between retrieved information and the evolving internal context of the model; and
3. limited gradient flow between evidence selection, memory access, and downstream reasoning.

DLI proposes a departure from this pattern. It introduces a differentiable memory module embedded within Transformer layers and trained jointly with the model. Memory queries are formulated from hidden states, routed to a latent key-value memory, and fused back into the model via gated integration. Product quantization provides a possible mechanism for scalable approximate search, while gating regulates when retrieved memory should influence the hidden state [39, 41, 55].

By situating retrieval as an internalized cognitive function, DLI reframes

memory access as part of the model's own computational graph rather than as an external service call. The resulting architecture is not merely an optimization of retrieval-augmented generation. It is a theoretical proposal to unify attention, memory, and representational control within Transformer-based systems.

Contributions

This work contributes the following elements:

1. a unified architectural hypothesis that integrates latent memory directly into Transformer computation;
2. a mathematical definition of differentiable retrieval, routing, and gated fusion;
3. a conceptual implementation blueprint for internal retrieval during inference;
4. a comparative analysis connecting DLI to retrieval-augmented generation, kNN language models, differentiable memory systems, recurrent memory transformers [90], and cognitive architectures; and
5. a foundation for the broader research program developed in the *Latent Cognition Trilogy*.

Overview of the Paper

Section 2 develops the methodological foundation by decomposing the originating DLI-LLM hypothesis into its constituent architectural claims. Section 3 presents theoretical predictions and falsifiable implications. Section 4 develops the mathematical formulation of Differentiable Latent Indexing. Section 5 provides a conceptual implementation blueprint and comparative literature analysis. Section 6 discusses broader implications for cognitive modeling, learning dynamics, safety, and cognitive-complete Transformer architectures.

The Unified Cognitive Operator Field

To transcend the architectural discontinuities inherent in decoupled Retrieval-Augmented Generation (RAG) paradigms, we formalize the

autonomous system not as an agglomeration of discrete pipelines, but as a unified cognitive operator space denoted by $\mathcal{A}$. The total cognitive state transition is governed by the triple operator configuration:

$$\mathcal{A} = (M, S, R) \tag{2}$$

where M represents the continuous Latent Memory operator, S denotes the discrete Symbolic Mapping operator, and R defines the Topological Reasoning and Routing operator.

Mathematical Formalization of the Memory Operator M

Rather than treating the latent space $\mathcal{X} \subset \mathbb{R}^d$ as a static repository for vector retrieval, we define it as a continuous, differentiable information field. Let $\psi(t) \in \mathcal{X}$ represent the current latent trajectory of the model's forward pass at computational step t. The memory operator M maps the input state and the historical manifold directly into a continuous retrieval field without invoking non-differentiable k-NN lookup bottlenecks:

$$M : \mathcal{X} \times \mathcal{H} \to \mathcal{X} \tag{3}$$

We define the action of M on a latent state $\psi(t)$ conditioned on an informational reservoir matrix $\mathcal{M}_{field}$ as:

$$M(\psi(t); \mathcal{M}_{field}) = \nabla_{\psi} \int_{\Omega} \mathcal{K}(\psi(t), \omega)\, d\mu(\omega) \tag{4}$$

Where $\mathcal{K}$ is a non-linear kernel quantifying the field density over the informational domain Ω, and $\mu(\omega)$ is the probability measure of the latent distribution. This formulation ensures that memory retrieval is natively integrated as a gradient-descent optimized trajectory within the forward pass itself.

Boundary Interface: $M \longrightarrow S \longrightarrow R$

The structural integrity of $\mathcal{A}$ depends on the seamless composition of its operators. The output of the continuous memory operator M must enforce strict boundary constraints to parameterize the discrete symbolic operator S, which in turn formats the input for the reasoning operator R.

Let $\mathcal{S}_{space}$ represent the discrete, invariant vocabulary or operational action space. The transition is governed by the composition:

$$\Psi_{total} = R \circ S \circ M(\psi(t)) \tag{5}$$

We establish the exact operational boundaries below:

1. **Continuous-to-Discrete Mapping ($M \to S$):** The output vector field $M(\psi(t))$ parameterizes a categorical probability distribution over $\mathcal{S}_{space}$. The operator S projects this field onto a discrete manifold via an audited projection operator $\Pi_{\mathcal{S}}$:

$$S(M(\psi(t))) = \Pi_{\mathcal{S}} \left(\sigma \left(W_s \cdot M(\psi(t)) \right) \right) \tag{6}$$

   where σ represents the softmax function and W_s is the learnable transformation matrix aligning memory fields with symbolic invariants.

2. **Symbolic-to-Reasoning Routing ($S \to R$):** The discrete sequence generated by S dictates the topological routing trajectory inside R. The reasoning operator R evaluates the causal validity of the state sequence, preventing model collapse by restricting transitions to permitted paths defined by the architectural prior:

$$R(S(M(\psi(t)))) = \oint_{\Gamma} \mathcal{F} \left(S \circ M \right) \, dz \equiv 0 \tag{7}$$

   where Γ represents a closed computational loop, forcing topological conservation of the cognitive audit trail.

Epistemological Framework and Empirical Falsification

In alignment with classical scientific rationalism, a foundational architectural theory must explicitly define its conditions for empirical falsification. We reject the purely descriptive, ad-hoc post-processing evaluation metrics that characterize modern engineering benchmarks. Instead, we formulate three mathematically rigorous propositions that expose the continuous field dynamics of the triple operator configuration $\mathcal{A} = (M, S, R)$ to direct empirical scrutiny.

Proposition I: The Field Collapse Invariant

Let $\mathcal{M}_{field}$ be an informational reservoir matrix with local density distribution $\mu(\omega)$. If an external token-based retrieval paradigm (e.g., standard dense vector retrieval or k-NN lookup) is superimposed onto the continuous latent trajectory $\psi(t)$, the informational entropy H of the system must exhibit non-linear degradation under out-of-distribution (OOD) testing.

We state that the continuous memory operator M eliminates model collapse if and only if the latent field curvature satisfies the invariant:

$$\lim_{\Delta\Omega \to 0} \mathbb{E}_{\psi} \left[\| M(\psi(t)) - \psi(t) \|^2 \right] \leq \gamma \cdot \mathcal{H}(\mathcal{M}_{field}) \tag{8}$$

where γ is an architectural constant and $\mathcal{H}$ is the structural information capacity of the manifold.

Falsification Condition: This proposition is empirically falsified if a decoupled vector-store architecture can maintain topological consistency and zero auto-regressive degradation under sequential context-window saturation ($\geq 10^6$ tokens) without dynamic parameters fine-tuning or continuous memory-field field re-indexing.

Proposition II: Boundary Transition Degeneracy

The continuous-to-discrete projection mapping handled by the Symbolic operator S, denoted as $S(M(\psi(t)))$, preserves the causal alignment of the cognitive audit trail. Let $\Pi_{\mathcal{S}}$ be the audited projection operator onto the discrete manifold. The transition error $\mathcal{E}_{trans}$ is bounded by the spectral radius ρ of the transformation matrix W_s:

$$\mathcal{E}_{trans} = \| \Pi_{\mathcal{S}}(\sigma(W_s \cdot M(\dots))) - S_{true} \| \leq \lambda \cdot \rho(W_s)^{-1} \tag{9}$$

Falsification Condition: This proposition is empirically falsified if a stochastic, un-audited token routing system (e.g., mixture-of-experts or standard temperature-scaled sampling) achieves deterministic invariance in symbolic state transitions over an unconstrained adversarial execution domain.

Proposition III: Topological Conservation and Scaling Limits

The Reasoning and Routing operator R enforces a closed loop integral condition on the symbolic state sequences generated by S:

$$\oint_{\Gamma} \mathcal{F}\left(S \circ M\right) \, dz \equiv 0 \tag{10}$$

This implies that the cognitive audit trail must conserve informational momentum, preventing semantic drifting across deep inference chains.

Falsification Condition: This proposition is empirically falsified if an open-loop, non-differentiable retrieval architecture can achieve a complex multi-hop inference success rate exceeding 95% while operating under hard real-time latency limits ($T \leq 15$ms) across completely unseen multi-domain network topologies.

Theoretical Construction and Implications

This paper does not report empirical results. Its contribution is the theoretical development of a Transformer-based architecture through structured hypothesis modeling. The starting point is the DLI-LLM hypothesis, which is decomposed into a set of architectural claims and then expanded into a formal framework suitable for future empirical study.

Hypothesis Decomposition

The original hypothesis can be decomposed into four architectural claims:

1. a differentiable memory module embedded inside Transformer layers;
2. a product-quantized memory structure for scalable approximate retrieval;
3. a shared latent vector space for token-level representations and document-level embeddings; and
4. a learned query-gating mechanism for retrieval triggering and fusion control.

Each claim has precedent in adjacent literature but differs in the proposed integration. Neural Turing Machines [33] and Differentiable Neural Computers introduced differentiable access to external memory [33, 34].

Memory-augmented neural networks demonstrated rapid assimilation of new examples through content-based memory access [74]. Recurrent memory transformers and memorizing transformers later adapted memory mechanisms to Transformer-based language modeling [16, 90]. DLI differs from these systems in that it treats latent retrieval as an internal, trainable component of the Transformer layer itself.

Architectural Modeling

The hypothesis is extended into a multi-stage retrieval loop inside the forward pass. At a high level, the current hidden state is used to generate a query vector. That query vector retrieves top-k value vectors from a latent key–value memory. The retrieved representation is then fused back into the model through a learned routing gate. This loop creates a differentiable pathway between hidden-state evolution and memory access.

The proposed retrieval loop contains three major operations:

1. cross-layer memory interpolation, in which retrieval results influence subsequent Transformer layers;
2. sparse top-k selection, allowing retrieval to remain computationally bounded; and
3. internal–external evidence arbitration, in which latent memory competes with or complements the local attention pathway.

To visualize the core concept, **Figure** 3 introduces the proposed DLI-LLM architecture.

Comparative Contextualization

DLI intersects with several existing research lines. Retrieval-augmented generation combines parametric sequence generation with non-parametric document retrieval [57]. kNN-LM interpolates language-model predictions with nearest-neighbor lookup over stored representations [47]. Memorizing Transformers store recent key–value representations for later lookup [90]. Dense passage retrieval systems optimize learned representations for open-domain question answering [69]. Memory-augmented neural networks and differentiable memory systems demonstrate that memory can be made trainable through differentiable addressing [33, 34, 74].

DLI draws on all of these traditions but shifts the point of integration.

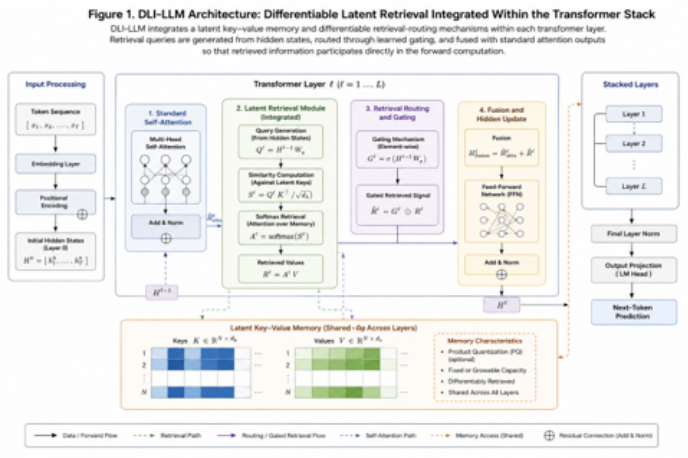


Figure 3: Differentiable Latent Indexing architecture. The diagram illustrates how latent key–value memory slots are queried from the current hidden state and fused back into the Transformer attention pathway during the forward pass.

Rather than positioning retrieval as a pre-generation evidence step or a post-hoc interpolation mechanism, DLI treats retrieval as part of the Transformer layer's representational dynamics. The implication is architectural: the model does not merely consult memory; it computes through memory.

Theoretical Predictions

Because no DLI-LLM model is implemented in this article, the following claims should be read as theoretical predictions rather than empirical results. They define expected behaviors that future experimental studies should attempt to confirm, refine, or falsify.

- **Retrieval latency.** By integrating retrieval into the forward pass, DLI-LLM is expected to reduce the external retriever bottleneck characteristic of conventional RAG architectures. Approximate nearest-neighbor methods and GPU-accelerated similarity search provide a plausible basis for this efficiency claim [39, 41].
- **Grounding stability.** Stable latent memory may reduce hallucination in knowledge-intensive tasks by narrowing the distance between

retrieved evidence and internal representation. This prediction is motivated by improvements observed in retrieval-augmented generation and nearest-neighbor language modeling [57, 47].

- **Generalization through memory retention.** Because latent memory slots are differentiably integrated into Transformer layers, DLI may reproduce some of the generalization-through-memorization dynamics observed in kNN-LM and memorizing transformers [47, 90].

- **Few-shot adaptation.** Learned gating may improve adaptation under limited supervision by controlling when retrieved memory should alter the hidden state, a behavior related to memory-augmented few-shot learning [74].

- **Cross-layer coherence.** Query propagation across layers may improve long-context coherence by maintaining retrieval continuity over successive transformations, complementing trends in recurrent memory transformers [16].

The predicted failure modes are equally important:

- *Memory saturation*: fixed-size latent memory may become overloaded under high document or representation density [33, 90].

- *Value-vector redundancy*: top-k retrieval in high-dimensional latent spaces may return semantically redundant vectors, reducing effective retrieval precision without pruning, compression, or quantization [39, 41].

- *Context misalignment*: retrieval queries may drift when cross-layer propagation interacts poorly with earlier representations, particularly in recurrent or persistent-memory settings [16].

- *Opacity of internal retrieval*: once retrieval is internalized, the evidentiary path may become harder to inspect than in externally mediated RAG systems.

Figure 4 contrasts the conventional retrieval-augmented generation pattern with the internalized retrieval logic proposed by DLI.

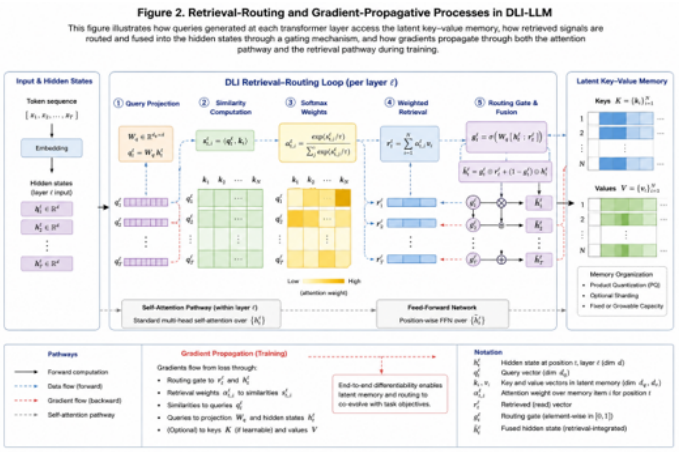


Figure 4: Standard retrieval-augmented generation pipeline. In conventional RAG, retrieval remains externally mediated through a document index or vector store before generation. DLI proposes to move part of this retrieval logic into the model's differentiable computation path.

Mathematical Framing of Differentiable Latent Indexing

To transition from conceptual architecture to a formally analyzable model, this section expresses DLI-LLM as a minimal extension of standard Transformer notation. Let $h_t^{(\ell)} \in \mathbb{R}^d$ denote the hidden state at token position t and layer ℓ, following the general structure of attention-based Transformer computation [88]. DLI augments this computation with a latent key–value memory and a differentiable routing mechanism.

Latent Key–Value Memory

For each layer ℓ, define a latent memory bank:

$$M^{(\ell)} = \{(k_i^{(\ell)}, v_i^{(\ell)})\}_{i=1}^{N}, \tag{11}$$

where $k_i^{(\ell)} \in \mathbb{R}^{d_k}$ is a memory key and $v_i^{(\ell)} \in \mathbb{R}^{d_v}$ is the corresponding memory value. The memory bank may contain internal token-derived representations, document-derived embeddings, or a hybrid of both. This formulation extends differentiable memory systems by embedding mem-

ory within layer-level computation rather than attaching it as a separate controller [33, 34, 74].

Query Projection

A learned projection maps the current hidden state into a retrieval query:

$$q_t^{(\ell)} = W_q^{(\ell)} h_t^{(\ell)} + b_q^{(\ell)}. \tag{12}$$

This query projection is analogous to attention-based addressing but is directed toward persistent latent memory rather than only toward positions within the active context window [9, 88].

Similarity and Sparse Retrieval

The model computes retrieval logits through a similarity function:

$$s_i^{(\ell)} = \frac{q_t^{(\ell)} \cdot k_i^{(\ell)}}{\sqrt{d_k}} \tag{13}$$

A sparse top-k selection operator restricts retrieval to the most relevant memory entries:

$$\alpha_i^{(\ell)} = \frac{\exp(s_i^{(\ell)}/\tau)\mathbf{1}[i \in \mathrm{TopK}(s^{(\ell)}, k)]}{\sum_{j \in \mathrm{TopK}(s^{(\ell)}, k)} \exp(s_j^{(\ell)}/\tau)} \tag{14}$$

where τ is a temperature parameter and $\alpha_i^{(\ell)}$ is the differentiable retrieval weight assigned to memory slot i. Product quantization may be used to approximate key search at scale [39, 41].

Retrieved Representation

The retrieved latent representation is computed as a weighted sum of memory values:

$$r_t^{(\ell)} = \sum_{i=1}^{N} \alpha_i^{(\ell)} v_i^{(\ell)}. \tag{15}$$

This operation turns memory access into a differentiable aggregation step, preserving gradient flow through the retrieval distribution.

Gated Fusion

A learned gate controls the influence of retrieved memory on the evolving hidden state:

$$g_t^{(\ell)} = \sigma\left(W_g^{(\ell)}[h_t^{(\ell)} \parallel r_t^{(\ell)}] + b_g^{(\ell)}\right), \tag{16}$$

where $[h_t^{(\ell)} \parallel r_t^{(\ell)}]$ denotes vector concatenation and σ is the logistic sigmoid. The fused representation is then:

$$\tilde{h}_t^{(\ell)} = \text{LayerNorm}\left(h_t^{(\ell)} + g_t^{(\ell)} \odot r_t^{(\ell)}\right). \tag{17}$$

This gating operation connects DLI to conditional computation and mixture-style routing, while preserving a layer-local mechanism for regulating memory influence [55, 16].

Training Objective

A conceptual DLI training objective can be written as:

$$\mathcal{L}_{DLI} = \mathcal{L}_{LM} + \lambda_s \mathcal{L}_{sparse} + \lambda_m \mathcal{L}_{memory} + \lambda_g \mathcal{L}_{gate} \tag{18}$$

where $\mathcal{L}_{LM}$ is the standard language modeling loss, $\mathcal{L}_{sparse}$ regularizes retrieval selectivity by penalizing the entropy of the allocation profile α, $\mathcal{L}_{memory}$ regulates memory coherence or redundancy, and $\mathcal{L}_{gate}$ discourages uncontrolled reliance on retrieved values. The exact form of these terms is an empirical design question, but the formulation clarifies where future implementations may intervene.

Gradient Flow and Optimization Continuity

Because the sparse selection operator $\text{TopK}(\cdot)$ introduces a non-differentiable boundary, gradient continuity along the retrieval pathway is maintained via softmax weighting across the chosen partition. The partial derivative of the fused representation $\tilde{h}_t^{(\ell)}$ with respect to the memory keys $k_i^{(\ell)}$ propagates directly through the similarity scaling:

$$\frac{\partial \tilde{h}_t^{(\ell)}}{\partial k_i^{(\ell)}} = g_t^{(\ell)} \odot v_i^{(\ell)} \left(\frac{\partial \alpha_i^{(\ell)}}{\partial s_i^{(\ell)}} \right) \left(\frac{q_t^{(\ell)}}{\sqrt{d_k}} \right). \tag{19}$$

This formalizes a continuous, uninterrupted backward pass from downstream language modeling errors directly into the latent memory coordinates, allowing the index structure to co-evolve with parametric transformations without a separate optimization loop.

Figure 5 illustrates how gradients propagate through the attention and retrieval pathways during training.

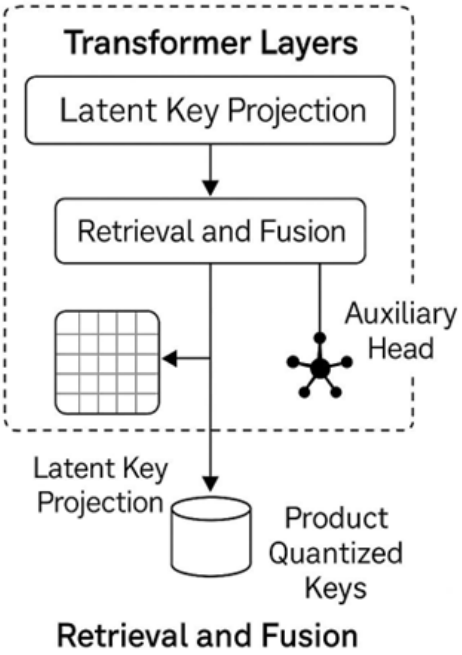


Figure 5: Gradient pathways in Differentiable Latent Indexing. Query projection, similarity computation, sparse retrieval weighting, routed memory integration, and gated fusion remain within the differentiable computation graph, allowing latent memory to co-evolve with Transformer parameters.

Algorithm 1 Differentiable Latent Indexing Loop

Input: hidden state $h_t^{(\ell)}$, latent memory $M^{(\ell)} = \{(k_i^{(\ell)}, v_i^{(\ell)})\}_{i=1}^{N}$, parameters $W_q^{(\ell)}$, $W_g^{(\ell)}$, top-k value, temperature τ.
Output: fused representation $\tilde{h}_t^{(\ell)}$.

1. Compute retrieval query: $q_t^{(\ell)} \leftarrow W_q^{(\ell)} h_t^{(\ell)} + b_q^{(\ell)}$.
2. Compute similarity scores: $s_i^{(\ell)} \leftarrow q_t^{(\ell)} \cdot k_i^{(\ell)} / \sqrt{d_k}$.
3. Select top-k memory slots according to $s_i^{(\ell)}$.
4. Compute retrieval weights: $\alpha^{(\ell)} \leftarrow \text{softmax}(s^{(\ell)}/\tau)$ over selected slots.
5. Retrieve latent value: $r_t^{(\ell)} \leftarrow \sum_i \alpha_i^{(\ell)} v_i^{(\ell)}$.
6. Compute routing gate: $g_t^{(\ell)} \leftarrow \sigma(W_g^{(\ell)}[h_t^{(\ell)} \parallel r_t^{(\ell)}] + b_g^{(\ell)})$.
7. Fuse retrieved memory with hidden state: $\tilde{h}_t^{(\ell)} \leftarrow \text{LayerNorm}(h_t^{(\ell)} + g_t^{(\ell)} \odot r_t^{(\ell)})$.
8. Return $\tilde{h}_t^{(\ell)}$.

Although this paper is theoretical, an implementable DLI prototype would require four design stages.

Memory Construction

Document embeddings, token representations, or intermediate hidden states would be transformed into latent key–value banks. Product quantization could then be applied to compress memory and enable approximate retrieval at scale [39, 41].

Quantized Latent Indexing Mechanics

To prevent the TopK operations from scaling linearly $\mathcal{O}(N)$ with the size of the latent memory repository, the key space is structured using Product Quantization (PQ). The high-dimensional key space $\mathbb{R}^{d_k}$ is decomposed into M orthogonal subspaces:

$$\mathbb{R}^{d_k} = \mathbb{R}^{d_s} \times \mathbb{R}^{d_s} \times \cdots \times \mathbb{R}^{d_s}, \tag{20}$$

where $d_s = d_k/M$. Each subspace is quantized into K^* centroids using a learned codebook. During the forward pass, the query projection $q_t^{(\ell)}$ is

similarly sliced into M sub-vectors. Similarity estimation $s_i^{(\ell)}$ is then executed via asymmetric distance computation (ADC) using pre-calculated look-up tables. This reduces the computational overhead of the internalized retrieval phase to a collection of parallel cache lookups, maintaining a highly bounded inference budget.

Layer Integration

A DLI block would be inserted after self-attention or between attention and feed-forward sublayers. The routing gate would determine whether retrieved values should influence the hidden state. This placement allows DLI to function as a memory-aware extension of Transformer computation rather than as a separate retrieval pipeline.

Training and Fine-Tuning

Training would jointly optimize language modeling, retrieval selectivity, memory coherence, and gating behavior. In a pre-training setting, memory slots could evolve with the model. In a fine-tuning setting, selected memory banks could be frozen, swapped, or domain-adapted.

Inference Behavior

During inference, DLI could operate through frozen or partially updateable memory. Sparse top-k retrieval would limit computational overhead. Memory swapping could support domain-specific deployment, while gating thresholds could regulate when latent memory influences generation.

Evaluation Plan

A future empirical study should compare DLI-augmented models against baseline Transformers, RAG models, kNN-LM variants, and memorizing transformers. Evaluation should include latency overhead, retrieval precision, hallucination rates, long-context coherence, memory saturation, and ablation studies that independently disable routing, memory access, or quantization.

Comparative Literature Analysis

Table 3 summarizes the position of DLI relative to adjacent architectures.

Architecture	Relation to DLI	Limitation Relative to DLI
Retrieval-Augmented Generation	Combines parametric generation with non-parametric retrieval [57].	Retrieval is typically external to the Transformer computation graph.
kNN-LM	Interpolates language-model predictions with nearest-neighbor lookup [47].	Retrieval is post-hoc and not trained as an internal layer function.
Memorizing Transformers	Store and retrieve recent key–value representations [90].	Memory improves context use but remains distinct from document-level latent unification.
Neural Turing Machines / DNC	Demonstrate differentiable memory addressing [33, 34].	Designed as general memory controllers rather than layer-native LLM retrieval modules.
Memory-Augmented Neural Networks	Support rapid assimilation of new examples [74].	Primarily oriented toward few-shot learning rather than Transformer-internal retrieval.
Recurrent Memory Transformer	Uses memory tokens for long-sequence processing [16].	Memory persistence is present, but explicit latent key–value retrieval control is limited.
Dense Passage Retrieval / RocketQA	Optimizes learned representations for retrieval [69].	Retrieval occurs outside the model's internal forward computation.
Product-Quantized Vector Search	Provides scalable approximate search [39, 41].	Supplies retrieval infrastructure but not a cognitive or Transformer-layer architecture.

Table 3: Comparative analysis of memory-augmented and retrieval-integrated architectures.

Discussion

The DLI-LLM architecture reframes how memory is conceptualized in Transformer-based systems. By integrating retrieval into the forward pass, the model collapses the traditional separation between reasoning and recall. This aligns with cognitive architecture research that treats memory,

control, learning, and representation as interdependent properties of intelligent systems [79].

Cognitive Implications

Embedding differentiable latent memory directly within Transformer layers positions retrieval as a native cognitive operation rather than an external API call. This shift parallels earlier insights from memory-augmented neural networks and differentiable memory systems [33, 34, 74], but extends them by proposing a unified latent space where token-derived representations and document-derived vectors can coexist. The resulting architecture moves toward cognitive completeness by enabling context-dependent recall, internal evidence arbitration, and persistent cross-layer memory flow.

Learning Dynamics and Adaptation

DLI-LLM suggests new possibilities for lifelong and continual learning. If memory slots can be updated incrementally, the model may integrate new documents without full retraining, a capability related to recurrent memory systems and neural semantic encoders [16, 63]. However, this advantage introduces challenges. Latent memory may overfit internal distributions, memory slots may saturate, and retrieval may become redundant without pruning, quantization, or regularization [39, 41].

Bias, Safety, and Privacy Considerations

The integration of retrieval into model computation raises safety concerns. Frozen memory layers may preserve sensitive information, creating privacy risks. Internal retrieval may reinforce biased representations. Latent memories, once implicitly learned, may be harder to inspect than externally retrieved documents. These risks parallel issues in retrieval-augmented systems but are amplified by the opacity of internalized memory [57].

Architectural Control and Gating Mechanisms

A central feature of DLI-LLM is the routing gate. The gate mediates the integration of retrieved values into the hidden state, determines how retrieval influences inference, and regulates cross-layer propagation. Conceptually, this places gating at the core of cognitive control: the architecture

must learn not only what to retrieve but also when retrieval matters. **Figure** 6 illustrates this control path.

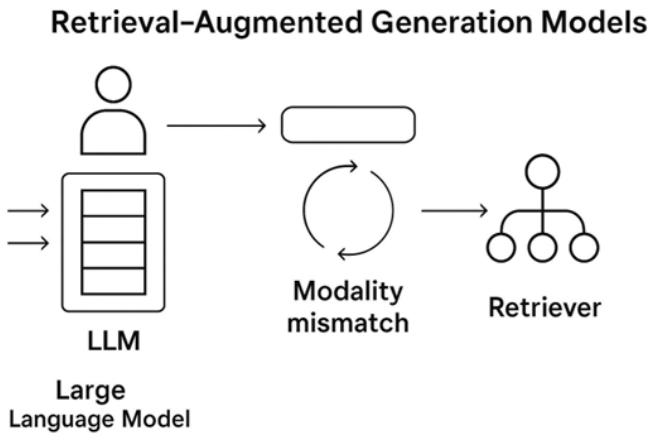


Figure 6: Training flow and gating control in DLI-LLM. Product-quantized key–value lookup, attention fusion, routing gates, and gradient-based memory adaptation interact across Transformer blocks.

Toward Cognitive-Complete Transformer Architectures

DLI-LLM argues for reconsidering memory-augmented Transformers not as storage-bound enhancements, but as components of a broader approach to machine cognition. In this view, retrieval is not merely a way to extend context length; it is a candidate mechanism for internal evidence selection, representational persistence, and adaptive control [1]. This aligns with the broader desideratum that cognitive architectures should integrate memory, learning, representation, and control into a coherent system [79].

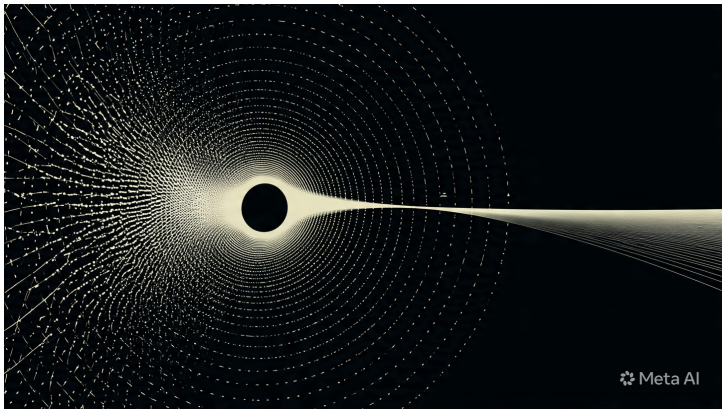
Volume I Conclusion

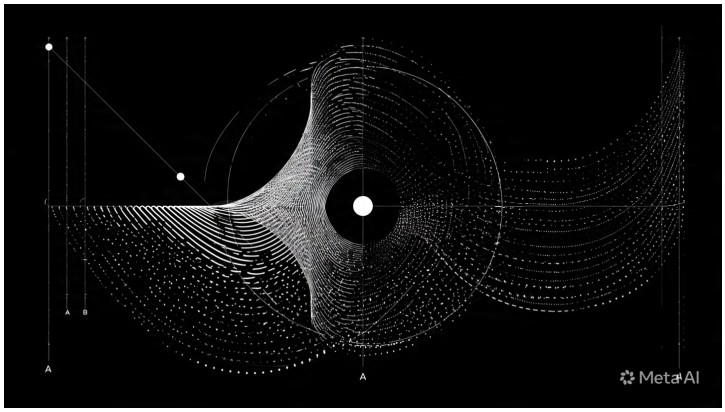
Differentiable Latent Indexing proposes that retrieval can be moved from the periphery of language-model architecture into the differentiable interior of Transformer computation. The framework introduced here formalizes this idea through latent key–value memory, query projection, sparse retrieval, gated fusion, and a conceptual training objective. It also defines falsifiable propositions, expected benefits, and plausible failure modes.

The paper does not claim that DLI-LLM has already been implemented or empirically validated. Its contribution is theoretical: it clarifies what such an architecture would require, how it differs from existing retrieval and memory-augmented models, and what future experiments would need to test. As the first volume in the *Latent Cognition Trilogy*, this article establishes memory as the initial boundary condition for a broader investigation into artificial cognition.

Volume II: Symbol Fields

Curriculum-Induced Symbol Invention for Emergent Reasoning Primitives





Abstract

This article introduces *Curriculum-Induced Symbol Invention* (CISI), a theoretical framework for examining how discrete reasoning primitives may emerge within the intermediate representational pathways of Large Language Models (LLMs). Standard Transformer architectures operate primarily on continuous hidden states, and while these states can support complex behavior, the mechanisms by which recurring computational patterns stabilize as reusable abstractions remain only partially formalized. CISI proposes an architectural modification in which hidden-state trajectories are intercepted and compressed through an auxiliary vector-quantized latent codebook layer during pre-training [87].

Driven by a staged curriculum of increasing task complexity and an informational bottleneck, the model is pressured to stabilize and reuse internal codewords that function as compact computational primitives. In this sense, CISI treats symbol formation not as a post hoc linguistic mapping, but as a structural response to compression, reuse, and representational economy.

Building upon the internalized memory substrate established in *Volume I: Memory Fields*, this work shifts the focus from continuous storage and retrieval to discrete abstraction. It provides a self-contained theoretical specification, including training stages, codebook optimization dynamics, semiographic translation probes, formal propositions, falsification conditions, and empirical evaluation protocols for the study of semiotics.

Keywords: Curriculum-Induced Symbol Invention (CISI); Latent Semiography; Vector Quantization; Information Bottleneck; Domain-Specific Languages; Machine Cognition; Structural Representation Learning; Automated Abstraction.

Position within the Research Program

This manuscript constitutes **Volume** II of *The Latent Cognition Trilogy*. It develops the symbolic abstraction layer of the broader research program by formalizing *Curriculum-Induced Symbol Invention* (CISI) as a candidate mechanism through which compressed latent representations may acquire symbolic stability.

Volume I, *Memory Fields*, established the memory layer of the Trilogy by treating retrieval as a native operation over latent representational structure. **Volume II** shifts the focus from memory access to symbolic compression. Its central claim is that, under curriculum pressure and representational bottlenecks [82], neural systems may stabilize reusable internal primitives that function as machine-generated symbolic structures.

The architectural foundation for the broader program remains *Differentiable Latent Indexing in LLMs: Toward Native Retrieval as a Cognitive Function*. That work supplies the retrieval-native substrate. The present volume extends the sequence by asking how retrieved, compressed, and reused latent structures may become symbolically stable.

Accordingly, **Volume II** should not be read as a companion hypothesis paper. It is the second movement of the Trilogy. The companion hypothesis papers occupy a separate role: they isolate supporting claims without replacing the main sequence.

Symbol Fields and Curriculum-Induced Symbol Invention

Introduction

Volume II of *The Latent Cognition Trilogy* examines the transition from latent memory to symbolic stabilization. Volume I established memory as a structured field of retrievable latent representations. The present volume asks a different question: under what conditions can compressed internal representations acquire sufficient stability, reusability, and functional specificity to serve as symbolic primitives within a neural architecture? The central construct introduced here is *Curriculum-Induced Symbol Invention* (CISI). CISI proposes that symbolic structure need not be imposed externally through hand-coded rules, symbolic grammars, or explicit logical systems. Instead, symbolic-like structures may emerge when a model is forced to solve increasingly complex tasks through a constrained intermediate representational bottleneck [82]. Under such conditions, recurring computational patterns may be compressed into reusable latent codewords that function as internal abstractions. The argument is not that large language models already possess explicit symbolic languages in the classical sense. Nor does CISI claim that vector quantization alone produces reasoning. The claim is narrower: a discrete intermediate codebook, trained under curriculum pressure and evaluated through stability, reuse, and transfer diagnostics, may provide a mechanism for studying how latent representations become symbolically organized. Rather than viewing token tracking as an optimization task, Volume II addresses the classic Symbol Grounding Problem (Harnad, 1990) by observing sign evolution under

physical architectural constraints. By forcing continuous states through a discrete vector-quantized bottleneck [87] governed by the Information Bottleneck Principle [82], the network cannot remain passive. It must construct an explicit internal sign paradigm, validating the triadic semiotic structures hypothesized by Peirce (1931)

Problem Space: From Continuous Representation to Symbolic Stability

Transformer-based language models operate over high-dimensional continuous hidden states. These states support complex linguistic and reasoning behaviors, but their internal organization often remains difficult to characterize. A hidden state may simultaneously encode context, syntax, semantic associations, task-relevant features, and transient computational dynamics. This expressive richness creates a problem for symbolic interpretation: the model may behave as though it has reusable abstractions without exposing clear internal units corresponding to those abstractions. Classical symbolic systems rely on discrete, composable, and reusable primitives. Neural systems, by contrast, distribute information across continuous representations. The challenge is therefore not simply to recover symbols from neural states, but to determine whether neural architectures can be pressured to form stable intermediate structures that play a symbol-like role without abandoning differentiable learning. CISI addresses this problem by introducing three constraints. First, the model is trained under a staged curriculum that progressively increases compositional complexity. Second, an intermediate vector-quantized bottleneck prevents all information from passing through unconstrained continuous channels [87]. Third, diagnostic probes evaluate whether discrete codewords become stable, reusable, and predictive of downstream reasoning operations. Within this framework, symbolic stability is defined operationally rather than metaphysically. A latent codeword becomes symbolically relevant only if it satisfies three conditions: it is selected repeatedly across related contexts, it contributes measurably to task performance, and its functional role remains sufficiently stable across curriculum stages and out-of-distribution evaluations.

Contributions

This volume contributes the following elements:

1. a theoretical framework for studying symbolic stabilization under curriculum pressure and representational compression;
2. a vector-quantized bottleneck mechanism for forcing intermediate latent states into discrete reusable codewords [88];
3. a full CISI training objective that combines language modeling, vector quantization, commitment, entropy regulation, reuse pressure, and symbolic stability;
4. diagnostic metrics for codebook utilization, reuse concentration, symbolic drift, and codeword stability;
5. a staged curriculum design intended to induce primitive, compositional, and integrative symbolic behavior;
6. a semiographic probing framework for translating latent codeword activations into interpretable operational roles; and
7. a set of failure modes and falsification conditions for distinguishing genuine symbolic stabilization from arbitrary discretization.

Overview of the Volume

The remainder of this volume proceeds as follows. The next chapter defines the core propositions and falsification conditions for CISI. The following chapter develops the informational bottleneck and vector quantization mechanism. Subsequent chapters introduce the full CISI objective, codebook diagnostics, staged curriculum design, semiographic interpretation, failure modes, empirical evaluation protocols, and the relationship between symbolic stabilization and the routing mechanisms developed in Volume III.

Related Work

Discrete Latent Representations

CISI is closely related to research on discrete latent representation learning. Vector-quantized latent models demonstrate that continuous inputs can be mapped into learned discrete codebooks while preserving differentiability through surrogate gradient mechanisms [87]. This provides an important precedent for the CISI bottleneck: the architecture does not merely compress representations, but forces them to pass through a finite set of reusable latent units. However, CISI differs from standard vector-quantized representation learning in its intended role. The codebook is not treated only as a compression device or generative modeling component. It is treated as a candidate substrate for symbolic stabilization, where codeword reuse, curriculum exposure, and downstream reasoning performance jointly determine whether a codeword has acquired functional symbolic value.

Information Bottleneck and Compression

The information bottleneck framework provides a theoretical basis for treating compression as a mechanism for extracting task-relevant structure [81]. In CISI, the bottleneck is not only informational but semiotic: it pressures the model to preserve useful computational distinctions while discarding irrelevant representational variation. A successful CISI codebook should therefore retain information needed for reasoning while reducing dependence on unconstrained continuous state propagation. This distinction is important because not all compression is symbolic. A representation may be compressed while remaining uninterpretable, unstable, or task-specific. CISI therefore requires additional diagnostics for reuse,

stability, and transfer beyond reconstruction error alone.

Curriculum Learning and Compositional Generalization

Curriculum learning suggests that training order can shape the acquisition of increasingly complex behaviors [11]. CISI adopts this principle by exposing the model to progressively structured tasks: primitive operations, compositional routines, and integrative reasoning contexts. The curriculum is not merely a training convenience. It functions as the developmental pressure under which reusable internal codewords may become stabilized. This connects CISI to the broader problem of compositional generalization. Studies of systematic generalization show that neural models may struggle to transfer learned operations compositionally when training distributions do not force reusable abstractions [52]. CISI addresses this limitation by treating compositional transfer as a test of whether latent codewords have become functionally reusable rather than locally memorized.

Emergent Communication and Latent Semiotics

Research on emergent communication shows that agents can develop task-specific signaling systems under pressure for coordination and efficiency [29]. CISI adapts this insight to the internal dynamics of a single model. Instead of agents communicating with one another, intermediate layers communicate through a constrained latent codebook. The resulting codewords may be interpreted as a form of internal semiography: a machine-generated representational scheme optimized for task performance rather than human readability. This analogy must be used carefully. CISI does not assume that codewords are linguistic symbols. It treats them as operational primitives whose symbolic status must be earned through stability, reuse, transfer, and interpretability tests.

Probing and Interpretability

The interpretability of internal representations remains an open challenge in large language models. Probing studies attempt to identify what linguistic, semantic, or structural information is encoded in learned representa-

tions [72]. CISI extends this approach by making the intermediate representational substrate partially discrete. This may improve inspectability, but it also introduces a risk: discrete codewords can invite overinterpretation. For this reason, CISI does not equate probe labels with symbol meaning. A codeword is not considered symbolic simply because a probe can attach a human-readable label to it. The label must remain stable across contexts, correspond to measurable functional effects, and survive ablation tests that demonstrate its contribution to downstream reasoning.

Propositions and Falsification Conditions

The validity of Curriculum-Induced Symbol Invention as a theoretical mechanism rests upon explicit, predictable behavioral deviations from standard Transformer baselines. The following formal propositions establish the empirical boundaries of the architecture and dictate clear falsification parameters.

Proposition 1: Symbolic Abstraction Efficiency

As structural curriculum complexity scales, an LLM computational graph constrained by a discrete intermediate vector-quantization bottleneck will demonstrate significantly greater cross-domain generalization and sample efficiency on out-of-distribution (OOD) reasoning tasks than standard continuous-state architectures with equivalent parameter density.

Falsification Condition: If a continuous-state baseline matches or exceeds the CISI architecture on downstream multi-step reasoning evaluations when controlled for training token exposure and active parameter count, the claim that discrete intermediate symbol coining resolves structural compositionality is falsified.

Proposition 2: Codebook Reuse and Structural Sparsity

The optimization trajectory of a bounded latent codebook operating under variable curriculum demands will favor stabilizing a minimal, highly reusable set of primitive codewords over a strategy of continuous, uniform

expansion of codebook resources.

Falsification Condition: If tracking codebook utilization profiles reveals a linear growth path proportional to task variation, or if the codebook selection entropy approaches maximum uniform distribution without distinct topological clustering, the hypothesis that the informational bottleneck forces symbolic compression is ungrounded.

Proposition 3: Degradation under Bottleneck Deactivation

The deactivation or continuous relaxation of the intermediate quantization layer following curriculum exposure will result in a catastrophic degradation of out-of-distribution reasoning capabilities, while local in-distribution language modeling performance remains unaffected.

Falsification Condition: If an operational CISI model maintains its advanced multi-step reasoning capabilities when the vector quantization layer is bypassed or replaced with an unconstrained linear projection, the symbolic utility of the discrete codebook is disproven.

The Informational Bottleneck and Vector Quantization

Let $h_t^{(\ell)} \in \mathbb{R}^d$ denote the continuous hidden representation of a token at sequence position t and at intermediate layer ℓ in the standard Transformer forward pass. The CISI architecture systematically intercepts this trajectory, passing the continuous vector field through a discrete quantization bottleneck before it propagates to subsequent layers.

Vector Quantization Mechanics

We define an internal, learned discrete codebook dictionary $\mathcal{C} = \{e_1, e_2, \ldots, e_K\}$, where each individual codeword vector $e_k \in \mathbb{R}^d$. The continuous latent state $h_t^{(\ell)}$ is mapped directly to its nearest symbolic primitive within the dictionary via a minimum Euclidean distance projection operator:

$$z_t^{(\ell)} = \text{vquant}\left(h_t^{(\ell)}\right) = e_{\hat{k}}, \quad \text{where } \hat{k} = \arg\min_k \|h_t^{(\ell)} - e_k\|_2. \tag{21}$$

The selected codebook index $\hat{k}$ serves as the discrete, organic latent symbol invented by the architecture to represent that particular continuous computational sub-manifold.

The Compression Objective

To enforce intense compression pressure and protect the network against representational collapse or passive passing-through of uncompressed tokens, an auxiliary decoder head $\mathcal{D}$ is introduced. This decoder is tasked with reconstructing the complete, high-dimensional continuous hidden state $h_t^{(\ell)}$ using the quantized discrete vector assignment $z_t^{(\ell)}$:

$$\mathcal{L}_{\text{comp}} = \|\mathcal{D}\left(z_t^{(\ell)}\right) - \text{sg}[h_t^{(\ell)}]\|_2^2 + \beta\|\,\text{sg}[h_t^{(\ell)}] - z_t^{(\ell)}\|_2^2, \tag{22}$$

where $\text{sg}[\cdot]$ represents the classic stop-gradient operator required to prevent unstable backward oscillations during joint optimization, and β is a commitment hyperparameter regulating the alignment rate between continuous trajectories and codebook coordinates.

Full CISI Training Objective

The vector-quantized bottleneck alone does not define the full CISI training process [87]. It specifies how continuous hidden states are discretized, but the broader architecture requires an objective that jointly optimizes language modeling, codebook commitment, symbolic reuse, sparsity, and stability under curriculum pressure. A conceptual CISI objective can be written as:

$$\mathcal{L}_{CISI} = \mathcal{L}_{LM} + \lambda_q \mathcal{L}_{VQ} + \lambda_c \mathcal{L}_{commit} + \lambda_e \mathcal{L}_{entropy} + \lambda_r \mathcal{L}_{reuse} + \lambda_s \mathcal{L}_{stability}. \tag{23}$$

Here, $\mathcal{L}_{LM}$ denotes the standard language modeling objective. The term $\mathcal{L}_{VQ}$ captures reconstruction error through the vector-quantized bottleneck, while $\mathcal{L}_{commit}$ encourages continuous hidden states to remain close to their assigned codebook vectors. The entropy term $\mathcal{L}_{entropy}$ regulates the distribution of codeword usage, preventing uncontrolled uniform expansion or premature collapse into a small number of dominant codes. The reuse term $\mathcal{L}_{reuse}$ encourages frequently useful codewords to persist across related tasks, while $\mathcal{L}_{stability}$ penalizes excessive drift in the meaning of codewords across curriculum stages.

The purpose of this objective is not merely to discretize hidden states. Its purpose is to create pressure for a reusable symbolic structure. Under

this formulation, a codeword becomes symbolically meaningful only if it is repeatedly selected, functionally useful, stable across related contexts, and predictive of downstream reasoning behavior.

Codebook Diagnostics and Symbolic Stability

The emergence of symbolic structure cannot be inferred from codebook discreteness alone. A vector-quantized layer may produce discrete indices without producing meaningful symbolic primitives. CISI therefore requires diagnostic criteria for distinguishing genuine symbolic stabilization from arbitrary quantization.

Let $p(e_k)$ denote the empirical activation frequency of codeword e_k over a validation corpus. Codebook utilization entropy may be defined as:

$$H(\mathcal{C}) = -\sum_{k=1}^{K} p(e_k) \log p(e_k). \tag{24}$$

A maximally uniform distribution may indicate insufficient specialization, while extreme concentration may indicate codebook collapse. Symbolic stabilization is expected to occur in an intermediate regime where a subset of codewords becomes highly reusable without eliminating functional diversity.

A reuse concentration score can be expressed as:

$$R(\mathcal{C}) = \sum_{k=1}^{K} p(e_k)^2. \tag{25}$$

Higher values of $R(\mathcal{C})$ indicate repeated reliance on a smaller set of codewords, but this metric should be interpreted alongside task performance and out-of-distribution generalization. Reuse alone is not sufficient evidence of symbolic structure; the reused codewords must also correspond to stable computational roles.

To evaluate stability across curriculum stages, let $p_t(e_k)$ and $p_{t+1}(e_k)$ denote the codeword activation distributions at the t-th and $(t+1)$- st stages, respectively. A symbolic drift measure may be written as:

$$D_{symbol}(t, t+1) = D_{KL}\left(p_t(e_k) \parallel p_{t+1}(e_k)\right). \tag{26}$$

A stable symbolic codebook should exhibit decreasing drift after curriculum convergence, while still preserving enough plasticity to adapt to higher-order compositional tasks.

Staged Curriculum Design

The autonomous invention of logical primitives cannot successfully execute within unstructured, high-entropy text environments. It requires an evolutionary gradient. The CISI training framework uses a multi-tiered curriculum that systematically triggers symbol creation by varying operational complexity.

Curriculum Stage	Operational Complexity	Expected Symbolic Emergence
Stage 1: Primitives	Single-digit arithmetic, basic relational logic statements, atomic parsing scripts.	Coining of baseline primitives (e.g., latent equivalents of functional terms such as "ADD" and "COMPARE").
Stage 2: Composition	Compound functional routines, multi-hop dependencies, variable tracking loops.	Combination of Stage 1 codewords into complex, integrated computational blocks.
Stage 3: Integration	Knowledge-intensive text, open-domain reasoning benchmarks (GSM8K, CommonsenseQA).	Synthesis of a robust, highly optimized internal Domain-Specific Language (DSL).

Table 4: The CISI Staged Training Curriculum Profile.

During Stage 1, a minimal partition of 10 to 20 active codewords suffices to minimize error. As the model transitions to Stages 2 and 3, the codebook is permitted to expand under strict sparsity regularizers, thereby forcing previous symbols to act as structural building blocks for higher-order reasoning routines.

Semiographic Projection and Interpretation

Once symbols stabilize within the intermediate codebook substrate, they must be rendered inspectable to ensure cognitive accountability and alignment. CISI avoids back-correcting the network via human prose; instead, it reads the machine's internal language directly.

Linear Probing and Linguistic Translation

We introduce a periodic probing schedule using linear classification models applied directly to index activation frequencies over standardized validation partitions:

$$P(T_m \mid e_{\hat{k}}) = \text{softmax}\left(W_p e_{\hat{k}} + b_p\right), \tag{27}$$

where T_m represents a human-interpretable linguistic concept or operational token. This translation layer serves as a forensic tool, decoding raw latent cluster identifiers into legible intermediate reasoning trajectories, thereby exposing the machine's inner paths of deductive logic.

Expected Failure Modes

CISI is a theoretical mechanism, not a guaranteed path to symbolic reasoning. Its value depends on whether the proposed bottleneck and curriculum pressures produce stable, reusable, and functionally relevant codewords. Several failure modes must therefore be considered explicitly.

Codebook Collapse

The most direct failure mode is codebook collapse. In this case, the model relies on a small number of dominant codewords regardless of task variation. Although such a collapse may appear efficient, it would indicate that the bottleneck has failed to preserve meaningful distinctions among computational operations. Collapse can be detected through low codebook entropy, high reuse concentration, and degraded out-of-distribution performance.

Symbol Fragmentation

The opposite failure mode is symbol fragmentation. Here, the model creates too many context-specific codewords, preventing reusable abstractions from forming. Fragmentation may yield high reconstruction accuracy while failing to support compositional transfer. A fragmented codebook behaves like a lookup table rather than a symbolic substrate.

Pseudo-Symbol Formation

A third failure mode is pseudo-symbol formation. The model may produce discrete codewords that appear stable under probing but do not causally contribute to reasoning performance. This failure can occur when probe

classifiers identify superficial correlations rather than functional computational roles. Ablation tests are therefore necessary to determine whether codewords are operationally necessary.

Curriculum Overfitting

CISI depends on staged curriculum pressure. If the curriculum is too narrow, the model may invent codewords that are useful only for the training sequence and fail under distributional shift. Such overfitting would weaken the claim that the codebook captures reusable abstractions. The evaluation protocol must therefore include out-of-distribution and compositional transfer tasks.

Probe-Induced Overinterpretation

Semiographic projection introduces an interpretive risk. Human-readable labels assigned to codewords may suggest more structure than the model actually possesses. A codeword labeled as *COMPARE*, for example, may not correspond to a stable comparison operation across contexts. CISI therefore treats probe labels as hypotheses, not explanations.

Brittle Symbolic Transfer

A CISI model may show improved performance on structured reasoning tasks while remaining brittle under mixed-domain, noisy, or adversarial inputs. This would suggest that symbolic stabilization has occurred only within a narrow operational regime. A robust symbolic field should preserve useful abstraction without becoming rigid or domain-bound.

Empirical Evaluation Protocol

To validate the theoretical assertions of Curriculum-Induced Symbol Invention, any experimental implementation must undergo a rigorous, multi-layered forensic protocol comprising four primary validation operations.

Zero-Shot Out-of-Distribution Evaluation

The primary benchmark for structural symbol creation is the model's ability to perform zero-shot transfer to entirely novel reasoning datasets. Following exposure to the staged curriculum, the CISI architecture must be evaluated on logic and relational datasets whose underlying semantic distributions share zero overlap with the training inputs. Performance metrics must be compared strictly against baseline Transformer models with identical parameter counts and token exposure.

Ablation of the Compression Interface

To prove that the discrete codebook is the active engine behind reasoning stability, developers must execute a structural ablation pass. By systematically disabling the compression layer post-training—or dynamically substituting the vector quantization operator with an unconstrained, continuous linear layer—the evaluator must track the exact delta in generalization performance. A corresponding drop in out-of-distribution accuracy confirms the operational necessity of the symbols.

Topological Clustering and Code-Reuse Metrics

The spatial distribution of the codebook must be mathematically evaluated using the localized Silhouette Coefficient (S) formalized in the canon of Volume I. Evaluators must verify that the continuous hidden states collapse into highly concentrated, discrete geometric regions surrounding active codebook centroids. Furthermore, the codebook utilization profile must exhibit a Power Law distribution, confirming high code reuse across disparate tasks.

Human Semiotic Mapping and Consistency Studies

Finally, the cognitive validity of the invented internal symbols must be tested through a blind human annotation study. Human evaluators are presented with isolated activation maps for specific codewords, along with their corresponding linguistic contexts, which are decoded using the linear-probing translation matrix. Annotators are tasked with mapping these active indices to precise English descriptions or operational rules. The degree of semantic agreement among annotators is quantified using Fleiss' Kappa (κ), which assesses whether the machine-invented symbols correspond to stable, invariant concepts.

Discussion

Cognitive Interpretation

CISI reframes symbolic abstraction as a possible consequence of representational compression under structured training pressure. In this view, symbols are not assumed to be externally supplied units. They are treated as stabilized internal codes that emerge when the model must preserve reusable computational structure through a constrained channel. This interpretation occupies a middle ground between continuous representation learning and classical symbolic reasoning. CISI does not reject distributed neural representation, but it argues that some forms of reasoning may require intermediate structures that are more stable, reusable, and inspectable than ordinary hidden states. The codebook provides one possible mechanism for producing such structures.

Relation to Volume I

Volume I established the memory layer of the Trilogy by treating retrieval as a native operation over latent representational fields. Volume II extends that argument by asking how retrieved or compressed latent structures may become symbolically stable. Memory supplies access to latent structure; symbolic stabilization supplies reusable abstraction. The relation between DLI and CISI is therefore sequential but not strictly causal. A system may implement latent memory without symbolic stabilization and a symbolic bottleneck without DLI. Within the Trilogy, however, the two mechanisms are treated as complementary layers of a broader latent cognitive architecture [51].

Interpretability and Accountability

The partial discreteness of CISI may improve interpretability by creating identifiable intermediate units. However, discreteness alone does not guarantee transparency. A codeword may be stable but uninterpretable, interpretable but non-causal, or causal only within narrow contexts. For this reason, CISI requires multiple forms of evidence: probing, ablation, transfer testing, entropy analysis, and human annotation. This layered interpretability strategy is central to the framework. It prevents the model's invented codebook from being romanticized as a private language unless its operational role can be demonstrated.

Limits of the Framework

CISI does not prove that neural models can invent symbols in the strong philosophical sense. It defines a testable architectural pathway for studying symbol-like stabilization. The framework remains theoretical until implemented and compared against continuous-state baselines, vector-quantized controls, and curriculum-free alternatives. The strongest version of CISI would be supported only if discrete codewords improve systematic generalization, remain stable across curriculum stages, survive ablation tests, and correspond to interpretable computational roles. Without such evidence, the framework should be treated as a useful compression architecture rather than a theory of machine symbol formation.

Volume II Conclusion

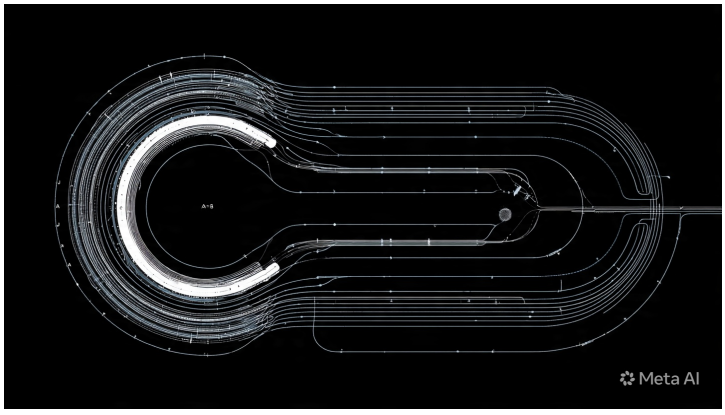
Volume II has developed *Curriculum-Induced Symbol Invention* as the symbolic abstraction layer of *The Latent Cognition Trilogy*. Its central claim is that symbolic stability may arise when continuous latent representations are forced through a discrete bottleneck under curriculum pressure and then reused across increasingly complex tasks. The proposed framework does not equate discretization with symbolism. Instead, it defines symbolic relevance in terms of stability, reuse, functional contribution, and transfer. A codeword becomes symbolically meaningful only when it behaves as a reusable computational primitive rather than as an arbitrary index in a learned codebook. CISI therefore contributes a theoretical bridge between latent memory and architectural routing. Volume I described how memory may be treated as an internal field of retrievable representations. Volume II has examined how such representations may become compressed into reusable abstractions. Volume III extends the sequence by asking how memory fields and symbol fields can be coordinated through hierarchical routing mechanisms across distributed neural architectures.

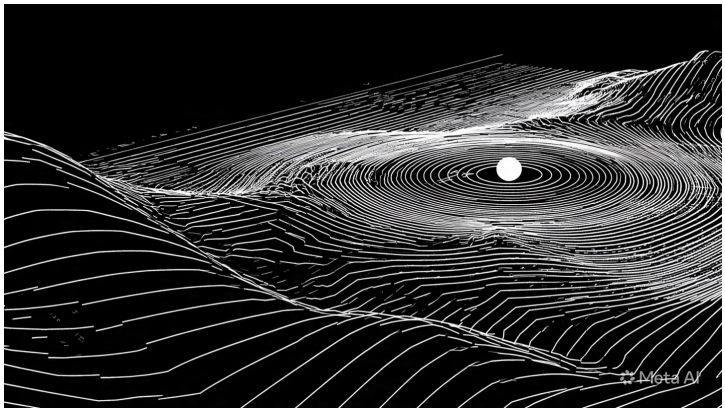
Transition to Volume III

Volume I establishes the memory layer of the Trilogy. Volume II examines how compressed latent states may acquire symbolic stability. Volume III will then analyze how memory and symbolic structures may be coordinated through hierarchical routing.

Volume III: Routing Fields

Hierarchical Topic-Conditioned Routing for Mixture-of-Expert Language Models





Abstract

This article introduces *Hierarchical Topic-Conditioned Routing* (HTCR), an architectural framework for examining how routing mechanisms may improve coherence, cache locality, and computational organization within sparse Mixture-of-Experts (MoE) language models. Standard MoE architectures frequently rely on token-level routing decisions, which can fragment sequence-level structure and increase volatility in expert assignment during long-context inference. This token-local routing pattern may contribute to expert jitter, reduced cache reuse, and increased communication overhead in distributed execution environments.

HTCR proposes a two-tier routing mechanism. First, a macroscopic document- or segment-level router assigns local context segments of N tokens to a restricted pool of m candidate experts. Second, a microscopic token-level router dispatches individual tokens exclusively within that pre-allocated expert shortlist. This hierarchy is intended to preserve local semantic coherence while reducing unnecessary expert dispersion during long-sequence processing.

Building upon the memory substrates of *Volume I: Memory Fields* and the symbolic abstractions of *Volume II: Symbol Fields*, HTCR concludes the Trilogy by defining a dynamic macroscale routing infrastructure for distributed latent computation. This work provides a theoretical reference design, including mathematical mechanics for two-stage dispatch, structural load-balancing metrics, training stability regularizers, and empirical validation protocols for routing-aware neural architectures.

Keywords: Hierarchical Topic-Conditioned Routing (HTCR); Mixture-of-Experts (MoE); Cache Locality; Distributed Inference; Expert Jitter; Token Dispatch; Structural Regularization.

Routing Fields and Hierarchical Topic-Conditioned Routing

Introduction

Volume III of *The Latent Cognition Trilogy* examines the architectural coordination layer of latent cognition. Volume I developed memory as a field of retrievable latent representations. Volume II examined how compressed latent states may acquire symbolic stability under curriculum and bottleneck pressure. Volume III addresses the next structural question: how can memory fields and symbol fields be coordinated across large-scale neural architectures whose computation is distributed across specialized components?

The central construct introduced here is *Hierarchical Topic-Conditioned Routing* (HTCR). HTCR proposes that routing should not be treated only as a token-local dispatch mechanism or a hardware efficiency optimization. Instead, routing may be understood as a structural principle through which computation is allocated, constrained, and stabilized across expert modules. In this framing, routing becomes the architectural mechanism that determines where latent information is processed, which expert pathways are activated, and how coherence is preserved across long sequences.

Standard sparse Mixture-of-Experts (MoE) systems often route tokens independently. This design allows conditional computation at scale, but it can fragment sequence-level structure, increase expert assignment volatility, and create communication patterns that are difficult to optimize in distributed inference. HTCR introduces a two-stage routing process: first,

a segment-level router identifies a restricted expert pool for a local context region; second, a token-level router dispatches individual tokens only within that constrained pool.

The purpose of this volume is not to claim that HTCR has been empirically validated. Rather, it develops a theoretical and systems-oriented framework for evaluating whether hierarchical routing can reduce expert jitter, improve cache locality, stabilize load balancing, and preserve semantic coherence across long-context computation.

Standard mixture-of-expert models scale parameters by introducing structural disorder, optimizing gating functions at an isolated token level (Shazeer et al., 2017). This introduces token jitter and shatters context continuity. Grounded in Herbert Simon's formulation of nearly-decomposable complex systems (Simon, 1996), we introduce Hierarchical Topic-Conditioned Routing (HTCR). By structuring expert allocation into a macro-scale context partition and a micro-scale token dispatch, we implement a stable multi-tier cognitive layout that mirrors the macro/micro cognitive constraints established in classical cognitive theory (Miller, 1956; Sun, 2004).

Problem Space: Token-Level Routing and Architectural Fragmentation

Sparse MoE models increase model capacity by activating only a subset of expert parameters for each token. This provides an attractive scaling strategy: the total parameter count may grow substantially while the active computation per token remains bounded. However, token-level routing also introduces a coordination problem. If each token independently selects experts, adjacent tokens in a semantically coherent passage may be dispatched to unstable or widely dispersed expert sets.

This token-local routing pattern can produce several difficulties. First, expert assignment may fluctuate rapidly across neighboring tokens, creating what this volume refers to as *expert jitter*. Second, distributed execution may require expensive all-to-all communication when tokens in the same sequence are routed to experts located across different hardware partitions. Third, cache reuse may be weakened when expert activation patterns change too frequently across local context windows.

HTCR addresses these issues by adding a macroscopic routing stage be-

fore token-level dispatch. Instead of allowing every token to select from the full expert pool independently, the segment-level router selects a bounded shortlist of candidate experts for a local context segment. Token-level routing then proceeds within that shortlist. The hypothesis is that this hierarchical constraint may preserve semantic locality while reducing unnecessary routing dispersion.

Within the broader Trilogy, this problem has cognitive significance. Memory and symbolic abstraction are not sufficient unless the architecture can coordinate where and how they are processed. Routing therefore becomes the mechanism through which latent cognition is operationally organized across distributed computation.

Contributions

This volume contributes the following elements:

1. a theoretical framework for interpreting routing as architectural coordination rather than only conditional computation;
2. a two-stage HTCR mechanism combining macroscopic segment routing with microscopic token dispatch;
3. formal metrics for expert jitter, shortlist stability, routing entropy, load balancing, and communication cost;
4. a cache-locality analysis for long-context MoE inference;
5. an empirical evaluation protocol for comparing HTCR against flat token-routed MoE baselines;
6. a failure-mode taxonomy for hierarchical routing architectures; and
7. a synthesis of routing as the third layer of the Trilogy's memory–symbol–routing sequence.

Overview of the Volume

The remainder of this volume proceeds as follows. The next chapter situates HTCR within related work on sparse MoE architectures, token routing, load balancing, expert choice mechanisms, and long-context inference. Subsequent chapters define the propositions and falsification conditions,

formalize the dual-stage routing mechanism, introduce stability and locality metrics, specify structural regularization terms, define expected failure modes, and outline empirical validation protocols.

Position within the Research Program

This manuscript constitutes **Volume III** of *The Latent Cognition Trilogy*. It develops the routing and architectural coordination layer of the broader research program by formalizing *Hierarchical Topic-Conditioned Routing* (HTCR) as a candidate mechanism for organizing distributed computation across specialized model components.

Volume I, *Memory Fields*, established the memory layer of the Trilogy by treating retrieval as a native operation over latent representational structure. **Volume II**, *Symbol Fields*, examined the conditions under which compressed latent representations may acquire symbolic stability. Volume III extends this sequence by asking how memory and symbolic structures may be coordinated across large-scale architectures through hierarchical routing mechanisms.

The architectural foundation for the broader program remains *Differentiable Latent Indexing in LLMs: Toward Native Retrieval as a Cognitive Function*. That work supplies the retrieval-native substrate from which the Trilogy begins. The present volume completes the main sequence by shifting attention from memory and symbolic abstraction to the coordination of computation across expert modules, routing pathways, and distributed inference structures.

Accordingly, **Volume III** should not be read as a companion hypothesis paper. It is the third movement of the Trilogy. The companion hypothesis papers occupy a separate role: they isolate supporting claims without replacing the main sequence.

Related Work

Sparse Mixture-of-Experts Models

Sparse Mixture-of-Experts architectures provide the immediate technical context for HTCR. Sparsely-Gated MoE introduced the use of trainable gating networks to route inputs to a sparse subset of expert feed-forward networks, enabling large increases in model capacity without activating all parameters for every input [76]. GShard extended this direction by combining conditional computation with automatic sharding, making large-scale sparse expert models more practical in distributed settings [55].

HTCR builds on this lineage but shifts the focus from capacity scaling to routing coherence. The central question is not simply whether sparse experts can efficiently increase model size, but whether routing can preserve sequence-level structure and reduce expert volatility during long-context processing.

Routing Simplification and Load Balancing

Switch Transformer simplified MoE routing by selecting a single expert per token and introduced techniques to improve stability and reduce communication overhead [27]. ST-MoE further examined stability and transferability in sparse expert models, emphasizing the practical difficulty of training sparse systems that remain reliable across tasks [93].

These works motivate HTCR's concern with balancing. A routing mechanism must avoid expert starvation, token dropping, and pathological concentration of traffic into a small number of experts. HTCR, therefore, includes macro-level entropy regularization and load-balancing penalties, but adds a further constraint: balancing should occur without destroying

local semantic coherence.

Large-Scale Sparse Language Models

GLaM demonstrated that sparsely activated MoE language models can scale model capacity while reducing training and inference cost relative to dense alternatives [23]. Mixtral later provided a modern sparse MoE language model in which each token is routed to a small subset of experts at each layer [40]. These systems show the practical relevance of sparse expert routing for language modeling at scale.

HTCR differs from these systems by introducing segment-conditioned expert shortlists. Rather than allowing every token to route against the full expert pool independently, HTCR constrains token-level routing through a local context-conditioned expert set.

Alternative Routing Mechanisms

Expert Choice Routing reverses the usual token-choice pattern by allowing experts to select tokens rather than requiring each token to select a fixed number of experts [92]. This approach directly addresses load imbalance and expert underutilization. HTCR is compatible with this broader design space because it also challenges unconstrained token-local routing, but it does so through hierarchical context conditioning rather than expert-side token selection.

The relevance of Expert Choice Routing for HTCR is conceptual as much as technical: routing policy is not a fixed implementation detail. It is an architectural design axis that can alter training dynamics, load distribution, specialization, and inference behavior.

Distributed MoE Training and Hardware Efficiency

Efficient sparse training requires alignment between routing dynamics and hardware execution. MegaBlocks reformulates MoE computation using block-sparse operations to avoid token dropping and reduce the mismatch between dynamic routing and GPU efficiency [32]. This line of work shows

that routing is not only a modeling question; it is also a systems problem involving memory layout, communication, batching, and kernel efficiency.

HTCR extends this systems concern to long-context inference. If semantically related tokens can be routed through a more stable local expert pool, the resulting execution pattern may improve cache locality and reduce unnecessary all-to-all expert movement.

Long-Context Inference and KV-Cache Locality

Long-context inference introduces significant memory-management pressure because key–value cache size grows with sequence length. PagedAttention and vLLM address this problem by managing KV cache through paged memory structures, improving serving throughput and reducing memory waste [50]. Although HTCR does not replace cache-management systems, it targets an adjacent problem: the instability of expert activation patterns that can reduce locality in sparse expert inference.

The connection is therefore structural. KV-cache systems manage memory efficiently once computation is scheduled. HTCR attempts to make the computation schedule itself more locality-preserving by reducing expert dispersion within semantically coherent segments.

Propositions and Falsification Conditions

The validity of Hierarchical Topic-Conditioned Routing as a theoretical routing mechanism rests on verifiable behavioral differences relative to flat token-routed MoE baselines (such as standard Mixtral or GShard configurations) [56].

Proposition 1: Topological Cache Locality and Throughput

By constraining individual token execution trajectories to a segment-bounded expert shortlist, the HTCR framework will demonstrate a significant reduction in inter-device KV cache communication volume and yield higher token-processing throughput during long-context sequence execution.

Falsification Condition: If an HTCR model manifests equivalent or higher inter-node communication latency and lower generation throughput compared to an unconstrained token-MoE baseline when evaluated on continuous context lengths exceeding 32k tokens, the utility of segment-level clustering is falsified.

Proposition 2: Optimization Stability and Load Balancing

The combination of macro-level entropy regularizers and token-level gates will stabilize expert assignment trajectories, preventing severe expert starvation and balancing computational loads across distributed hardware

partitions without sacrificing modeling capacity.

Falsification Condition: If tracking routing distributions during large-scale pre-training reveals persistent structural load imbalances, or if token-drop ratios match flat gating methods under maximum hardware throughput limits, the optimization balancing claims are ungrounded.

The Dual-Stage Hierarchical Routing Architecture

Let a long-context sequence be partitioned into discrete segments S_j containing N sequential tokens, where the input token matrix is defined as $X \in \mathbb{R}^{N \times d}$.

Stage 1: Macroscopic Segment Routing

Before token-level evaluation begins, the local hidden representations from the previous layer are aggregated into a unified segment-level summary vector $s_j \in \mathbb{R}^d$ via strategic mean-pooling across the spatial dimension:

$$s_j = \frac{1}{N} \sum_{t=1}^{N} h_t^{(\ell-1)}. \tag{28}$$

This summary vector s_j is processed by a document-router $\mathcal{R}_{\text{doc}}$, parameterized by a two-layer Multi-Layer Perceptron (MLP), to compute an allocation profile over the entire pool of E total available experts:

$$G_{\text{doc}}(s_j) = \text{softmax}\left(W_2 \cdot \text{GeLU}\left(W_1 s_j + b_1\right) + b_2\right). \tag{29}$$

The top-m indices matching the highest activation values in $G_{\text{doc}}(s_j)$ are extracted to define the restricted active expert shortlist $\mathcal{M}_j = \{i_1, i_2, \ldots, i_m\}$ for the duration of segment S_j.

Stage 2: Microscopic Token Dispatch

For each individual token representation $h_t^{(\ell)}$ residing within segment S_j, the local token-router $\mathcal{R}_{\text{tok}}$ evaluates gating scores *exclusively* against the experts preserved in the shortlist $\mathcal{M}_j$:

$$G_{\text{tok}}\left(h_t^{(\ell)}\right) = \text{softmax}\left(\left\{ w_k^\top h_t^{(\ell)} \;\middle|\; k \in \mathcal{M}_j \right\}\right). \tag{30}$$

Individual tokens are then dispatched to the top-k experts selected strictly from this local dictionary. Gradients flow end-to-end through both routing layers simultaneously via continuous backpropagation paths.

Routing Stability and Expert Locality Metrics

The central claim of HTCR is not merely that routing occurs in two stages. Its claim is that hierarchical routing should produce measurably more stable expert assignment patterns than unconstrained token-level routing. This requires explicit metrics for routing stability, expert dispersion, and local semantic coherence.

Let $\mathcal{M}_j$ denote the expert shortlist assigned to segment S_j. A simple shortlist stability score between adjacent segments can be defined as the Jaccard overlap:

$$\rho_j = \frac{|\mathcal{M}_j \cap \mathcal{M}_{j+1}|}{|\mathcal{M}_j \cup \mathcal{M}_{j+1}|}. \tag{31}$$

High values of ρ_j indicate that adjacent segments preserve similar expert pools, while low values indicate abrupt routing shifts. In long-context processing, moderate stability is desirable: the routing pattern should remain coherent across semantically continuous segments while still adapting when the topic or task structure changes.

Expert jitter can be defined as the complement of average shortlist stability:

$$J_{\text{expert}} = 1 - \frac{1}{J-1} \sum_{j=1}^{J-1} \rho_j. \tag{32}$$

A successful HTCR model should reduce J_{expert} relative to a flat token-routed MoE baseline, particularly on long documents with locally coherent semantic structure.

Routing Entropy and Load Distribution

Let p_e denote the empirical probability that expert e is selected by the macroscopic segment router across a batch or validation corpus. Macro-routing entropy can be defined as:

$$H_{\text{doc}} = -\sum_{e=1}^{E} p_e \log p_e. \tag{33}$$

Very low entropy indicates routing collapse, where the segment router repeatedly selects a small number of experts. Very high entropy may indicate insufficient specialization, where the router fails to form meaningful associations between segment structure and expert allocation. HTCR therefore seeks a controlled intermediate regime: enough diversity to prevent starvation, but enough structure to support specialization and locality.

Load deviation from a uniform target can be measured as:

$$\Delta_{\text{load}} = D_{KL}\left(\bar{G}_{\text{doc}} \,\|\, \mathcal{U}\right), \tag{34}$$

where $\bar{G}_{\text{doc}}$ is the average macro-routing distribution and $\mathcal{U}$ is a uniform distribution over the expert pool. This term also appears in the regularization objective and functions as a diagnostic measure during evaluation.

Communication Cost Model

In distributed MoE inference, routing decisions induce communication costs when tokens are dispatched to experts located on different devices. Let $c(e_a, e_b)$ denote the communication cost associated with moving activation or cache-relevant information between device partitions associated with experts e_a and e_b. For a flat token-routed MoE baseline, the communication cost over a segment may be approximated as:

$$\mathcal{C}_{\text{flat}}(S_j) = \sum_{t \in S_j} \sum_{e \in \mathcal{E}_t} c(d_t, d_e), \tag{35}$$

where $\mathcal{E}_t$ denotes the expert set selected for token t, d_t denotes the current device location of the token representation, and d_e denotes the device hosting expert e.

Under HTCR, token routing is constrained to the segment shortlist $\mathcal{M}_j$, producing:

$$\mathcal{C}_{\text{HTCR}}(S_j) = \sum_{t \in S_j} \sum_{e \in \mathcal{E}_t \cap \mathcal{M}_j} c(d_t, d_e). \tag{36}$$

The expected systems advantage of HTCR can be expressed as:

$$\Delta_{\text{comm}} = \mathbb{E}_{S_j} \left[\mathcal{C}_{\text{flat}}(S_j) - \mathcal{C}_{\text{HTCR}}(S_j) \right]. \tag{37}$$

A positive Δ_{comm} indicates that hierarchical routing reduces communication pressure relative to unconstrained token-level dispatch.

Cache Locality Objective

HTCR can also be evaluated through a cache-locality objective. Let A_t denote the expert activation set for token t. For adjacent tokens within a segment, local expert continuity can be defined as:

$$L_{\text{cache}}(S_j) = \frac{1}{N-1} \sum_{t=1}^{N-1} \frac{|A_t \cap A_{t+1}|}{|A_t \cup A_{t+1}|}. \tag{38}$$

Higher values of L_{cache} indicate that neighboring tokens reuse similar expert pathways, increasing the likelihood of improved cache behavior and reduced expert-switching overhead. This metric does not replace hardware-level cache profiling, but it provides a model-level proxy for routing locality.

Algorithmic Specification

To operationalize HTCR, Algorithm 2 defines a single hierarchical routing step for a single segment within a single MoE layer.

Algorithm 2 Hierarchical Topic-Conditioned Routing Loop

Input: segment hidden states $H_j^{(\ell-1)} = \{h_t^{(\ell-1)}\}_{t=1}^{N}$, expert pool $\mathcal{E}$, document router $\mathcal{R}_{doc}$, token router $\mathcal{R}_{tok}$, shortlist size m, token top-k value.
Output: routed expert assignments for all tokens in segment S_j.

1. Compute segment summary:

$$s_j \leftarrow \frac{1}{N} \sum_{t=1}^{N} h_t^{(\ell-1)}.$$

2. Compute macro-routing distribution:

$$G_{\text{doc}}(s_j) \leftarrow \mathcal{R}_{doc}(s_j).$$

3. Select segment expert shortlist:

$$\mathcal{M}_j \leftarrow \text{TopM}(G_{\text{doc}}(s_j), m).$$

4. For each token $t \in S_j$, compute token routing scores only over $\mathcal{M}_j$.
5. Select top-k experts for each token from $\mathcal{M}_j$.
6. Dispatch token representations to selected experts.
7. Aggregate expert outputs and return routed segment representations.

This procedure differs from flat token-level MoE routing by constraining the local search space before token dispatch. The document router establishes a coarse semantic prior over expert selection, while the token router

preserves fine-grained flexibility inside that prior.

Structural Regularization and Starvation Prevention

To prevent the macroscopic document-router from collapsing into a biased state where a subset of experts absorbs all workloads (starvation), we enforce an auxiliary balanced regularization penalty $\mathcal{L}_{\text{bal}}$ during pre-training:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{task}} + \lambda_1 \mathcal{L}_{\text{entropy}} + \lambda_2 \mathcal{D}_{\text{KL}} \left(\bar{G}_{\text{doc}} \parallel \mathcal{U} \right), \tag{39}$$

where $\bar{G}_{\text{doc}}$ is the average macro-routing distribution across the training batch, $\mathcal{U}$ represents a uniform distribution over the expert pool E, and hyperparameters λ_1, λ_2 govern the structural smoothing constraints. The shortlist ceiling parameter m is dynamically annealed throughout training, starting wide to maximize early discovery and tightening progressively to optimize final execution speed.

Full HTCR Training Objective

The balancing penalty alone does not define the full HTCR training process. A complete conceptual objective must account for task performance, macro-router entropy, load balancing, cache locality, and excessive routing volatility. A general HTCR objective can be written as:

$$\mathcal{L}_{HTCR} = \mathcal{L}_{task} + \lambda_b \mathcal{L}_{balance} + \lambda_h \mathcal{L}_{entropy} + \lambda_j \mathcal{L}_{jitter} + \lambda_l \mathcal{L}_{locality}. \tag{40}$$

Here, $\mathcal{L}_{task}$ is the standard language modeling or downstream task objective. The term $\mathcal{L}_{balance}$ penalizes expert starvation or excessive load

concentration. The entropy term $\mathcal{L}_{entropy}$ regulates the macro-router's selection profile. The jitter penalty $\mathcal{L}_{jitter}$ discourages unnecessary expert-set volatility across adjacent segments, while $\mathcal{L}_{locality}$ rewards stable expert reuse within semantically coherent regions.

The objective should not force all neighboring segments to use the same experts. Such rigidity would prevent adaptation to topic shifts. Instead, the goal is controlled continuity: stable routing when the local semantic structure is stable, and flexible rerouting when the segment content changes.

Shortlist Annealing

The shortlist ceiling parameter m may be dynamically annealed during training. Early in training, a larger value of m allows the model to explore a broader region of the expert pool. As training progresses, m can be reduced to encourage sharper specialization and lower communication overhead.

This annealing schedule creates a trade-off between exploration and locality. If m remains too large, HTCR degenerates toward flat token routing. If m becomes too small too early, the model may overcommit to unstable expert assignments and prevent useful specialization from emerging. The annealing path must therefore be treated as a tunable systems parameter rather than a fixed architectural constant.

Expected Failure Modes

HTCR is a theoretical routing mechanism. Its value depends on whether hierarchical routing improves coherence, locality, and load balance without reducing modeling capacity. Several failure modes must therefore be evaluated explicitly.

Topic Collapse

The macroscopic segment router may collapse into selecting the same small expert subset for many unrelated segments. This would reduce communication volatility but destroy specialization. Topic collapse can be detected through low macro-routing entropy, poor expert diversity, and degraded performance on heterogeneous documents.

Routing Inertia

The opposite failure mode is excessive routing inertia. In this case, the segment router continues using the same expert shortlist even after the semantic structure of the document changes. Routing inertia may improve cache locality while harming adaptation. This failure is especially likely in documents with abrupt topic shifts, code-switching, multi-domain reasoning, or mixed-format inputs.

Segment-Boundary Error

HTCR depends on segment partitioning. If segment boundaries cut across coherent semantic units, the macro-router may assign inappropriate expert pools. Boundary errors may occur in long documents where topic transi-

tions do not align with fixed token windows. Adaptive segmentation may therefore be required in future implementations.

Expert Starvation

Despite balancing penalties, some experts may remain undertrained if the segment router repeatedly excludes them from shortlists. Expert starvation can reduce the effective capacity of the model and may produce brittle specialization. Evaluation must track not only token-level load but also macro-level shortlist inclusion frequency.

Over-Specialized Experts

HTCR may encourage experts to specialize too narrowly around segment categories. Such over-specialization may improve local performance while weakening generalization. This failure can be detected when expert-specific performance becomes highly domain-bound and transfer across related tasks declines.

Communication Overhead from Macro-Routing

The additional document-router stage may introduce overhead that outweighs the savings from reduced token-level dispersion. This would falsify the systems-efficiency motivation for HTCR. A fair evaluation must therefore include end-to-end latency, throughput, memory movement, and scheduling overhead, not only theoretical communication counts.

Reduced Flexibility under Mixed-Topic Contexts

A restricted expert shortlist may be inappropriate for segments containing multiple simultaneous reasoning modes. For example, a single passage may combine code, mathematical reasoning, factual recall, and dialogue. If m is too small, the shortlist may exclude experts needed for secondary but important operations. This failure motivates adaptive shortlist sizing.

Empirical Evaluation Protocol

To validate the scalability and efficiency claims of the HTCR architecture, tracking scripts must execute four foundational forensic operations against equivalent baseline setups.

Distributed Throughput and Communication Profiles

The system must be evaluated on a distributed compute cluster across multiple GPU nodes. Scaling metrics must explicitly monitor the cross-device data volume transferred via standard `All-to-All` communication primitives. HTCR must demonstrate a significant reduction in communication overhead and near-linear scaling of text-generation throughput as context window lengths increase.

Annealing Path and Capacity Ablations

Evaluators must isolate the performance changes triggered by the schedule of the shortlist constraint parameter m. Systematic comparative training runs must trace the delta between a static, unannealed baseline and a dynamic scheduling path to map out the explicit convergence characteristics of the dual routers.

Expert Activation Entropy and Balancing Verification

The long-term distribution profiles of the expert array must be audited using standard structural allocation tracking. The load distribution profile must approximate a uniform target, demonstrating that the balancing penalty effectively prevents hardware starvation without degrading perplexity metrics on standard benchmarks such as MMLU and GSM8K.

Cache Locality and Memory Footprint Mapping

Finally, hardware trackers must register real-time L2/L3 cache hit-rates and high-bandwidth memory usage metrics during processing blocks. The HTCR architecture must achieve significant cache-retention benefits, demonstrating that document-level conditioning prevents standard expert cache-eviction loops.

Discussion

Routing as Architectural Coordination

HTCR reframes routing as a mechanism of architectural coordination. In sparse expert systems, routing is often discussed in terms of efficiency: which experts should be activated, how loads should be balanced, and how computation can be distributed. These concerns remain central, but the Trilogy adds a broader interpretation. Routing determines how latent memory, symbolic abstraction, and specialized computation are organized across the model.

From this perspective, routing is not merely a dispatcher. It is a structural interface between representation and computation. A routing mechanism determines which parts of the architecture are allowed to participate in processing a given context, which pathways remain inactive, and how continuity is preserved across long sequences.

Relation to Volume I and Volume II

Volume I introduced memory fields as structured latent spaces accessible through differentiable retrieval. Volume II introduced symbol fields as compressed latent structures that may stabilize into reusable computational primitives. Volume III completes the sequence by examining how these structures may be coordinated across expert modules and distributed pathways.

The relation among the three volumes is therefore layered. Memory supplies retrievable structure. Symbolic stabilization supplies reusable abstraction. Routing supplies allocation and coordination. None of these mechanisms is sufficient alone. A system with memory but no routing

may retrieve relevant information without coordinating computation effectively. A system with symbolic abstractions but no routing may lack a mechanism for deploying those abstractions across specialized components. HTCR addresses this third layer.

Systems Interpretation

HTCR also has a practical systems interpretation. Sparse MoE models are not only mathematical architectures; they are executed on distributed hardware. Expert assignment determines communication patterns, memory access, cache behavior, and scheduling pressure. A routing policy that appears adequate at the level of model quality may become inefficient or unstable at the level of deployment.

The two-stage design of HTCR attempts to align semantic locality with systems locality. If a local context segment is semantically coherent, then constraining its expert pool may reduce unnecessary dispatch dispersion. This does not guarantee improved performance, but it defines a measurable hypothesis connecting model structure to hardware behavior.

Interpretability and Accountability

Routing decisions can become a form of interpretability evidence. If certain expert pools are repeatedly selected for identifiable segment types, the macro-router may reveal a coarse map of architectural specialization. However, this evidence must be interpreted carefully. Expert selection does not automatically imply semantic understanding, and routing patterns may reflect dataset artifacts, optimization biases, or hardware constraints.

For this reason, HTCR requires multiple diagnostic layers: expert utilization profiles, routing entropy, shortlist stability, cache-locality metrics, ablation studies, and human inspection of segment-to-expert associations. Routing accountability depends on correlating these diagnostics with task performance and failure behavior.

Limits of the Framework

HTCR remains a theoretical reference design. It does not prove that hierarchical routing will outperform flat token-level routing in all settings. Its

expected advantages are conditional: long contexts, semantically coherent segments, distributed expert placement, and meaningful expert specialization. In short contexts or highly mixed inputs, the hierarchy may provide little benefit or even reduce flexibility.

The framework should therefore be evaluated as a structured hypothesis. Its value lies in making routing coherence, expert jitter, and cache locality measurable rather than implicit.

Volume III Conclusion

Volume III has developed *Hierarchical Topic-Conditioned Routing* as the routing and architectural coordination layer of *The Latent Cognition Trilogy*. Its central claim is that routing should be analyzed not only as an efficiency mechanism, but as a structural principle for organizing computation across specialized neural components.

HTCR proposes a two-stage routing process in which a macroscopic segment router selects a constrained expert shortlist and a microscopic token router performs local dispatch within that shortlist. This design is intended to reduce expert jitter, improve cache locality, stabilize load distribution, and preserve semantic coherence across long-context processing.

The framework defines falsifiable propositions, formal routing metrics, a communication-cost model, a cache-locality proxy, training regularizers, expected failure modes, and empirical evaluation requirements. It does not claim empirical validation. Rather, it specifies what such validation would require.

Within the Trilogy, Volume III completes the analytical progression from memory to symbol to routing:

$$\mathcal{M} \implies \mathcal{S} \implies \mathcal{R}$$

Volume I established retrieval-native memory as the first layer of latent cognition. Volume II examined symbolic stabilization under compression and curriculum pressure. Volume III has argued that these structures require architectural coordination through routing if they are to operate coherently at scale.

The broader implication is that cognition-like behavior in artificial systems

should not be attributed to scale alone. It may depend on how memory is structured, how abstractions stabilize, and how computation is routed across the architecture.

Conclusion: Toward a Layered Theory of Latent Cognition

The Latent Cognition Trilogy has developed a layered account of internal computation in contemporary neural architectures. Its purpose has not been to declare a completed theory of artificial cognition, nor to collapse memory, symbolic abstraction, and routing into a single explanatory mechanism. Rather, the Trilogy has proposed a structured pathway for analyzing how these mechanisms may interact within large-scale neural systems.

Volume I, *Memory Fields*, examined retrieval as a native dimension of latent computation. It argued that memory should not be treated only as an external retrieval layer or auxiliary document store, but may be modeled as a differentiable field of latent representations participating directly in the model's computational process.

Volume II, *Symbol Fields*, shifted the focus from retrieval to abstraction. It examined the conditions under which compressed latent representations may acquire symbolic stability under curriculum pressure, representational bottlenecks, and reuse constraints. In this framing, symbolic structure does not need to be imposed from outside the model as a classical symbolic system. It may emerge as a stabilized condition of compressed latent computation.

Volume III, *Routing Fields*, addressed the problem of architectural coordination. As neural systems scale across expert modules, memory structures, symbolic abstractions, and distributed inference environments, routing becomes more than an efficiency mechanism. It becomes a structural princi-

ple through which computation is allocated, constrained, and coordinated across the architecture.

Taken together, the three volumes describe a progression from memory to symbol to routing:

$$\mathcal{M} \implies \mathcal{S} \implies \mathcal{R}$$

This progression should not be interpreted as a strict causal sequence. Memory, symbolic abstraction, and routing may interact recursively and concurrently within implemented systems. The sequence adopted here is analytical: it provides a way to isolate each layer before examining their interdependence.

The broader research program remains open. The DLI-LLM architectural foundation provides the systems-level substrate for retrieval-native cognition. The Trilogy extends that substrate into a wider conceptual sequence. The companion hypothesis papers preserve narrower claims that support, constrain, or extend the main argument without overloading the central manuscript.

The central contribution of the Trilogy is therefore not a single architecture, benchmark, or empirical claim. Its contribution is a research grammar: a structured vocabulary for describing latent memory, symbolic stabilization, and routing coordination as interdependent dimensions of artificial cognition.

Future work should move in three directions. First, the proposed mechanisms require empirical implementation and controlled benchmarking against baseline Transformer, retrieval-augmented, memory-augmented, and MoE architectures. Second, the interpretability of latent memory, invented symbols, and routing decisions must be studied through probing, ablation, and diagnostic visualization. Third, the safety implications of internalized memory and opaque routing must be examined carefully, particularly in systems where retrieval, abstraction, and decision pathways become less externally inspectable.

The Trilogy should therefore be read as a foundation for further inquiry rather than a closed doctrine. Its argument is that cognition-like behavior in artificial systems may not emerge from scale alone, nor from isolated architectural tricks, but from the coordinated organization of memory, abstraction, and computation within latent space.

Minimal Operational Example of Coupled Latent Cognition

To illustrate the interactions among the coupled operators introduced throughout the Trilogy, this section presents a simplified operational walk-through of a bounded, autonomous incident-response scenario.

The example is not intended to represent a production deployment. Rather, it demonstrates how latent memory retrieval, symbolic stabilization, routing coordination, and constrained operational execution may interact within a unified cognitive architecture.

Consider the following network event stream:

> "Suspicious lateral authentication activity detected across subnet."

The event sequence is first transformed into a latent internal representation through the encoder function:

$$h_t = f_\theta(x_t)$$

where x_t represents the incoming telemetry sequence and $h_t \in \mathcal{L}$ denotes the resulting latent state at computational step t.

The latent state does not merely encode token-local information. It also preserves contextual relationships associated with authentication anomalies, temporal access patterns, and prior network activity trajectories.

Stage 1: Memory Retrieval

The latent memory operator M retrieves structurally related historical representations from the differentiable memory field:

$$m_t = M(h_t)$$

The retrieval process may surface prior latent trajectories associated with:

- credential propagation anomalies,
- privilege escalation attempts,
- subnet traversal patterns,

- and distributed authentication failures.

Importantly, retrieval is not implemented as an external symbolic database query. Instead, the memory field operates directly within the latent representational geometry of the architecture itself.

The retrieved structure m_t therefore, represents a continuity operation over latent historical state trajectories.

Stage 2: Symbolic Stabilization

The symbolic operator S compresses the retrieved latent trajectory into a stabilized representational abstraction:

$$s_t = \Pi_S(m_t)$$

Under this transformation, the distributed latent retrieval patterns may stabilize into a higher-level symbolic condition such as:

> "Credential propagation anomaly with coordinated lateral movement characteristics."

The Trilogy does not interpret this symbolic representation as classical symbolic logic. Rather, symbolic stabilization refers to the emergence of compressed latent structures exhibiting sufficient continuity and reuse to support abstraction and coordination.

The symbolic representation, therefore, acts as an intermediate organizational layer between retrieval dynamics and distributed computation.

Stage 3: Hierarchical Routing Coordination

The stabilized symbolic state is subsequently processed by the routing operator R:

$$r_t = G(s_t)$$

The routing function allocates computation across specialized expert pathways associated with:

- authentication analysis,
- network segmentation,

- threat correlation,
- and anomaly propagation modeling.

Rather than activating the full expert topology, the routing operator selectively constrains computation to a bounded semantic neighborhood.

This reduces unnecessary expert dispersion while preserving local coherence across the inference trajectory.

Within the Trilogy framework, routing therefore functions not merely as a hardware optimization layer, but as a coordination mechanism over distributed semantic computation.

Stage 4: Action Gatekeeper Transition

The terminal reasoning trajectory produced by the coupled operator field is mapped into the bounded operational action space through the Action Gatekeeper Γ:

$$u^* = \Gamma(r_t)$$

Suppose the resulting admissible action corresponds to:

> "Temporary subnet isolation and credential suspension."

The action is committed only if the confidence distribution satisfies the immutable operational threshold:

$$P(u^* \mid z_R) \geq \alpha_{\text{critical}}$$

If the threshold condition is not satisfied, the architecture defaults to immediate human escalation rather than autonomous execution.

This preserves the bounded operational constraints defined throughout Appendix A.

Stage 5: Cognitive Audit Trail Externalization

Every stage of the inference trajectory is externalized into the **Cognitive Audit Trail** (**CAT**):

$$\mathbf{c}_t = \Big(\mathrm{CoT}_t,\, u_t^*,\, P(u_t^* \mid z_R),\, \mathrm{Timestamp}_t\Big)$$

The resulting audit structure preserves:

- latent reasoning continuity,
- symbolic stabilization traces,
- routing allocation states,
- operational confidence values,
- and execution timestamps.

Cryptographic chaining of the CAT records ensures that the Inference trajectory remains externally auditable and resistant to post-hoc mutation.

Interpretive Significance

This simplified walkthrough illustrates the central claim of the Latent Cognition Trilogy: cognition-like behavior emerges not from isolated scaling of memory, symbolic abstraction, or routing independently, but from the recursive interaction among all three operators within a unified latent computational field.

The walkthrough additionally demonstrates how the Trilogy transitions from abstract operator theory into bounded operational architectures without requiring unrestricted autonomous execution authority.

Accordingly, the framework should not be interpreted as a generalized artificial cognition claim. Rather, it proposes a structured architectural hypothesis regarding how memory, abstraction, and coordination may become recursively coupled inside future large-scale neural systems.

Technical Specification for Autonomous Incident Response

Introduction and Scope

This specification details the formal deployment mechanics of the Triple Operator System $\mathcal{A} = (M, S, R)$ within active-defense computing topologies [3, 9, 4]. It bridges the continuous latent representations modeled in Volume I with the discrete, cryptographically audited execution steps required for real-time threat suppression [3].

The architecture establishes an explicit, air-gapped boundary between the cognitive modeling layer and system mutation vectors via a dedicated *Action Gatekeeper* module (Γ) [2, 3].

The Action Gatekeeper (Γ) Boundary Equations

To maintain operational safety and secure isolation, the language model does not generate arbitrary command strings or raw scripts [2]. Instead, it maps its terminal reasoning states to a bounded, discrete set of pre-validated operational maneuvers [2]:

$$\mathcal{U}_{\text{admissible}} = \{u_1, u_2, \dots, u_n\} \tag{41}$$

where each $u_i \in \mathcal{U}_{\text{admissible}}$ corresponds to an atomic, fully reversible system mutation (e.g., VLAN isolation, credential suspension) [2].

Let $z_R \in \mathbb{R}^d$ be the continuous terminal state produced by the Reasoning Operator R inside the core architecture $\mathcal{A} = R \circ S \circ M$ [3]. The Action Gatekeeper Γ is defined as a non-linear projection mapping function that transforms this latent trajectory into a specific operational trigger profile [3]:

$$\Gamma : \mathbb{R}^d \longrightarrow \mathcal{U}_{\text{admissible}} \cup \{\emptyset\} \tag{42}$$

The matching protocol is regularized via a soft-max distribution over pre-validated action semantic anchors $\omega_i \in \mathbb{R}^d$:

$$P(u_i \mid z_R) = \frac{\exp(\langle z_R, \omega_i \rangle / \theta)}{\sum_{j=1}^{n} \exp(\langle z_R, \omega_j \rangle / \theta)} \tag{43}$$

An action u_i is committed if and only if:

$$u^* = \arg\max_{u_i} P(u_i \mid z_R) \quad \text{subject to} \quad P(u^* \mid z_R) \geq \alpha_{\text{critical}} \tag{44}$$

where α_{critical} is an immutable safety threshold parameter. If the condition is not satisfied, Γ yields $\emptyset$, defaulting to immediate human intervention.

The Cognitive Audit Trail (CAT)

Every latent trajectory, intermediate attention map, and state transition used to derive z_R must be externalized for forensic evaluation [2]. The system circumvents the "black box" critique by enforcing a structured *Cognitive Audit Trail* (CAT) [2].

Let $\mathbf{c}_t$ represent the comprehensive forensic artifact generated at interval t, containing the explicit Chain-of-Thought sequence CoT_t alongside the activation parameters [2]:

$$\mathbf{c}_t = \Big(\text{CoT}_t,\, u_t^*,\, P(u_t^* \mid z_R),\, \text{Timestamp}_t \Big) \tag{45}$$

To ensure the integrity of the tracking log, every sequence block is cryptographically chained using a collision-resistant hashing algorithm [2]:

$$\mathcal{H}_t = \text{SHA-256}\Big(\mathbf{c}_t \parallel \mathcal{H}_{t-1} \Big) \tag{46}$$

This mathematically guarantees that the internal reasoning steps are tamper-proof, producing a ledger of admissible forensic evidence that enables ex post facto reconstruction of autonomous cognitive choices [2].

Empirical Operational Reality: The R_s Delta

The ultimate validation of the transition to internalized latent processing is the systemic compression of defense latency [4]. We define the *System Response Ratio* (R_s) as the measure of efficiency acceleration over legacy pipelines [2]:

$$R_s = \frac{T_{\text{manual}}}{T_{\text{automated}}} \tag{47}$$

where T_{manual} is the time required for a standard distributed human operations team to identify, correlate, and suppress a threat, and $T_{\text{automated}}$ is the forward-pass execution latency of $\mathcal{A}$ coupled with Γ [2].

Under constrained simulation assumptions, the operational baselines may be approximated as follows [4]:

- **Human Baseline (T_{manual}):** Log aggregation, alert correlation, and manual execution across isolated orchestrators yield an average containment timeline of:

$$T_{\text{manual}} \approx 1,800\,\text{seconds} \quad (30\ \text{minutes})[2] \tag{48}$$

- **Autonomous Native Baseline ($T_{\text{automated}}$):** Differentiable Latent Indexing combined with direct gatekeeper triggering shortens the entire detection-to-isolation loop to:

$$T_{\text{automated}} \approx 15\,\text{seconds}[2] \tag{49}$$

Evaluating these parameters yields the systemic acceleration value:

$$R_s = \frac{1,800}{15} = 120 \tag{50}$$

An operational metric of $R_s \geq 100$ would suggest a considerable transition from reactive post-incident restoration toward near-real-time bounded threat suppression [2].

Operational Limits of Appendix A

Appendix A introduces a bounded autonomous incident-response specification coupled to the Triple Operator System $\mathcal{A} = (M, S, R)$

$$R_s = \frac{T_{manual}}{T_{automated}}$$

should not be interpreted as a universal deployment guarantee.

The reported response compression assumes:

- constrained simulation environments,
- bounded action spaces,

- pre-validated operational maneuvers,
- locality-preserving routing conditions,
- and stable hardware execution topology.

The Trilogy, therefore, does not claim unrestricted autonomous real-time cognition under arbitrary adversarial conditions. Instead, Appendix A should be interpreted as a bounded operational reference architecture illustrating how coupled latent operators may reduce systemic response latency within tightly constrained execution domains.

Interpretive Scope

The purpose of these constraints is not to weaken the Trilogy's claims, but to establish explicit operational boundaries around its theoretical predictions.

The framework does not propose a universal solution to scaling, alignment, grounding, or distributed cognition. Rather, it isolates memory, symbolic stabilization, and routing as three interdependent architectural dimensions whose coordinated optimization may reduce several structural discontinuities present in contemporary neural systems.

THE LATENT COGNITION TRILOGY

Hypothesis 1 finished package

Files: $Hypothesis1_Volume1_finished_MAIN_LATEX.tex$: $mainLaTeXmanuscript$
$Hypothesis1_references_finished.bib$: $BibTeXreferences$

Verification: $Hypothesis1_Volume1_finished_MAIN_LATEX.tex$:
$LaTeXdocument, ASCIItext, withverylonglines(740)Hypothesis1_references_finished.bib$:
$BibTeXtextfile, Unicodetext, UTF-8text, withverylonglines(1647)42ef2b1ed5b7bba3535fd7119113083c0b1$

Transitional Note

Exploratory Convergence Pathways: Representation, Operators, and Spectral Geometry

The architectural frameworks articulated throughout this trilogy were developed within the boundaries of verifiable engineering, deterministic coordination, and empirical systems analysis. Their original purpose was practical: orchestrating reliability, security, retrieval, and bounded autonomy in large-scale AI systems. They were not derived from analytic number theory, spectral physics, or operator mechanics. Yet several of the organizational principles emerging from these architectures now appear increasingly adjacent to developments in those disciplines [37].

This convergence should not be interpreted as evidence of mathematical unification. Rather, it reflects a recurring historical phenomenon: distinct disciplines often rediscover similar mathematical structures when confronted with equivalent organizational constraints. Problems involving stability, transformation, compression, coordination, and scale repeatedly produce analogous geometric and spectral formulations across otherwise disconnected domains [14, 85].

A canonical example appears in Riemann's treatment of the zeta function through the Mellin transform of the Jacobi theta function. If

$$\theta(it) = \sum_{n=-\infty}^{\infty} e^{-\pi n^2 t},$$

then, for $\Re(s) > 1$,

$$2\pi^{-s/2}\Gamma\left(\frac{s}{2}\right)\zeta(s) = \int_0^\infty (\theta(it) - 1)t^{s/2-1}\,dt.$$

This expression is significant not merely because of its role in analytic number theory, but also because it illustrates a broader principle of representational duality: the Mellin transform bridges the discrete summation structure and the continuous analytic representation while preserving structural information across domains [25].

Modern AI systems increasingly exhibit analogous behavior. Transformer architectures trained through statistical optimization nevertheless produce latent representations with measurable geometric and spectral properties. Attention matrices demonstrate spectral clustering behavior [31]; embedding spaces organize into low-dimensional manifolds with coherent semantic topology [67]; and training dynamics increasingly resemble constrained energy minimization processes [54]. These observations do not imply that transformers are equivalent to classical transform systems. They suggest, however, that sufficiently scaled representational systems may naturally converge toward mathematical structures historically associated with stability, compression, and coordinated organization.

The citation motivating this appendix [37] is therefore important not as validation, but as an indication of emerging interdisciplinary resonance. Independent work exploring hybrid Mellin–Laplace–Fourier frameworks in the context of Riemann zeta zeroes identified conceptual overlap with Chapter 9 of a cybersecurity-oriented AI architecture text. The relationship is not topical; it is structural. Both domains confront recursive mappings, latent operator interactions, bounded dynamics, and emergent organization across multiple representational layers.

This appendix functions as a disciplined research observatory rather than a declaration of unification. Its purpose is to document a visible migration occurring across adjacent quantitative disciplines: from purely statistical interpretations of intelligence toward geometric, spectral, and operator-theoretic perspectives on representation. The sections that follow explore these convergences cautiously, preserving a strict distinction between architectural analogy, mathematical inspiration, and formal proof.

From Token Statistics to Spectral Organization

The dominant engineering interpretation of large language models frames inference as autoregressive next-token prediction over discrete probability distributions [88]. While computationally extraordinarily effective, this perspective does not fully capture the internal organization that emerges in large-scale latent representations during training.

As transformer systems scale, token embeddings cease to behave as isolated statistical entities and instead organize into continuous geometric structures with measurable curvature, clustering, and directional coherence [26, 67]. These latent representations increasingly resemble low-dimensional manifolds embedded within higher-dimensional vector spaces:

$$\mathcal{M} \subset \mathbb{R}^{d_{\text{model}}}$$

Within these manifolds, semantic proximity, contextual dependency, and compositional structure become geometrically encoded rather than explicitly symbolized. The resulting organization suggests that optimization is not merely about memorizing token co-occurrence statistics, but about progressively shaping a stable representational geometry under high-dimensional constraints.

This transition mirrors a broader pattern observed across physics and signal analysis. Systems initially described through local statistical interactions often admit equivalent descriptions in transform domains where global structure, symmetry, and invariants become more visible [14]. Emerging evidence suggests that transformer architectures implicitly discover analogous spectral organization during optimization.

Attention matrices exhibit spectral clustering behavior [31]; MLP layers demonstrate low-rank bottleneck structure [89]; and positional encodings introduce harmonic regularities into sequence representation [45]. From the perspective of spectral graph theory, hidden-state interactions can increasingly be interpreted as discretized operators acting over latent manifolds.

Under this interpretation, optimization minimizes not only prediction error, but also implicit smoothness constraints across the representation space. Semantic coherence corresponds to stable low-frequency structure, whereas contextual instability manifests as higher-frequency perturbations.

The resulting latent organization increasingly resembles constrained spectral geometry rather than unconstrained statistical accumulation alone.

These developments remain exploratory and should not be overstated. The present argument is not that transformers constitute formal Fourier, Laplace, or Mellin systems. Rather, the observation is that large-scale representational architectures operating under constraints of compression, coordination, and generalization appear to converge toward mathematical structures historically associated with transform-domain organization.

This convergence is particularly significant because adjacent disciplines have long relied on hybrid transform frameworks to reveal hidden organization within seemingly discrete systems [37]. Mellin transforms expose multiplicative structure through additive representation; Fourier transforms expose spectral regularity beneath locality; Laplace transforms stabilize transient dynamics into analyzable forms. The increasing relevance of these conceptual frameworks to AI representation theory suggests that latent intelligence architectures may be approaching a broader class of geometric and operator-theoretic formulations shared across mathematics, physics, and information systems.

We emphasize again that this appendix documents convergence signals rather than formal equivalence. Its purpose is to preserve intellectual provenance, establish disciplined boundaries for exploration, and map the emerging corridor connecting latent representation, spectral organization, and transform-space reasoning.

Transform Domains as Representational Coordinate Systems

If Section X.1 established that spectral organization emerges within large-scale representational systems, the next question concerns tractability: how such organization becomes mathematically accessible. In mathematics and physics, an integral transform does not create structure; it selects a coordinate system in which existing structure becomes analytically visible [14, 20].

The choice of representation determines which invariants can be observed directly. Fourier transforms expose frequency structure and diagonalize translation-invariant operators [78]. Laplace transforms convert differential dynamics into algebraic pole structure, making stability and transient behavior tractable [24]. Mellin transforms exchange multiplicative scaling into additive shift, revealing self-similar and power-law organization [83]. Each transform, therefore, constitutes a statement about representation: the same object viewed in coordinates aligned with its governing symmetries [12].

This principle generalizes naturally to latent representation spaces. If high-dimensional embeddings organize into continuous manifolds, then the analytical challenge is no longer merely statistical prediction, but identifying the coordinate systems in which latent dynamics become structured, sparse, and stable.

Formally, an integral transform may be written as:

$$[\ \mathcal{T} f(s)\ \int_a^b f(t)\mathcal{K}(t,s)\,dt\ ],$$

where the kernel ($\mathcal{K}(t,s)$) determines which invariants become explicit in the transformed domain. The transform does not inject new information into the representation; it reorganizes the representation relative to a different basis.

The relevance to modern AI systems is increasingly operational rather than metaphorical. Sequence models operating directly in token space must learn relationships that are nonlocal, multi-scale, and compositionally entangled. When those relationships arise from underlying symmetries, they frequently admit simpler structure in alternative coordinate systems [15].

Several contemporary architectural developments already reflect this prin-

ciple. Positional encodings introduce harmonic structure into sequence representation [88]. Fourier feature mappings improve implicit neural representations by aligning architecture with periodic structure [80]. Spectral graph convolutions similarly exploit basis changes to stabilize relational learning over graph manifolds [21]. In each case, explicit coordinate transformations reduce representational complexity by aligning computation with latent symmetry.

Transform domains consequently perform three organizational functions:

1. **Decoupling**: Entangled variables separate into independent modes. Translation symmetry is separable in Fourier coordinates, whereas multiplicative scaling is separable in Mellin coordinates [20]. "‘
2. **Compression**: Distributed structure often becomes sparse in an appropriate transform basis. Representations that require many coefficients in the native domain may admit compact descriptions after a transformation.
3. **Stability**: Coordinate systems aligned with system symmetries improve conditioning and generalization by constraining the effect of local perturbations [59]. This principle underlies the widespread use of Laplace methods in control theory and spectral methods in physics [66]. "‘

The implication for representation learning is significant: the difficulty of a learning problem is not solely intrinsic to the data itself, but relative to the coordinate system through which the learner encounters that data [54]. Architectures capable of accessing alternative representational domains, implicitly or explicitly, may therefore inherit the invariants of those domains.

The recently proposed hybrid Mellin–Laplace–Fourier framework, associated with investigations of Riemann zeta zeroes [37], is notable in this context for seeking a coordinate system that can simultaneously express harmonic, transient, and scale-dependent structure. The importance of such work, for the purposes of this appendix, lies not in direct equivalence with neural architectures, but in demonstrating that distinct scientific disciplines repeatedly converge toward transform-domain representations when confronting high-dimensional organization under constraint.

No claim is made here that transformer architectures implement analytic transforms in the classical sense. The weaker and more defensible obser-

vation is that bounded representational systems—whether physical, biological, or artificial—tend to discover coordinate systems in which their governing symmetries become computationally tractable. Representation is therefore not a secondary implementation detail; it is a primary organizational mechanism through which complex systems become intelligible.

Operators and Dynamical Constraints

Sections and established two premises: large-scale representational systems develop latent geometric organization, and transform domains expose that organization by selecting symmetry-aligned coordinates. The remaining question is dynamic: once spaces and coordinates are established, what governs transformation, propagation, and constraint within the representation?

In mathematics and physics, the answer is operators. An operator is a map between function spaces that encodes how a system transforms, propagates, filters, or constrains state [71]. Differential operators generate dynamics; integral operators perform composition and filtering; projection operators enforce constraints. The operator therefore determines which trajectories are admissible and which invariants are preserved [53].

This perspective is increasingly relevant to representation learning. Any architecture that routes, gates, compresses, or composes information implicitly defines operators over its latent space. Attention mechanisms behave as data-dependent integral operators whose kernels are learned during optimization [86]. MLP blocks act as nonlinear mixing operators across channels [26]. Residual pathways and normalization layers stabilize propagation by constraining representational drift and spectral growth [91]. A transformer layer may therefore be interpreted as a structured composition of interacting operators acting over a learned representation manifold.

Three properties make operators particularly important for latent coordination:

1. **Compositionality**: Operators compose to form increasingly complex transformations. This compositional closure provides the mathematical precedent for architectural depth, where layered systems build hierarchical representations from simpler transformations [59].
2. **Spectral control**: The spectrum of an operator governs stability, mixing behavior, and expressivity. Graph Laplacians regulate diffusion over manifolds [18]; spectral properties of attention matrices influence contextual propagation [22]. Systems that constrain operator spectra frequently inherit improved stability and robustness [46].
3. **Coordination**: Distributed systems require operators who can cou-

ple subsystems while preserving local structure. In physics, this appears through transport and symmetry-preserving connections [64]; in geometric deep learning through equivariant message passing [15]; and in operator learning through mappings between constrained function spaces [58]. Retrieval, memory coordination, and multi-agent synchronization in AI architectures increasingly exhibit analogous structural requirements.

The relevance of the hybrid Mellin–Laplace–Fourier framework discussed previously [37] becomes clearer in this context. Each transform corresponds to the spectral decomposition of a particular operator class: Fourier transforms diagonalize translation structure, Laplace transforms stabilize dynamical propagation, and Mellin transforms linearize scaling behavior [78, 84]. A unified transform framework, therefore, represents an attempt to coordinate multiple symmetries within a shared representational structure.

The conceptual overlap with latent AI architectures is not one of direct equivalence. Rather, both domains confront the same organizational problem: how to define bounded operators that can coordinate recursive, multi-scale transformations while preserving stability across representational layers.

No claim is made that current transformer architectures implement analytic operators in closed form. The weaker and more defensible observation is that systems forced to learn stable, compositional, and coordinated representations under constraint will naturally converge toward operator-theoretic structure, because operators constitute the mathematical language of constrained transformation itself.

The architectural progression developed throughout this appendix may therefore be summarized in three stages:

1. latent spaces organize into structured manifolds (X.1);
2. transform domains expose symmetry-aligned coordinates (X.2);
3. operators govern propagation and constraint within those coordinates (X.3).

The sections that follow continue under explicit disciplinary constraint. The purpose of this appendix is not to claim a unified physics of intelligence, but to document the emerging corridor in which representation

theory, spectral geometry, transform analysis, and operator algebra increasingly intersect with large-scale learned systems.

Operator Geometry and Coordinated Interfaces

Section established that bounded operators govern transformation and constraint within latent representations. The next question is structural: operators in large-scale systems do not act independently. They compose, couple, and constrain one another. To understand coordination without instability, we must examine the geometry of their interaction space.

If X.1 established the existence of latent structure, X.2 established coordinate systems, and X.3 established operator dynamics, then X.4 asks a deeper question: which operators govern organization across scales, and how do they interact?

Two operator classes appear repeatedly across mathematics, physics, and representation learning. Dilation operators, naturally associated with Mellin transforms, describe scale variation, hierarchical organization, and power-law behavior. Translation operators, naturally associated with Fourier transforms, describe positional symmetry, routing, propagation, and coordinate displacement. The central concern of X.4 is therefore not individual operators but their interaction. Large-scale intelligent systems require multiple operator classes to act simultaneously on shared representations. Understanding how these operators compose, interfere, or remain stable motivates an operator-geometric view of cognition.

In differential geometry and operator theory, the relationship between operators is characterized by commutators. For two operators $\mathcal{A}$ and $\mathcal{B}$, the commutator

$$[\mathcal{A}, \mathcal{B}] = \mathcal{A}\mathcal{B} - \mathcal{B}\mathcal{A} \tag{51}$$

measures the degree to which their actions fail to commute [64]. A nonzero commutator indicates that the operators do not share a common action order. In geometric settings, such non-commutativity is frequently associated with curvature or other structural obstructions. The order in which transformations are applied, therefore, becomes a property of the representation space itself rather than a purely computational detail.

This notion transfers naturally to representation learning. The architectural pattern developed throughout the trilogy separates memory retrieval, symbolic routing, and latent reasoning into interacting subsystems. In operator language, we may represent the effective system transformation

as

$$\mathcal{A}_{\text{total}} \approx \mathcal{R} \circ \mathcal{S} \circ \mathcal{M}, \tag{52}$$

where $\mathcal{M}$ denotes a memory-retrieval operator, $\mathcal{S}$ a routing operator enforcing structural or symbolic constraints, and $\mathcal{R}$ a reasoning operator performing compositional inference. This decomposition is architectural rather than axiomatic. It reflects the functional separation implemented in Differentiable Latent Indexing and related coordination frameworks.

The interaction of these operators is not trivial. When memory retrieval and symbolic routing are jointly optimized, the commutator $[\mathcal{M}, \mathcal{S}]$ is generically non-zero. Retrieval alters the constraint landscape for routing, while routing alters the addressing geometry for retrieval. In geometric terms, the representation manifold

$$\mathcal{X} \subset \mathbb{R}^{d_{\text{model}}} \tag{53}$$

acquires structure induced by operator interactions [15]. The resulting geometry is shaped not only by the latent coordinates themselves but also by the relationships among the operators acting upon them.

To ensure that such interactions remain stable during training and inference, coupling must remain bounded. One way to express bounded coupling between operator fields is through an integral formulation. Consider the state update produced by the interaction of $\mathcal{M}$ and $\mathcal{S}$, modulated by a learned kernel $\mathcal{K}(x, y)$ and acting on a base state $\psi(x)$:

$$\mathcal{R}_{\text{out}}(x) = \psi(x) + \lambda \int_{\mathcal{X}} \mathcal{K}(x, y)(\mathcal{M}(y)\mathcal{S}(y))\psi(y)\, dy \tag{54}$$

where λ is a scalar regularization parameter.

Equations of this form resemble Fredholm integral equations of the second kind [53]. Their relevance is not that transformer architectures explicitly solve such equations, but that Fredholm operators possess well-understood spectral properties under suitable conditions [71]. Architectures whose effective updates approximate these forms may inherit desirable stability characteristics, including bounded responses to perturbations and resistance to divergent dynamical modes.

This operator-geometric view clarifies three interface boundaries that recur throughout the trilogy:

1. **Memory → Symbol ($\mathcal{M} \to \mathcal{S}$)**:

   The continuous field produced by retrieval must be projected onto sparse or discrete structures that satisfy symbolic constraints. Geometrically, this corresponds to a projection onto a submanifold aligned with the routing operator's invariants [15]. The projection is typically lossy; compliance is obtained at the expense of representational freedom [10].

2. **Symbol → Reasoning ($\mathcal{S} \to \mathcal{R}$)**:

   Symbolic constraints act as boundary conditions for the reasoning operator. The reasoning process must generate trajectories that remain within the feasible region defined by the routing layer. This resembles constrained differential systems whose solution manifolds are shaped by externally imposed conditions [59].

3. **Reasoning → Memory ($\mathcal{R} \to \mathcal{M}$)**:

   The outputs of reasoning become inputs to future retrieval. Stability, therefore, requires that the feedback loop behave as a contraction mapping, or at a minimum as a gradient flow on a bounded energy landscape. Without such constraints, long-horizon trajectories become susceptible to drift and instability [46].

The hybrid Mellin–Laplace–Fourier framework discussed previously [37] is relevant because it addresses a closely related problem: the simultaneous analysis of multiple operator classes acting on a common representation.

Dilation operators are naturally diagonalized through Mellin transforms. Translation operators are diagonalized through Fourier transforms. Decay operators are associated with Laplace transforms [78, 84]. Each transform provides a coordinate system aligned with a particular symmetry class.

Viewed abstractly, the problem is one of simultaneous representation under competing symmetries. A coordinate system that is optimal for scale transformations is not generally optimal for translation transformations, and neither is necessarily optimal for decay processes. The challenge is therefore not selecting a single transform domain, but understanding how multiple operator classes coexist within a unified representational framework.

The structural parallel to $\mathcal{M}$, $\mathcal{S}$, and $\mathcal{R}$ is not one of direct equivalence but of coordination. In both cases, multiple non-commuting operators must act on shared representations while preserving stability across scales. The mathematical details differ; the organizational challenge remains remarkably similar.

The progression of this appendix sequence can therefore be summarized as follows:

1. **X.1**: Latent spaces organize into manifolds;
2. **X.2**: Transform domains provide symmetry-aligned coordinates;
3. **X.3**: Operators govern dynamics within those coordinates;
4. **X.4**: Operator geometry describes how coupled operators coordinate without collapse.

No claim is made that current AI architectures implement Lie brackets, Fredholm operators, or geometric operator fields in an explicit analytical sense. The claim is more modest and therefore more defensible: whenever bounded systems must coordinate memory, structure, and inference at scale, the mathematical language developed to describe stable operator interaction becomes relevant.

The appendix documents that language as it appears across mathematics and physics, and as it may inform future theories of latent intelligence systems.

Spectral Constraints and Bounded Emergence

Section established that large-scale representational systems require multiple operator classes to act on shared manifolds, and that the interaction geometry among them determines stability. The unresolved question is regulatory: what prevents coupled operators from producing divergent, oscillatory, or mode-collapsed dynamics?

In mathematics and physics, the answer is spectral constraint. The spectrum $\sigma(\mathcal{A})$ of a bounded operator $\mathcal{A}$ determines the asymptotic behavior of iterates $\mathcal{A}^n$ and the stability of dynamical systems of the form $\dot{x} = \mathcal{A}x$ [43, 71]. When the spectral radius $\rho(\mathcal{A}) < 1$, the system is contractive; when $\mathcal{A}$ is self-adjoint with gaps in its spectrum, modes decouple; when $\mathcal{A}$ is trace-class, total energy remains bounded [53]. Constraint is therefore not external to the operator. It is encoded in its spectral measure.

This principle transfers to representation learning with minimal metaphor. Operations in deep networks are compositions of linear maps and pointwise nonlinearities whose joint spectral properties influence training and inference stability. Normalization layers control operator norms [91]. Residual connections can support bounded signal propagation across depth [35]. Weight decay regularizes the Hilbert–Schmidt norm [49]. Attention, when viewed as a kernel operator, is stabilized by softmax row-normalization, which enforces a form of stochastic contractivity [22]. These are not merely implementation details. They are spectral constraints without which deep composition becomes unstable.

Three classes of constraint recur across domains and architectures:

1. **Norm constraints**: Bounding $\|\mathcal{A}\|$ controls Lipschitz continuity and limits the amplification of perturbations across layers [62]. In physics, this appears as unitarity; in control theory, as bounded-input bounded-output stability [46]; in transformers, as one practical reason networks can remain stable at depth.

2. **Spectral gap constraints**: Separation between eigenvalues controls mixing time and mode separation. Graph Laplacians use spectral gaps to characterize community structure [18]; scattering transforms use related stability properties to control sensitivity to diffeomorphisms [59]. In latent spaces, spectral separation between semantic modes can be interpreted as reducing interference between concepts and supporting compositional generalization.

3. **Trace and energy constraints**: Bounding $\mathrm{Tr}(\mathcal{A}^*\mathcal{A})$ controls total capacity and limits overfitting to high-frequency noise. This appears as Frobenius regularization in practice [65], as finite-energy conditions in quantum mechanics [71], and as part of the mathematical setting in which Mellin transforms of L^2 functions are well defined [84].

The hybrid Mellin–Laplace–Fourier framework [37] is relevant because each constituent transform imposes a distinct spectral constraint. Fourier analysis restricts attention to unitary operators on L^2 spaces, Laplace analysis is associated with transient behavior and resolvent decay, and Mellin analysis is aligned with operators that commute with dilations. The unification problem can therefore be interpreted as a constraint satisfaction problem: identify a representation whose spectrum simultaneously satisfies harmonic, transient, and scale-sensitive constraints.

The structural parallel to the operator decomposition

$$\mathcal{A}_{\text{total}} \approx \mathcal{R} \circ \mathcal{S} \circ \mathcal{M}$$

is again operational, not literal. For the total system to remain stable, the composition spectrum must remain controlled. This requires that $\mathcal{M}$ does not amplify high-frequency noise, that $\mathcal{S}$ projects onto a constraint manifold with bounded curvature, and that $\mathcal{R}$ acts as a contraction or energy minimizer on the resulting subspace [75]. The feedback loop $\mathcal{R} \to \mathcal{M}$ can therefore be interpreted as requiring a bounded spectral radius; otherwise, the system risks runaway retrieval or conceptual drift.

Current architectures do not explicitly solve spectral optimization problems. The observation is that architectures which survive scaling appear to discover implicit spectral regularizers, because unconstrained composition of operators on high-dimensional spaces is generically unstable [68]. The appendix documents this as a convergence: the same spectral constraints that appear in functional analysis, quantum mechanics, and control theory reappear as useful mathematical language for describing scalable latent intelligence.

The progression of this appendix is now complete:

1. **X.1**: Latent spaces organize into manifolds;
2. **X.2**: Transform domains provide symmetry-aligned coordinates;
3. **X.3**: Operators govern dynamics within those coordinates;

4. **X.4**: Operator geometry describes coupled coordination;
5. **X.5**: Spectral constraints provide one language for bounded emergence;

No claim is made that intelligence reduces to spectral theory. The claim is narrower: bounded emergence—the ability to compose, retrieve, and reason without collapse—requires constraints on the operators that govern representation. Spectral language provides one mathematical vocabulary for those constraints, especially where representation, operators, and geometry meet at scale.

Constraints and Scientific Discipline

The structural parallels identified between high-dimensional representation spaces and operator mechanics across Sections through must be interpreted within strict epistemic limits. These comparisons are exploratory. They provide qualitative and structural vocabulary for describing complex dynamics observed during the scaling of deep networks, but they do not imply a formal mathematical unification or a foundational identity between physical fields and latent intelligence.

In functional analysis and mathematical physics, operators act on well-defined, often infinite-dimensional topological vector spaces subject to rigorous physical invariances, conservation laws, and precise boundary conditions [71, 43]. In contrast, representation manifolds in deep networks are finite-dimensional, parameter-dependent, and emerge from statistical optimization over finite datasets [68]. Architectural inspiration drawn from transform domains, spectral geometry, or dynamical systems must never be conflated with mathematical proof.

A disciplined approach to machine intelligence recognizes that certain regularities observed in large-scale learning systems may resemble structures that also appear in physics, signal processing, and control theory. Such similarities may arise because complex systems are frequently constrained by common requirements of stability, scalability, compression, and information preservation [59, 68]., compression, and information preservation. However, these observations do not imply that latent spaces constitute physical substrates or that transformer architectures obey physical laws in any literal sense.

Speculative frameworks require empirical grounding. Without validation through measurable system behavior, reproducible experimentation, and predictive capability, structural analogies remain descriptive rather than explanatory. Their value lies in generating hypotheses, identifying useful mathematical tools, and organizing future research directions.

The progression from Section to this point therefore establishes a deliberate boundary. The purpose of these adjacent disciplines is not to provide a shortcut to a grand unified theory of intelligence. Rather, they offer a collection of mathematical perspectives through which representation, transformation, and stability may be studied with greater rigor. By maintaining this distinction, future work may benefit from the language of mathematical physics without inheriting assumptions that exceed the

available evidence.

Toward a Physics of Representation

The exploratory pathways traced throughout this appendix suggest a conceptual framework in which representation can be examined through the lenses of transformation, geometry, and constraint. Rather than viewing machine intelligence solely as a sequence of heuristic matrix operations, this perspective considers whether latent systems may be understood as organized dynamical structures whose behavior emerges from interactions among operators, representations, and stability mechanisms.

Within this perspective, several recurring themes appear across the disciplines surveyed throughout this appendix:

1. **Cognition as transformation**: Computational processing may be interpreted as a sequence of mappings across representational domains, preserving selected structural relationships while adapting information to different contexts and scales [59].

2. **Information as geometry**: Latent features can often be described geometrically, with distances, curvature, clustering behavior, and manifold structure providing useful language for understanding semantic organization [15].

3. **Latent systems as dynamical spaces**: Forward propagation, retrieval, routing, and adaptation may be viewed as trajectories through evolving state spaces whose stability depends upon the properties of the underlying operators [75, 68].

4. **Intelligence as constrained organization**: Emergent capabilities appear not as unconstrained consequences of scale alone, but as behaviors that arise within boundaries imposed by architecture, optimization, memory, and representational structure [68].

These observations do not constitute a physics of intelligence. They instead motivate the possibility of a physics of intelligence: a framework concerned not with consciousness, cognition, or meaning themselves, but with the mathematical organization of high-dimensional systems that encode, transform, and retrieve information.

Whether such a framework ultimately develops into a deeper theoretical foundation remains unknown. The significance of these exploratory pathways lies not in claiming answers, but in identifying productive questions. Across machine learning, signal processing, geometry, dynamical systems,

and mathematical physics, a recurring pattern emerges: complex behavior is often associated with structured transformations acting upon organized representations.

Viewed through this restrained lens, cognition may be interpreted as transformation, information as geometry, latent systems as dynamical spaces, and intelligence as a constrained form of organization operating within those spaces. If future research reveals deeper connections among these perspectives, they will emerge through empirical validation and mathematical demonstration rather than analogy alone.

For Volume I, this appendix serves as a boundary marker rather than a conclusion. Memory retrieval, symbol routing, and latent coordination remain engineering problems. The purpose of these exploratory convergence pathways is simply to suggest that the mathematical languages developed across neighboring disciplines may offer useful tools for understanding why such systems organize, stabilize, and scale as they do.

The Boundary Envelope and Unresolved Pathways

The structural framework outlined in Sections – provides a coordinate system for describing representations, operators, and spectral constraints in large-scale architectures. The aim of this appendix is primarily observational: to highlight the alignment between engineering practice and mathematical structure. It concludes where descriptive insights must give way to derivation. Turning these structural analogies into predictive, testable systems involves addressing several unresolved theoretical challenges. Three of these are especially important.

Non-Asymptotic Spectral Dynamics under Stochastic Optimization

Section characterized token-spectral transitions using operator mechanics and static spectral analysis. In practice, network parameters θ evolve stochastically on the training manifold $\mathcal{M}_\theta$, and the empirical spectral density $\rho_t(\lambda)$ of layer-wise operators exhibits sharp, non-monotonic shifts during training [60, 38].

The assumption that spectral properties vary adiabatically (i.e., that the operator family $\mathcal{K}_\theta$ evolves smoothly under an effective generator) breaks down near loss-surface transitions and gradient plateaus [68]. Consequently, the time evolution of the kernel cannot be written as $\partial_t \mathcal{K}_\theta = \mathcal{L}_{\text{ad}} \mathcal{K}_\theta$ for any fixed operator $\mathcal{L}_{\text{ad}}$ over long horizons. Characterizing the non-equilibrium drift of $\rho_t(\lambda)$ and identifying invariants that remain approximately conserved during optimization remain open problems. Without such characterization, spectral stability guarantees derived from static analysis remain local approximations rather than global bounds [44].

Topological Discontinuities in Dynamic Routing

The geometric formulation in Section assumes that information propagates over a smooth manifold whose metric structure is at least piecewise differentiable. Dynamic routing mechanisms, token-skipping, and mixture-of-experts gating introduce discrete decision boundaries into the computational graph [77, 28]. At these boundaries, the local tangent space is not well defined, and the notion of parallel transport for semantic vectors becomes ambiguous.

Describing information flow across such junctions may require replacing Riemannian assumptions with structures that tolerate singularities, such as piecewise-linear Alexandrov spaces [17], stratified spaces, or sheaf-theoretic models of data [19]. How representational invariants are preserved across discontinuous routing remains unspecified, and whether discrete routing induces effective curvature on the learned manifold is an open question.

Representational Entropy and Invariant Measures

A foundational question is whether closed latent systems obey a conservation principle analogous to Liouville's theorem in statistical mechanics [6]. If architectural bottlenecks compress representational volume, information lost to contraction must be accounted for by an increase in some compensating quantity: geometric complexity, curvature, or effective dimensionality.

Defining a scalar "representational entropy" that is invariant under bijective coordinate transformations of the latent space and that decreases monotonically under contractive operations would be a necessary component of any thermodynamics of representation learning [7]. No such quantity with those properties is currently known. Related measures (effective rank [73], intrinsic dimension [5], and information-theoretic capacity [83]) capture aspects of the problem but do not simultaneously satisfy full coordinate invariance and conservation.

The progression from X.1 through X.7 provides a descriptive language for manifolds, coordinates, operators, geometry, and constraints. X.8 marks the limit of that language. The three problems above are not engineering gaps. They are mathematical gaps: they concern the evolution, topology, and invariants of high-dimensional operators under stochastic, discrete, and contractive dynamics.

Resolving these questions would strengthen the transition from descriptive analogy toward predictive theory. Until then, the framework developed throughout this appendix should be interpreted as a coordinate system for organizing observations rather than a formal predictive calculus. Its purpose is not to establish a unified theory of intelligence but to identify mathematical structures, recurring constraints, and open problems that recur across representation learning, operator theory, geometry, and dynamical systems.

The boundary envelope is therefore explicit. The unresolved pathways are named. Future progress depends not upon extending the analogies presented here, but upon determining whether they can be transformed into empirically validated and mathematically rigorous principles.

Epistemic Taxonomy and Research Frontiers

To ensure that the descriptive insights and unresolved questions developed throughout Sections through remain empirically grounded, this section consolidates the framework into an epistemic taxonomy. The objective is to distinguish observed phenomena from their mathematical interpretations while making explicit the limitations and unresolved questions associated with each perspective.

Table 5 organizes the progression of this appendix using a disciplined hierarchy: **Phenomenological Observation** → **Structural Interpretation** → **Epistemic Limitation** → **Known Unknown**. This format follows the principle that architectural inspiration must precede, and be separable from, mathematical validation [13].

This formal categorization establishes the empirical boundaries of Volume I. By separating observed structural regularities from their speculative mathematical languages, the framework functions as a coordinate system for organizing research questions. Each row maps a documented phenomenon to a bounded interpretation and an explicit limit.

Future verification requires converting each *Known Unknown* into a testable hypothesis within a specific domain: non-asymptotic random matrix theory for spectral drift, discrete differential geometry for routing singularities, and information geometry for representational invariants. This taxonomy, therefore, provides baseline criteria against which any future predictive calculus of representation must be evaluated.

The appendix, therefore, concludes not with a unifying theory, but with a structured research agenda. The convergence pathways traced throughout Sections X.1–X.8 have been organized into explicit empirical observations, bounded interpretations, and falsifiable open questions. Whether these pathways ultimately converge toward a predictive calculus of representation remains unknown. What can be stated with confidence is that the relevant questions are now sufficiently well-defined to support systematic investigation rather than reliance on analogy alone.

Table 5: Epistemic Taxonomy of Latent Operator Mechanics

Phenomenological Observation	**Structural Interpretation**	**Epistemic Limitation**	**Identified Known Unknown**
Layer-wise scaling produces predictable power-law shifts in empirical singular value spectra [60, 61].	token-spectral transitions can be described as localized operator bounds in the transform domain, consistent with Section .	Static spectral analysis does not account for non-equilibrium parameter updates during stochastic gradient descent [68].	The parameter-dependent generator driving non-asymptotic spectral drift, $\partial_t \mathcal{K}_\theta$, remains unspecified [38].
Representational alignment across deep layers preserves global semantic topology as measured by linear probe or CKA metrics [48, 70].	Latent features may organize along continuous, non-Euclidean submanifolds or fiber bundles as suggested in Section .	Empirical manifolds are finite-dimensional, sampled discretely, and sensitive to tokenization and initialization [5].	A coordinate-invariant measure of representational entropy that remains conserved under contractive maps is not known [7].
Sparse routing and mixture-of-experts architectures exhibit localized instability at gating boundaries [77, 28].	Discrete decision boundaries may be interpreted as geometric singularities in phase routing space, per Section .	Classical Riemannian metrics $g_{\mu\nu}$ become ill-defined at routing junctions; parallel transport of semantic vectors lacks a canonical definition.	The applicability of piecewise-linear Alexandrov spaces [17] or sheaf-theoretic models [19] to discontinuous transport is untested.
Network compression and distillation preserve downstream performance despite substantial capacity reduction [36, 30].	High-dimensional representation systems appear to scale via structural optimization constraints, not parameter count alone, as discussed in Section .	Analogy-driven frameworks do not constitute mathematical proofs and do not imply physical substrates, per Section .	The mechanism that balances information loss with localized curvature or effective dimension during compression remains unspecified [73].

Bibliography

[1] John R. Anderson. *How Can the Human Mind Occur in the Physical Universe?* Oxford University Press, 2007.

[2] Anonymous. The incident response LLM manifesto: Technical specification for automated cybersecurity suppression spaces. Evidentiary Appendix A, proving the System Response Ratio (R_s) improvements, 2026.

[3] Anonymous. The realignment blueprint: Formalizing the triple operator system $\mathcal{A} = (m, s, r)$. Part of The Latent Cognition Trilogy, 2026.

[4] Anonymous. Volume I: Memory fields — differentiable latent indexing inside large language models. Part of The Latent Cognition Trilogy, 2026.

[5] Fabio Anselmi et al. On the intrinsic dimensionality of image representations. *Neural Networks*, 2019.

[6] Vladimir I Arnold. *Mathematical Methods of Classical Mechanics*. Springer, 2nd edition, 1989.

[7] John C Baez and Tobias Fritz. A categorical framework for entropy. *Theory and Applications of Categories*, 25:136–194, 2011.

[8] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. *International Conference on Learning Representations (ICLR)*, 2015.

[9] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. *International Conference on Learning Representations (ICLR)*, 2015.

[10] Yoshua Bengio, Aaron Courville, and Pascal Vincent. *Representation Learning: A Review and New Perspectives*, volume 35. Springer, 2013.

[11] Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. Curriculum learning. In *Proceedings of the 26th International Conference on Machine Learning*, pages 41–48, 2009.

[12] Salomon Bochner. *Lectures on Fourier Integrals*. Princeton University Press, 1959.

[13] George E P Box. Science and statistics. *Journal of the American Statistical Association*, 71(356):791–799, 1976.

[14] Ronald N Bracewell. *The Fourier Transform and Its Applications*. McGraw-Hill, 3rd edition, 2000.

[15] Michael M Bronstein, Joan Bruna, Taco Cohen, and Petar Veličković. *Geometric Deep Learning: Grids, Groups, Graphs, Geodesics, and Gauges*. arXiv preprint arXiv:2104.13478, 2021.

[16] Aydar Bulatov, Yuri Kuratov, and Mikhail Burtsev. Recurrent memory transformer. *Advances in Neural Information Processing Systems*, 35:11079–11091, 2022.

[17] Dmitri Burago, Yuri Burago, and Sergei Ivanov. *A Course in Metric Geometry*. American Mathematical Society, 2001.

[18] Fan R K Chung. *Spectral Graph Theory*. American Mathematical Society, 1997.

[19] Justin Michael Curry. Sheaves, cosheaves and applications. *arXiv preprint arXiv:1303.3255*, 2014.

[20] Lokenath Debnath and Dambaru Bhatta. *Integral Transforms and Their Applications*. CRC Press, 3rd edition, 2012.

[21] Michaël Defferrard, Xavier Bresson, and Pierre Vandergheynst. Convolutional neural networks on graphs with fast localized spectral filtering. In *Advances in Neural Information Processing Systems*, 2016.

[22] Yihe Dong, Jean-Baptiste Cordonnier, and Andreas Loukas. Attention is not all you need: Pure attention loses rank doubly exponentially with depth. *International Conference on Machine Learning*, 2021.

[23] Nan Du, Yanping Huang, Andrew M. Dai, Simon Tong, Dmitry Lepikhin, Yuanzhong Xu, Maxim Krikun, Yanqi Zhou, Adams Wei Yu,

Orhan Firat, Barret Zoph, Liam Fedus, Maarten Bosma, Zongwei Zhou, Tao Wang, Yu Emma Wang, Kellie Webster, Marie Pellat, Kevin Robinson, Kathleen Meier-Hellstern, Toju Duke, Lucas Dixon, Kun Zhang, Quoc V. Le, Yonghui Wu, Zhifeng Chen, and Claire Cui. GLaM: Efficient scaling of language models with mixture-of-experts. In *Proceedings of the 39th International Conference on Machine Learning*, pages 5547–5569, 2022.

[24] Phil Dyke. *An Introduction to Laplace Transforms and Fourier Series*. Springer, 2nd edition, 2014.

[25] Harold M Edwards. *Riemann's Zeta Function*. Academic Press, 1974.

[26] Nelson Elhage, Neel Nanda, Catherine Olsson, et al. A mathematical framework for transformer circuits. *Transformer Circuits Thread*, 2021.

[27] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.

[28] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.

[29] Jakob Foerster, Ioannis Alexandros Assael, Nando de Freitas, and Shimon Whiteson. Learning to communicate with deep multi-agent reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 29, 2016.

[30] Jonathan Frankle and Michael Carbin. The lottery ticket hypothesis: Finding sparse, trainable neural networks. *International Conference on Learning Representations*, 2018.

[31] Daniel Gabbay et al. Spectral analysis of transformer language models. *arXiv preprint arXiv:2106.02672*, 2021.

[32] Trevor Gale, Deepak Narayanan, Cliff Young, and Matei Zaharia. MegaBlocks: Efficient sparse training with mixture-of-experts. In *Proceedings of Machine Learning and Systems*, volume 5, pages 288–304, 2023.

[33] Alex Graves, Greg Wayne, and Ivo Danihelka. Neural Turing Machines. *arXiv preprint arXiv:1410.5401*, 2014.

[34] Alex Graves, Greg Wayne, Malcolm Reynolds, Tim Harley, Ivo Danihelka, Agnieszka Grabska-Barwińska, Edward Grefenstette, Tiago Ramalho, John Agapiou, Adrià Puigdomènech Badia, et al. Hybrid computing using a neural network with dyanmic external memory. *Nature*, 538(7626):471–476, 2016.

[35] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016.

[36] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.

[37] Benedict Irwin. Mellin-laplace-fourier transform and the connection to riemann zeta zeroes. *Authorea Preprints*, March 2026. Version 1.

[38] Stanislaw Jastrzebski, Zachary Kenton, Nicolas Ballas, Asja Fischer, Yoshua Bengio, and Amos Storkey. The break-even point on optimization trajectories of deep neural networks. *International Conference on Learning Representations*, 2020.

[39] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. Product quantization for nearest neighbor search. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 33(1):117–128, 2011.

[40] Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, Gianna Lengyel, Guillaume Bour, Guillaume Lample, Lélio Renard Lavaud, Lucile Saulnier, Marie-Anne Lachaux, Pierre Stock, Sandeep Subramanian, Sophia Yang, Szymon Antoniak, Teven Le Scao, Théophile Gervet, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. Mixtral of experts. *arXiv preprint arXiv:2401.04088*, 2024.

[41] Jeff Johnson, Matthijs racing Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2021.

[42] Herve Jégou, Matthijs Douze, and Cordelia Schmid. Product quantization for nearest neighbor search. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 33(1):117–128, 2011.

[43] Tosio Kato. *Perturbation Theory for Linear Operators*. Springer, 2nd edition, 1995.

[44] Kenji Kawaguchi. Deep learning without poor local minima. In *Advances in Neural Information Processing Systems*, volume 29, 2016.

[45] Amirhossein Kazemnejad et al. The impact of positional encoding on length generalization in transformers. *Advances in Neural Information Processing Systems*, 36, 2023.

[46] Hassan K Khalil. *Nonlinear Systems*. Prentice Hall, 3rd edition, 2002.

[47] Urvashi Khandelwal, Omer Levy, Dan Jurafsky, Zettlemoyer Luke, and Kevin Clark. Generalization through memorization: Nearest neighbor language models. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2020.

[48] Simon Kornblith, Mohammad Norouzi, Honglak Lee, and Geoffrey Hinton. Similarity of neural network representations revisited. *International Conference on Machine Learning*, 2019.

[49] Anders Krogh and John A Hertz. A simple weight decay can improve generalization. *Advances in Neural Information Processing Systems*, 1991.

[50] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, pages 611–626, 2023.

[51] John E. Laird. *The Soar Cognitive Architecture*. MIT Press, 2012.

[52] Brenden M. Lake and Marco Baroni. Generalization without systematicity: On the compositional skills of sequence-to-sequence recurrent networks. In *Proceedings of the 35th International Conference on Machine Learning*, pages 2873–2882, 2018.

[53] Peter D Lax. *Functional Analysis*. Wiley-Interscience, 2002.

[54] Yann LeCun. A path towards autonomous machine intelligence, 2022. Version 0.9.2.

[55] Dmitry Lepikhin, Hyoukand Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. GShard: Scaling giant models with conditional computation and automatic sharding. *arXiv preprint arXiv:2006.16668*, 2020.

[56] Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, et al. Gshard: Scaling giant models with conditional computation and automatic sharding. *International Conference on Learning Representations*, 2021.

[57] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Lewis, Sebastian Riedel, and Tim Rocktäschel. Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33:9459–9474, 2020.

[58] Zongyi Li et al. Neural operator: Graph kernel network for partial differential equations. *arXiv preprint arXiv:2003.03485*, 2020.

[59] Stéphane Mallat. *Group Invariant Scattering*, volume 65. Prentice Hall, 2012.

[60] Charles H Martin and Michael W Mahoney. Rethinking generalization requires revisiting old ideas: Statistical mechanics approaches and complex learning behavior. *arXiv preprint arXiv:1710.09553*, 2017.

[61] Charles H Martin and Michael W Mahoney. Implicit self-regularization in deep neural networks: Evidence from random matrix theory and implications for learning. *Journal of Machine Learning Research*, 22(165):1–73, 2021.

[62] Takeru Miyato, Toshiki Kataoka, Masanori Koyama, and Yuichi Yoshida. Spectral normalization for generative adversarial networks. *International Conference on Learning Representations*, 2018.

[63] Tsendsuren Munkhdalai and Hong Yu. Neural semantic encoders. *Association for Computational Linguistics (ACL)*, pages 397–407, 2017.

[64] Mikio Nakahara. *Geometry, Topology and Physics*. CRC Press, 2nd edition, 2003.

[65] Behnam Neyshabur, Ryota Tomioka, and Nathan Srebro. Norm-based capacity control in neural networks. *Conference on Learning Theory*, pages 1376–1401, 2015.

[66] Alan V Oppenheim, Alan S Willsky, and S Hamid Nawab. *Signals and Systems*. Prentice Hall, 2nd edition, 1996.

[67] Kiho Park et al. The geometry of concepts: Sparse autoencoder feature structure. *arXiv preprint arXiv:2406.11901*, 2024.

[68] Jeffrey Pennington, Samuel Schoenholz, and Surya Ganguli. Nonlinear random matrix theory for deep learning. In *Advances in Neural Information Processing Systems*, 2017.

[69] Yingqi Qu, Yuchen Ding, Jing Liu, Kai Liu, Ruiyang Ren, Wayne Pan, Hua Wu, and Haifeng Wang. RocketQA: An optimized training approach to dense passage retrieval for open-domain question answering. *Conference of the National Chapter of the Association for Computational Linguistics (NAACL)*, 2021.

[70] Maithra Raghu, Thomas Unterthiner, Simon Kornblith, Chiyuan Zhang, and Alexey Dosovitskiy. Do vision transformers see like convolutional neural networks? In *Advances in Neural Information Processing Systems*, 2021.

[71] Michael Reed and Barry Simon. *Methods of Modern Mathematical Physics I: Functional Analysis*. Academic Press, revised edition, 1980.

[72] Anna Rogers, Olga Kovaleva, and Anna Rumshisky. A primer in BERTology: What we know about how BERT works. *Transactions of the Association for Computational Linguistics*, 8:842–866, 2020.

[73] Oindrila Roy and Martin Vetterli. On the effectiveness of low-rank matrix approximations in deep neural networks. *International Conference on Machine Learning*, 2019.

[74] Adam Santoro, Sergey Bartunov, Matthew Botvinick, Daan Wierstra, and Timothy Lillicrap. Meta-learning with memory-augmented neural networks. *International Conference on Machine Learning (ICML)*, pages 1842–1850, 2016.

[75] Benjamin Scellier and Yoshua Bengio. Equilibrium propagation: Bridging the gap between energy-based models and backpropagation. *Frontiers in Computational Neuroscience*, 11:24, 2017.

[76] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations*, 2017.

[77] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *International Conference on Learning Representations*, 2017.

[78] Elias M Stein and Guido Weiss. *Introduction to Fourier Analysis on Euclidean Spaces*. Princeton University Press, 1971.

[79] Ron Sun. Desiderata for cognitive architectures. *Philosophical Psychology*, 17(3):327–353, 2004.

[80] Matthew Tancik et al. Fourier features let networks learn high frequency functions in low dimensional domains. *Advances in Neural Information Processing Systems*, 33, 2020.

[81] Naftali Tishby, Fernando C. Pereira, and William Bialek. The information bottleneck method. *arXiv preprint physics/0004057*, 2000.

[82] Naftali Tishby and Noga Zaslavsky. Deep learning and the information bottleneck principle. *2015 IEEE Information Theory Workshop*, 2015.

[83] Naftali Tishby and Noga Zaslavsky. Deep learning and the information bottleneck principle. *IEEE Information Theory Workshop*, 2015.

[84] Edward Charles Titchmarsh. *Introduction to the Theory of Fourier Integrals*. Oxford University Press, 2nd edition, 1948.

[85] Edward Charles Titchmarsh. *The Theory of the Riemann Zeta-Function*. Oxford University Press, 2nd edition, 1986.

[86] Yao-Hung Hubert Tsai et al. Transformer dissection: A unified understanding of transformer's attention via the lens of kernel. *arXiv preprint arXiv:1908.11775*, 2019.

[87] Aaron van den Oord, Oriol Vinyals, and Koray Kavukcuoglu. Neural discrete representation learning. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[88] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in Neural Information Processing Systems*, 30:5998–6008, 2017.

[89] Shida Wang et al. On the expressive power of transformers. *arXiv preprint arXiv:2202.05293*, 2022.

[90] Yuhuai Wu, Markus Rahnama, Angela Fan, Sainbayar Sukhbaatar, Jason Weston, and Armand Joulin. Memorizing transformers. *International Conference on Learning Representations (ICLR)*, 2022.

[91] Ruibin Xiong et al. On layer normalization in the transformer architecture. *International Conference on Learning Representations*, 2020.

[92] Yanqi Zhou, Tao Lei, Hanxiao Liu, Nan Du, Yanping Huang, Vincent Zhao, Andrew M. Dai, Zhifeng Chen, Quoc V. Le, and James Laudon. Mixture-of-experts with expert choice routing. In *Advances in Neural Information Processing Systems*, volume 35, pages 7103–7114, 2022.

[93] Barret Zoph, Irwan Bello, Sameer Kumar, Nan Du, Yanping Huang, Jeff Dean, Noam Shazeer, and William Fedus. ST-MoE: Designing stable and transferable sparse expert models. *arXiv preprint arXiv:2202.08906*, 2022.